CS604 Computer Vision · Team 06

Presentation Study Guide

Understanding our project from zero:
classifying 10,000 kinds of photos with ViT-B/16

What is this guide?
A study book for a reader with no background in machine learning. It follows the slides in order. It starts from the most basic ideas and explains every method, model, figure and formula we use. At the end it gives Q&A practice and a review checklist.

What it is based on
The slides `cs604_team06_slides_v2.pptx` (20 slides: 15 main slides and 5 backup slides), the speaker notes, the course report `report/main.pdf`, and the notes for our method notebooks. Every number in this guide comes from these files.

Presentation: 7 November 2026 (15 minutes of talk and 5 minutes of questions)

Speakers: A slides 1–3 B slides 4–6 C slides 7–9 D slides 10–15 backup slides 16–20

Contents

How to use this guide **1**

0 The basics, from zero **3**

0.1 Image classification: teaching a computer to label photos . . . 3
0.2 A neural network: a function with many ”knobs” . . . 3
0.3 How do we know if the answers are good? Loss and accuracy . . . 4
0.4 How do we turn the knobs? Gradient descent . . . 4
0.5 The danger of memorizing: overfitting . . . 5
0.6 Standing on the shoulders of giants: pretraining and transfer learning . . . 6
0.7 The idea behind ViT: cut the photo into pieces and read them like a sentence . . . 7
0.8 The language of computing power . . . 7
0.9 The three questions we answer, and the map of this guide . . . 7

1 Slide by slide **9**

1.1 Slide 1: Cover . . . 9
1.2 Slide 2: Many classes, few photos, one budget (problem analysis) . . . 11
1.3 Slide 3: Baselines first, then seven methods . . . 13
1.4 Slide 4: How we iterated: one question at a time . . . 15
1.5 Slide 5: Spread small changes over every block (comparing the seven methods) . . . 17
1.6 Slide 6: Linear-probe heads start high, finish low . . . 19
1.7 Slide 7: Our fix: start from class prototypes . . . 21
1.8 Slide 8: Lower training loss, yet lower accuracy (why it works) . . . 23
1.9 Slide 9: Each change, tested one at a time (ablations) . . . 25
1.10 Slide 10: Full fine-tuning trains every weight (final model 1) . . . 26
1.11 Slide 11: LoRA trains 1.38%, then merges (final model 2) . . . 28
1.12 Slide 12: Final results on every metric . . . 30
1.13 Slide 13: Input resolution: the largest gain we found (after the deadline) . . . 32
1.14 Slide 14: Takeaways . . . 34
1.15 Slide 15: Thank you (and who answers which questions) . . . 35
1.16 About the backup slides (slides 16–20) . . . 36
1.17 Slide 16 (backup): Linear probe: train only the head . . . 36
1.18 Slide 17 (backup): Partial fine-tuning: the last K blocks . . . 37

1.19 Slide 18 (backup): BitFit: train only the biases . . . 38
1.20 Slide 19 (backup): Adapter: a bottleneck after each MLP . . . 39
1.21 Slide 20 (backup): AdaptFormer: a scaled side branch . . . 40

2 Methods and models in depth 42
2.1 Our model: ViT-B/16 . . . 43
2.2 The classification head and its three starting points . . . 49
2.3 The seven adaptation methods . . . 54
2.4 How training works . . . 66
2.5 Evaluation: how to tell if a model is good . . . 71
2.6 Test-time augmentation (TTA) and input resolution . . . 73
2.7 Summary tables . . . 75

3 Key terms (glossary) 77
3.1 Data and evaluation . . . 78
3.2 Model and architecture . . . 80
3.3 Training . . . 82
3.4 Fine-tuning methods . . . 84
3.5 Experiments and engineering . . . 86

4 Figures, formulas and architecture diagrams 88
4.1 Slide 2: the data examples and the four numbers . . . 88
4.2 Slide 3: the baseline table and the two families of methods . . . 90
4.3 Slide 4: the timeline of our iterations . . . 92
4.4 Slide 5: the seven-method table and the accuracy-vs-parameters plot . . . 94
4.5 Slide 6: the head initialization curves . . . 97
4.6 Slide 7: the prototype head pipeline and the three numbers . . . 98
4.7 Slide 8: the training loss curves, the three-step explanation and the control experiment . . . 100
4.8 Slide 9: the ablation bar chart . . . 102
4.9 The architecture diagrams one by one (slides 10, 11, 16 to 20) . . . 104
4.10 Slide 12: the final results table . . . 110
4.11 Slide 13: the resolution table . . . 111
4.12 All the formulas . . . 114

5 Presentation Q&A 120
5.1 How to answer questions . . . 121
5.2 Basic concepts . . . 122

5.3 How the techniques work . . . 126
5.4 Choosing the model and the backbone . . . 132
5.5 Comparing the methods . . . 133
5.6 Reading the results . . . 136
5.7 Real-world use . . . 142
5.8 Process, budget and integrity . . . 143
5.9 The 10 questions you are most likely to get . . . 144

6 Final summary **146**
6.1 The core logic of the whole presentation . . . 146
6.2 The 10 most important points . . . 147
6.3 Terms you must understand . . . 148
6.4 The core difference between each model and method . . . 149
6.5 The technical questions the instructor is most likely to ask . . . 150
6.6 What to review before the presentation . . . 151

A Key numbers at a glance **153**
A.1 Task, data and budget . . . 153
A.2 The model (ViT-B/16) . . . 153
A.3 Reference baselines (holdout) . . . 154
A.4 The seven methods (prototype head, 3 epochs, holdout) . . . 154
A.5 How the head starts . . . 154
A.6 Ablations (one change at a time) . . . 155
A.7 The two final models . . . 155
A.8 Resolution (after the deadline, holdout, 3 epochs) . . . 155

How to use this guide

Our project in one paragraph

We take ViT-B/16, a model that has already learned to ”see” from tens of millions of photos. We teach it to recognize 10,000 new classes, with only 45 training photos per class. Each training run may use only about 1.2 GPU-hours. We compared 7 ways to teach it. The most important finding was not which method to pick. It was **how the new classifier (the head) starts**. A head that starts from ”class prototypes” scores 2.1 to 3.1 points higher than the common ”linear-probe” start.

The most important numbers on one page

Item	Numbers and what they mean
The task	10,000 fine-grained classes. Training set: 450,000 photos (45 per class). Validation set: 50,000 photos. Test set: 100,000 photos. We cannot see the validation and test labels; only the CodaBench website can score them.
Our ”practice exam”	We took 2 photos per class out of the training set and never trained on them: 20,000 photos in total. We call them the holdout. Holdout scores and CodaBench scores differ by less than 0.3 points.
The budget	Each run may use 2 hours $\times$ 48 GB of GPU memory $=$ 96 GB·h. On our 80 GB A100 GPU that is 1.2 hours (with $\pm$10% allowed).
The model (backbone)	ViT-B/16, first trained on ImageNet-21k and then on ImageNet-1k. It has 85.8 million parameters. The course recommends it, and we did not change it.
Final model 1: full fine-tuning	Trains every parameter. Validation 76.22%, test **76.09%** (5.4 points above the reference of 70.8%). 1.03 GPU-hours.
Final model 2: LoRA	Trains only 1.375% of the parameters. Validation 71.90%, test **71.67%** (3.6 points above the reference LoRA of 68.3%). 0.54 GPU-hours. It runs as fast as the original model.
Biggest finding	The common ”linear-probe head” actually hurts here. A ”prototype head” adds 2.1 to 3.1 points for all three methods we tested.
Found after the deadline	Making the input photos larger, from 224 to 288 or 320 pixels, adds another 4.4 points on the holdout. It has no test score, so it is not part of our official results.

How to read this guide

- **If you have 1 hour:** read this page, Chapter 6 (Final summary), the questions in Chapter 5 for your own speaker letter, and the number tables in Appendix A.
- **If you have 3 hours:** also read your own slides in Chapter 1, and the LoRA and prototype-head parts of Chapter 2.
- **If you have a whole day:** read in order. Chapter 0 fills in the basics. Chapter 1 explains each slide.

Chapter 2 explains each method in depth. Chapter 3 is the glossary (look up any word you do not know). Chapter 4 explains each figure, formula and architecture diagram. Chapter 5 is the Q&A. Chapter 6 is the summary and review checklist.

The colored boxes in this guide

Key point

Key point: a conclusion or number you must remember.

Everyday analogy

Everyday analogy: a technical idea explained with something from daily life.

Example

Example: a step-by-step calculation with simple numbers.

Easy to get wrong

Easy to get wrong: a common misunderstanding, or a spot where a follow-up question can trip you up.

Formula

Formula: what each symbol means and what the whole formula says.

Answer you can say

Answer you can say: a spoken English answer for the Q&A.

Who presents what

Speaker	Slides	Time (est.)	Answers questions about
A	1–3	about 3:26	the budget and GPU-hours, the choice of backbone, the data and the holdout
B	4–6	about 3:36	method details, parameter counts, latency, LoRA versus the adapters
C	7–9	about 3:47	why the prototype head beats the linear-probe head, and the evidence
D	10–15	about 3:51	full fine-tuning versus PEFT, fairness, limits, resolution

Times assume 130 English words per minute; the whole talk is about 14 minutes 40 seconds. Slides 16–20 are backup slides. Show them only when someone asks about a specific method.

Chapter 0 The basics, from zero

This chapter does not cover the details of our project. It only fills in the most basic ideas. After reading it, you will understand what every slide is talking about. Chapters 2, 3 and 4 explain each idea in more depth.

0.1 Image classification: teaching a computer to label photos

Image classification means: give the computer a photo, and it tells you which class (category) the photo belongs to.

- **Input:** a photo. To a computer, a photo is just a big grid of numbers. Our photos are resized to 224×224 pixels. Each pixel has 3 numbers (red, green and blue). So one photo is $224 \times 224 \times 3 = 150{,}528$ numbers.
- **Output:** the model gives a score to each of the 10,000 classes. The class with the highest score is its answer.

Our dataset is **fine-grained**: the classes look very much alike, for example different kinds of birds, snakes, moths and plants. The class names are hidden (we only know the class numbers). So the model cannot use any text. It must rely on the photos alone.

Everyday analogy

Most people can tell a cat from a dog. That is ordinary classification. A bird expert can tell apart two sparrows that differ only in a small wing pattern. That is fine-grained classification. We want to train an "expert" that knows 10,000 kinds of things.

0.2 A neural network: a function with many "knobs"

A **neural network** is a very complex function. It takes in the numbers of a photo, runs them through many layers of calculations, and gives out 10,000 scores. Each layer contains many numbers that can be adjusted. These numbers are the **parameters**, also called **weights**. Our model, ViT-B/16, has 85,798,656 parameters, about 85.8 million.

Training means adjusting these parameters again and again, so that the model's answers get closer and closer to the right ones.

Everyday analogy

Imagine a sound mixing desk with 85.8 million knobs. At first the sound (the model's answers) is a mess. Training means listening to what sounds wrong and turning the knobs a little at a time, until the sound is good.

0.3 How do we know if the answers are good? Loss and accuracy

0.3.1 Accuracy: the score for people

Top-1 accuracy is the share of photos where the model's first answer is right. For example, if the model gets 14,676 of 20,000 photos right, the accuracy is $14{,}676/20{,}000 = 73.38\%$. All accuracies we report are top-1 unless we say otherwise.

Top-5 accuracy counts a photo as right if the correct class is anywhere in the model's top 5 answers. So it is always higher than top-1.

0.3.2 Loss: the score for the model

Accuracy only says "right" or "wrong". That is too rough: the model cannot learn from it which way to change. So during training we use a **loss function** instead. The loss measures how badly the model is wrong. Smaller is better. We use **cross-entropy**. It looks at one thing only: how much probability the model gives to the **correct** class.

The idea behind cross-entropy

$$\text{loss} = -\log(p_{\text{correct}})$$

p_{correct} is the probability the model gives to "this photo belongs to the correct class" (a number between 0 and 1). log is the natural logarithm.

- The model is sure and right: $p = 0.9$, loss $= -\log 0.9 \approx 0.105$. Very small.
- The model is unsure: $p = 0.1$, loss $= -\log 0.1 \approx 2.30$.
- The model guesses blindly (equal chance for all 10,000 classes): $p = 1/10{,}000$, loss $= \log 10{,}000 \approx 9.21$.

So when the loss falls from 9.21, the model is getting more and more sure of the right answer.

Easy to get wrong

A low training loss does not always mean a good model! The loss is computed on the **training photos**. If the model simply memorizes the training photos, its training loss becomes very low, but it still fails on new photos. Slide 8 shows exactly this.

0.4 How do we turn the knobs? Gradient descent

The **gradient** tells us, for each parameter, which small change makes the loss fall the fastest. **Gradient descent** means taking one small step in that direction, again and again.

- **Learning rate (LR):** how big each step is. Too big and you overshoot; too small and you move very slowly. Our full fine-tuning uses 10^{-4} (0.0001); LoRA uses 10^{-3} (0.001).
- **Batch:** how many photos we use together to compute one gradient. We use 256.
- **Step:** one update of the parameters.
- **Epoch:** one full pass over all the training photos.

Everyday analogy

You walk down a mountain in thick fog. You cannot see the bottom. You can only feel with your feet which way goes down (the gradient), and then take a step (the step size is the learning rate). If your steps are too big, you may fall off a cliff. If they are too small, night falls before you reach the bottom.

How many steps did we train?

Full fine-tuning uses all 450,000 photos, 256 per batch. One epoch is $450{,}000/256 \approx 1{,}757$ steps. Eight epochs are $1{,}757 \times 8 = 14{,}056$ steps, exactly the number on slide 10.
LoRA holds out 2 photos per class, so it trains on $43 \times 10{,}000 = 430{,}000$ photos. One epoch is $430{,}000/256 \approx 1{,}679$ steps. Three epochs are $1{,}679 \times 3 = 5{,}037$ steps, exactly the number on slide 11.

0.5 The danger of memorizing: overfitting

Overfitting means the model does well on the training data but worse on new data. It has **memorized** the training photos instead of learning rules that also work on new photos. Working well on new photos is called **generalization**.

We have only 45 photos per class, but the model has tens of millions of parameters. That makes memorizing very easy. So our rule is: **always judge a model on photos it has never seen, never on the training loss**.

0.5.1 Four sets of photos, each with its own job

Set	Size	Job
Training set	450,000 photos, 45 per class	used to adjust the parameters
Holdout (our own)	20,000 photos, 2 per class	taken out of the training set and never trained on; used to compare methods and choose settings
Validation set	50,000 photos, 5 per class	labels hidden; we see the score only after uploading to CodaBench
Test set	100,000 photos, 10 per class	labels hidden; used for the final ranking; only a few uploads allowed

Everyday analogy

The training set is your homework. The holdout is a practice exam you wrote yourself. The validation set is the school's monthly test: you see your score, but not the answer to each question. The test set is the final exam: you take it only once. If your practice exam scores always match your monthly test scores, your practice exam is good, and you can trust it to make decisions. Our holdout and the CodaBench validation set differ by less than 0.3 points (for example, 72.31% vs. 72.57% for full fine-tuning).

0.6 Standing on the shoulders of giants: pretraining and transfer learning

Training a vision model from scratch needs a huge amount of data. We have only 45 photos per class, which is far too few. So we use a **pretrained** model: a model that has already learned "how to look at photos" on a big dataset.

- Our model was first trained on **ImageNet-21k** (about 14 million photos in over 20,000 classes), and then on **ImageNet-1k** (about 1.28 million photos in 1,000 classes). This is exactly the version the course documents recommend.
- **Transfer learning:** using what a model learned on one task to help with another task.
- **Fine-tuning:** continuing to train a pretrained model on new data, so it fits the new task.

0.6.1 Backbone and head

- **Backbone:** the main body of the model. It turns a photo into a list of meaningful numbers called features (768 numbers in our model). It comes from pretraining.
- **Head:** the last layer. It turns the 768 feature numbers into 10,000 class scores. The original head only knew ImageNet's 1,000 classes, so we **must replace it with a new 10,000-class head**. The new head has $768 \times 10{,}000 + 10{,}000 = 7{,}690{,}000$ parameters.

Everyday analogy

A student who studied art for many years (the backbone) starts a job at a bird research institute. The student does not need to learn "what is color and shape" again. They only need to learn "which features belong to which bird" (the head), and to sharpen their eye a little for bird details (fine-tuning the backbone).

Key point

How to initialize the new head (what its parameters are set to at the start) is the most important finding of our whole project. Slides 6–8 are about it.

0.6.2 Different ways to "teach" the model

Fine-tuning can train every parameter (**full fine-tuning**), or only a very small part of the parameters (**parameter-efficient fine-tuning**, PEFT), such as LoRA. We compared 7 methods in total. Chapter 2 explains each one.

0.7 The idea behind ViT: cut the photo into pieces and read them like a sentence

ViT (Vision Transformer) uses the **Transformer**, a model first built for text, on photos. The idea is:

1. Cut the 224×224 photo into small 16×16 squares called **patches**: $14 \times 14 = 196$ patches.
2. Turn each patch into a list of 768 numbers, called a **token**, like a word in a sentence. Add one special **[CLS] token** whose job is to "sum up the whole photo". That makes 197 tokens.
3. Pass them through 12 Transformer **blocks**. In each block, all the patches "talk to each other" (**self-attention**), and then each one "thinks" on its own (the MLP).
4. At the end, take the 768 numbers of the [CLS] token and give them to the head for scoring.

In "ViT-B/16", B means Base (medium size), and 16 means each patch is 16×16 pixels.

Everyday analogy

Tear a photo into 196 puzzle pieces. Each piece sends a delegate to a meeting. They hold 12 rounds of meetings. In each round, everyone listens to the others ("I see feathers here", "I see a beak here"). At the end, the note taker ([CLS]) writes the summary: "This is a bird with white spots on its wings…".

0.8 The language of computing power

- **GPU:** a chip built to do huge numbers of calculations in parallel. We use an NVIDIA A100 with 80 GB of memory.
- **GPU memory** (VRAM): the GPU's "desk space". During training, the model, the data and the in-between results must all fit on it.
- **GPU-hours:** how many hours of GPU time we used. The course allows at most "2 hours $\times$ 48 GB" per run, that is 96 GB·h. Our GPU has 80 GB, so that is $96/80 = 1.2$ hours.
- **Latency:** how long it takes to process 1 photo, in milliseconds (ms). **Throughput:** how many photos per second the model can process.
- **bf16:** doing the math with 16-bit numbers instead of 32-bit numbers. It uses half the memory and runs faster, and the accuracy barely changes.

0.9 The three questions we answer, and the map of this guide

The report sums up the whole project as three questions:

1. How should we initialize the new 10,000-class head? (slides 6–8)
2. Which adaptation method should we use, and where should its trainable parameters go? (slides 3, 5, 10, 11, and backup slides 16–20)
3. How should we split a fixed budget between training epochs, data and computation at test time? (slides 9, 12, 13)

What to read next

Chapter 1 explains the slides one by one. Chapter 2 explains each method and model in depth. Chapter 3 is the glossary. Chapter 4 explains each figure, formula and architecture diagram. Chapter 5 is the Q&A. Chapter 6 is the summary and review checklist.

Chapter 1 Slide by slide

This chapter explains all 20 slides in order. The main talk is slides 1–15, and the backup slides are 16–20. For each slide we answer the same questions. What does this slide say? What problem does it solve? What are the key ideas? How do you read the figure or table? How does it link to the slides before and after it? Each slide ends with a box called "The one sentence to remember". Its English sentence comes from our speaker notes, so you can say it as it is.

The story line of the whole talk (learn this first, then read each slide)

Analyze the problem (slide 2) → reproduce the baselines and introduce the 7 methods (slide 3) → how we ran our experiments, round by round (slide 4) → compare the 7 methods (slide 5) → find a problem: the linear-probe head (slide 6) → our fix: the prototype head (slide 7) → why it works (slide 8) → why we chose each of the other settings (slide 9) → the two final models (slides 10 and 11) → the final results (slide 12) → a finding after the deadline: resolution (slide 13) → takeaways (slide 14).

1.1 Slide 1: Cover

Speaker: A



Slide 1

What this slide says

It introduces our title, our team, and the scores of our two final models on the **hidden test set**. The title is long, so let us take it apart word by word:

- **Prototype-initialized**: the new classification head starts from "class prototypes" (explained on slide 7). This is the core of our method.
- **adaptation**: changing a pretrained model so that it fits a new task. Fine-tuning is one way to adapt a model.

- **ViT-B/16**: the model we use. It is a Vision Transformer of Base size, and it cuts each image into small patches of 16×16 pixels.
- **10,000-class fine-grained classification**: sorting images into 10,000 classes that look very much alike.

The subtitle has two sentences. The first is: "Seven adaptation methods, one budget, and the one choice that mattered most: how the new head starts." It means that we compared 7 adaptation methods under one fixed budget, and that the single most important choice was how the new head starts. The second sentence gives our scores: 76.09% with full fine-tuning and 71.67% with LoRA (top-1 accuracy on the test set).

What problem it solves

In the first 30 seconds, the audience should learn three things: what we did (compared 7 methods), what our most important finding is (how the head starts), and how good our results are (the two test scores).

Key concepts

The test set: we can submit to it only a few times, and it decides the final ranking. Full fine-tuning and LoRA: explained on slides 10 and 11. The head: the last layer of the model, which gives the class scores.

How it links to the later slides

"Prototype" is explained on slide 7. The "seven methods" are on slides 3 and 5. The scores 76.09% and 71.67% are explained in detail on slide 12.

The one sentence to remember

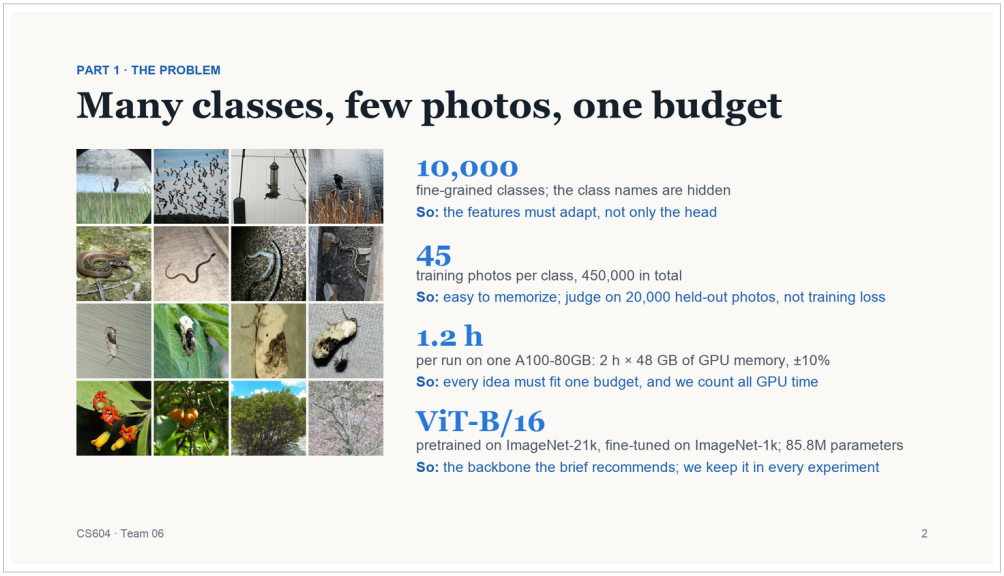
One design choice mattered more than the choice of method: how the new 10,000-class head starts.
In plain words: how the new head starts matters more than which method you pick.

Easy to get wrong

The names on the cover and on the last slide are still the placeholders [Name A] to [Name D]. Replace them with your real names before the talk.

1.2 Slide 2: Many classes, few photos, one budget (problem analysis)

Speaker: A



Slide 2

What this slide says

Before choosing any method, we first ask what makes this task hard. The right side of the slide lists 4 difficulties. Each one is followed by a "So:" line that says what we do about it:

1. **10,000 fine-grained classes, and the class names are hidden.** So: the features themselves must change; changing only the head is not enough. The reason: the pretrained model learned to tell apart broad ImageNet categories (cats, dogs, cars). It has never seen differences this small.
2. **Only 45 training photos per class, 450,000 in total.** So: the model can easily memorize the photos. We therefore judge models on 20,000 held-out photos, not on the training loss.
3. **1.2 hours per training run** (one A100 GPU with 80 GB; the rule is 2 hours $\times$ 48 GB of GPU memory, with $\pm 10\%$ allowed). So: every idea must fit the same budget, and we count all GPU time.
4. **ViT-B/16**: first pretrained on ImageNet-21k, then fine-tuned on ImageNet-1k; 85.8 million parameters. So: we use the backbone that the course brief recommends, and we keep it in every experiment.

What problem it solves, and why we need it

This answers the first point in the instructor's email: "how your team analyzed the problem". Each difficulty leads directly to a design decision, and all later experiments are built around these decisions.

Key concepts

- **Fine-grained**: the classes differ only in small details.
- **Memorization / overfitting**: the model remembers the training photos but cannot handle new ones.
- **Holdout**: from the training set we keep back 2 photos per class, 20,000 in total. They are never used for training, only for scoring. The holdout score is within about 0.3 points of the CodaBench score, so every comparison in the talk uses it.

- **Budget**: $2\text{ h} \times 48\text{ GB} = 96\text{ GB} \cdot \text{h}$. Our GPU has 80 GB, so we get $96/80 = 1.2$ hours. The clock includes extracting features for the head, training, and the holdout evaluation during training.
- **Backbone**: the main body of the model. We kept the version the course recommends. We also checked that the versions trained only on ImageNet-21k are not better (the last row of slide 4).

How to read the figure

On the left are 16 training photos. Each row shows 4 photos of one class. Row 1 is a kind of bird (seen through a scope, as a flock, at a feeder, on reeds). Row 2 is a kind of snake. Row 3 is a kind of moth. Row 4 is a kind of plant (flowers, fruit, a whole shrub, a branch). The figure shows that **photos of the same class can look very different**. One class can be a single bird or a whole flock, a single flower or a whole shrub. With only 45 photos this different, the model finds it hard to learn and easy to memorize.

How it links to the other slides

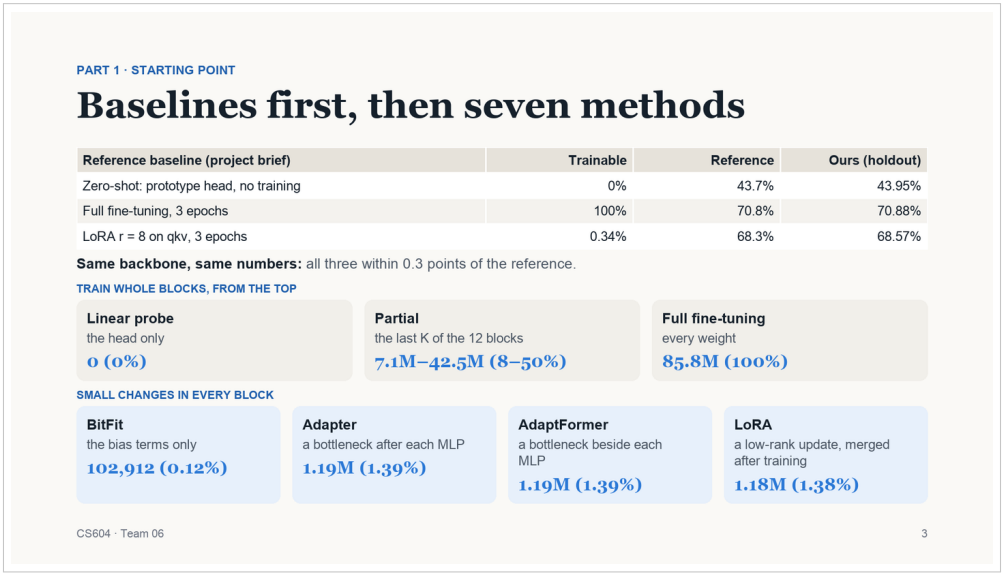
The risk of memorization becomes our most important finding on slides 6–8. The budget decides how many epochs we train (slide 9) and whether we can use larger images (slide 13). The check on the backbone is on slide 4. The next slide first reproduces the baselines, to show that our code is correct.

The one sentence to remember

Before choosing any method, we asked what makes this problem hard, because each difficulty points to a design choice.
In plain words: understand the problem first, because each difficulty tells you what to design.

1.3 Slide 3: Baselines first, then seven methods

Speaker: A



Slide 3

What this slide says

Two things.

First, we reproduce the reference results. The project brief gives three reference numbers (from a PhD student). We ran them again with our own code to see if we get the same results:

Baseline	Share trained	Reference	Ours (holdout)
Zero-shot: prototype head, no training at all	0%	43.7%	43.95%
Full fine-tuning, 3 epochs	100%	70.8%	70.88%
LoRA (r = 8, on qkv only), 3 epochs	0.34%	68.3%	68.57%

All three are within 0.3 points (“Same backbone, same numbers”).

Why is full fine-tuning 70.88% here, but 73.38% on slide 5?

When we reproduced the baselines (in September), full fine-tuning and LoRA still used the **linear-probe head**, the usual choice at that time. Zero-shot used the untrained prototype head. In the comparison of the seven methods on slide 5, every method uses the **prototype head**, so full fine-tuning rises from 70.88% to 73.38%. This gap of 2.5 points is itself the finding of slides 6–8.

Second, we introduce the seven adaptation methods, in two families:

- **Train whole blocks, from the top**: the linear probe (trains only the head; 0 backbone parameters), partial fine-tuning (trains the last K blocks; 7.1 million to 42.5 million parameters, that is 8%–50%), and full fine-tuning (trains every parameter; 85.8 million, 100%).
- **Small changes in every block**: BitFit (trains only the biases; 102,912, 0.12%), the Adapter (adds a small “bottleneck” module after each MLP; 1.19 million, 1.39%), AdaptFormer (puts the bottleneck module beside the MLP; 1.19 million, 1.39%), and LoRA (learns a low-rank update and merges it back into the weights after training; 1.18 million, 1.38%).

What problem it solves, and why we need it

- We **reproduce the baselines** to show that our pipeline is correct. If we could not even reproduce the reference results, any later gain might just be a bug. All three results are within 0.3 points, so our code, data and evaluation are fine.
- We **split the methods into two families** so that slide 5 can answer one question: should the trainable parameters sit together in the top blocks, or be spread over every block?

Key concepts

- **Zero-shot**: no training at all; we predict directly with the prototype head. Here 43.95% is the starting point when nothing is trained.
- **Share of trainable parameters**: the trained parameters as a percentage of the backbone's 85.8 million parameters. The smaller the share, the more "parameter-efficient" the method. Note: every method also trains the 10,000-class head (7.69 million parameters), but this share does not count the head.
- **The seven methods**: see the one-line versions above. Chapter 2 explains each method in detail, and the backup slides 16–20 show the diagrams of five of them.

How it links to the other slides

All seven methods use the same **protocol**: the same prototype head, 3 epochs, the same holdout and the same GPU. So their numbers can be compared directly. The results are on slide 5. Slide 4 first shows how we ran our experiments, step by step.

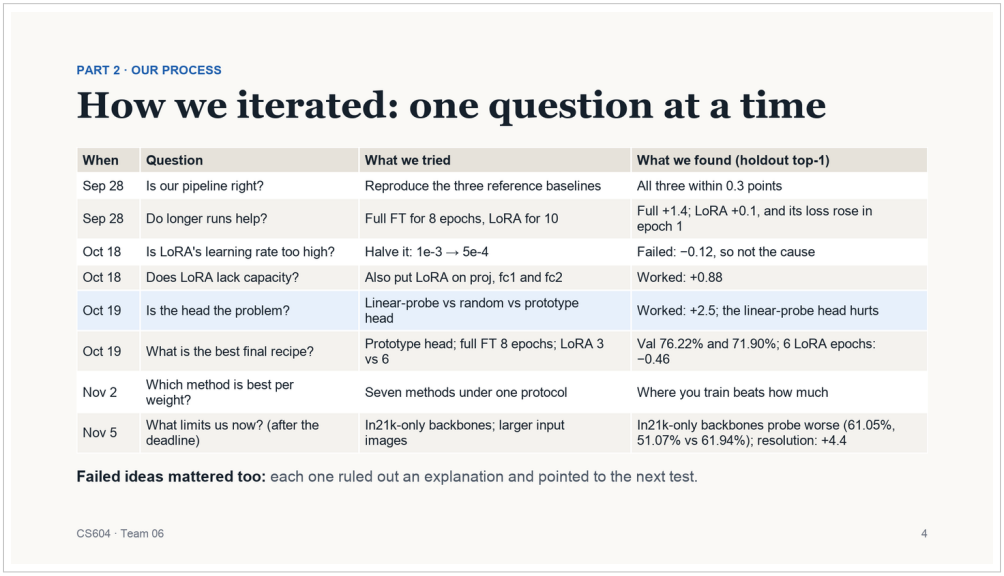
The one sentence to remember

Before trying anything new, we reproduced the three reference numbers in the brief, so our pipeline matches the reference.

In plain words: we first proved that our code gives the same numbers as the reference.

1.4 Slide 4: How we iterated: one question at a time

Speaker: B



PART 2 · OUR PROCESS

How we iterated: one question at a time

When	Question	What we tried	What we found (holdout top-1)
Sep 28	Is our pipeline right?	Reproduce the three reference baselines	All three within 0.3 points
Sep 28	Do longer runs help?	Full FT for 8 epochs, LoRA for 10	Full +1.4; LoRA +0.1, and its loss rose in epoch 1
Oct 18	Is LoRA's learning rate too high?	Halve it: 1e-3 → 5e-4	Failed: −0.12, so not the cause
Oct 18	Does LoRA lack capacity?	Also put LoRA on proj, fc1 and fc2	Worked: +0.88
Oct 19	Is the head the problem?	Linear-probe vs random vs prototype head	Worked: +2.5; the linear-probe head hurts
Oct 19	What is the best final recipe?	Prototype head; full FT 8 epochs; LoRA 3 vs 6	Val 76.22% and 71.90%; 6 LoRA epochs: −0.46
Nov 2	Which method is best per weight?	Seven methods under one protocol	Where you train beats how much
Nov 5	What limits us now? (after the deadline)	In21k-only backbones; larger input images	In21k-only backbones probe worse (61.05%, 51.07% vs 61.94%); resolution: +4.4

Failed ideas mattered too: each one ruled out an explanation and pointed to the next test.

CS604 · Team 06 4

Slide 4

What this slide says

A timeline of 8 rounds of experiments. Each row has four columns: When, Question (what we wanted to know), What we tried (the experiment), and What we found (holdout accuracy). Row by row:

1. **Sep 28**: Is our pipeline right? → Reproduce the three reference baselines → All three are within 0.3 points.
2. **Sep 28**: Do longer runs help? → Full fine-tuning for 8 epochs, LoRA for 10 → Full fine-tuning gains $+1.4$; LoRA gains only $+0.1$, and its loss even went up in epoch 1 (a warning sign!).
3. **Oct 18**: Is LoRA's learning rate too high? → Halve it: $10^{-3} \to 5 \times 10^{-4}$ → **Failed**: -0.12, so this is not the cause.
4. **Oct 18**: Does LoRA lack capacity? → Also put LoRA on proj, fc1 and fc2 → **Worked**: $+0.88$.
5. **Oct 19**: Is the head the problem? → Compare the linear-probe, random and prototype heads → **Worked**: $+2.5$; the linear-probe head hurts. (This row is highlighted on the slide. It is our biggest finding.)
6. **Oct 19**: What is the best final recipe? → Prototype head; full fine-tuning for 8 epochs; LoRA for 3 vs 6 epochs → Validation 76.22% and 71.90%; 6 LoRA epochs gave -0.46.
7. **Nov 2**: Which method is best per weight? → Seven methods under one protocol → Where you train matters more than how much you train.
8. **Nov 5** (after the deadline): What limits us now? → Backbones trained only on ImageNet-21k; larger input images → Those backbones give weaker linear probes (61.05% and 51.07%, against 61.94% for ours); resolution gives $+4.4$.

The line at the bottom says: "Failed ideas mattered too: each one ruled out an explanation and pointed to the next test."

What problem it solves, and why we need it

The instructor's grading includes "how you iterated through experiments", "how you identified and tested new solution ideas", and failed experiments and lessons. This slide answers all three directly. It shows the **scientific method**: notice something strange → suggest a possible cause → design an experiment to test it → use the result to ask the next question.

Key concepts

- **Hypothesis**: a possible explanation for what you see. For example: "LoRA's loss goes up because its learning rate is too high."
- **One change at a time**: change only one setting and keep everything else the same. Then you know that any change in the result comes from that one setting.
- **Failed experiment**: an experiment whose result is not what you expected. It is still useful, because it rules out a wrong explanation.
- **Ablation**: remove or replace one part and measure how much the result changes. This tells you what that part does.

Everyday analogy

It is like a doctor treating a patient. The patient has a fever (LoRA's loss goes up). The doctor first thinks it is a cold and gives cold medicine (halves the learning rate). It does not help, so it is not a cold. The doctor then runs tests (compares three heads) and finally finds the real cause (the linear-probe head). The step that did not help still mattered: it ruled out a wrong diagnosis.

How it links to the other slides

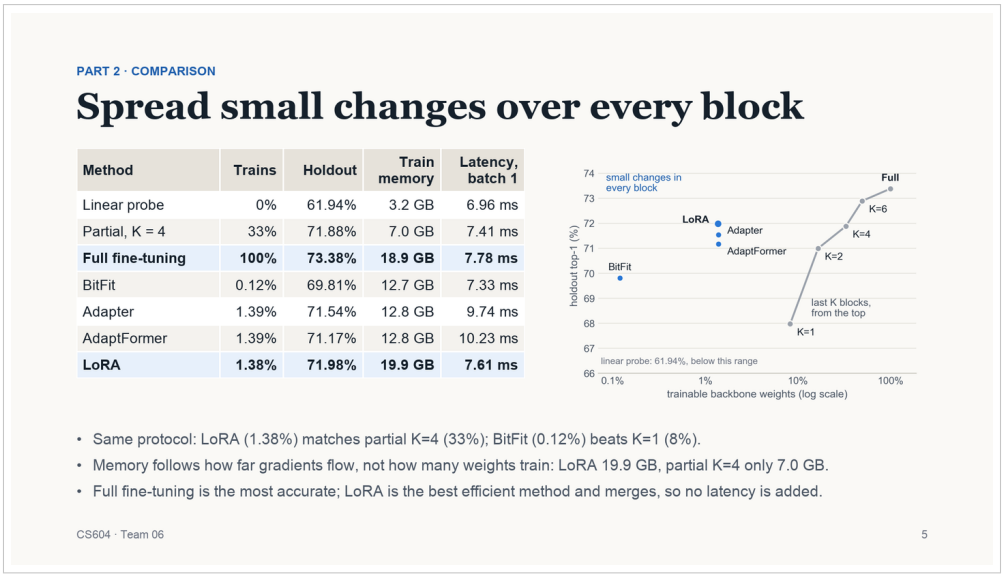
This table is the "table of contents" of the whole talk. Slide 5 expands row 7 (the seven methods). Slides 6–8 expand rows 2, 3 and 5 (the head problem). Slide 9 expands rows 4 and 6 (the effect of each setting). Slide 13 expands the last row (resolution).

The one sentence to remember

One question per round, and each answer chose the next experiment.
In plain words: every result told us what to test next.

1.5 Slide 5: Spread small changes over every block (comparing the seven methods)

Speaker: B



Slide 5

What this slide says

The results of the seven methods under the same protocol. On the left is a table with the method, the share of parameters it trains (Trains), its holdout accuracy, its memory use during training (Train memory), and the time to process one image (Latency, batch 1). On the right is a plot. The x-axis is the share of trainable parameters (on a log scale), and the y-axis is holdout accuracy. Three conclusions at the bottom:

1. Under the same protocol, LoRA (1.38%) is as good as partial fine-tuning with K=4 (33%), and BitFit (0.12%) beats K=1 (8%).
2. Memory depends on how far back the gradients must flow, not on how many weights are trained: LoRA needs 19.9 GB, but partial fine-tuning with K=4 needs only 7.0 GB.
3. Full fine-tuning is the most accurate. LoRA is the best efficient method, and it can be merged into the weights, so it adds no time at inference.

What problem it solves, and why we need it

It answers “which method gives the most accuracy per parameter”. This is the core of what the instructor asked for: a real study of adaptation methods. It also answers the question from slide 3: should the parameters sit in the top blocks or be spread over every block?

Key concepts

- **Where you train matters more than how much**: the grey line (training whole blocks from the top) climbs slowly. Training 1 block (8% of the parameters) reaches only 68%; 6 blocks reach 72.9%; training everything reaches 73.4%. The blue points (small changes in every block) all sit above and to the left of the grey line: they reach high accuracy with very few parameters.
- **Memory and how gradients flow**: to compute gradients during training, the model must keep the results of the forward pass in memory, for every layer that the gradients flow back through. Partial

fine-tuning with K=4 sends gradients back through only 4 blocks. LoRA, BitFit and the adapters have trainable parameters in every block, so the gradients must flow all the way back to block 1. Like full fine-tuning, LoRA must also keep the input of every linear layer (to compute the gradient of its matrix A), plus the results of 96 extra small matrix multiplications. So its memory (19.9 GB) is even higher than that of full fine-tuning (18.9 GB). The extra optimizer state of full fine-tuning is only about 1 GB.

- **Merge**: after training, LoRA can add what it learned back into the original weights. The model then has exactly the original structure, so inference is not slower. The Adapter and AdaptFormer are extra layers that cannot be merged, so each image takes 2–3 ms longer.
- **Log scale**: on the x-axis, the steps from 0.1% to 1% to 10% to 100% have equal length, and each step is 10 times bigger. This lets 0.12% and 100% fit on the same plot.

Easy to get wrong

In the latency column, models with the same structure still vary by about 0.5 ms. For example, the linear probe takes 6.96 ms and full fine-tuning 7.78 ms, but their structure is the same. Only the extra 2–3 ms of the Adapter and AdaptFormer is a real slowdown.

How to read the figure

Find the K=4 point on the grey line (about 33%, 71.88%). Then look straight to the left: LoRA (about 1.4%, 71.98%) sits at almost the same height. Across, there is a 24-fold gap in parameters; up and down, there is almost no gap. This is the evidence that where you train matters more than how much you train. Chapter 4 explains how to read this plot in detail.

How it links to the other slides

It builds on the two families of slide 3. The next slide turns to a problem we found in our experiments: how the head starts.

The one sentence to remember

Where the trainable weights sit matters more than how many there are: a little capacity in every block beats a lot in a few.
In plain words: a few trainable weights in every block beat many weights in a few blocks.

1.6 Slide 6: Linear-probe heads start high, finish low

Speaker: B



Slide 6

What this slide says

A problem we found during our experiments. The two charts show 3 epochs of training: full fine-tuning on the left, and LoRA (on all linear layers) on the right. The x-axis is the epoch. The leftmost point, "head only", is the moment before training starts, when only the head is set up. The y-axis is holdout accuracy. The orange line is the linear-probe head, and the blue line is the prototype head.

- **Before training**: the linear-probe head already has 62.1%, and the prototype head only 44.0%. The linear-probe head is 18 points ahead.
- **After 3 epochs**: full fine-tuning gives 70.9% against 73.4%, and LoRA gives 69.6% against 72.0%. Now the linear-probe head is about 2.5 points *behind*.

Three lines at the bottom:

1. **The recommended recipe, LP-FT** (Kumar et al., ICLR 2022): first train a linear-probe head on frozen features, then fine-tune the whole model.
2. **Starts high, ends low**: 18 points ahead before training, 2.5 points behind after it.
3. **First suspect**: LoRA's learning rate. Halving it changed nothing (-0.12), so we checked the head instead.

What problem it solves, and why we need it

This slide shows how we found the problem. First we saw something strange (LoRA's loss went up in epoch 1). We first suspected the learning rate (wrong), then the head (right). It also shows a surprising fact: **a good start does not mean a good finish**.

Key concepts

- **Linear probe** (LP): freeze the backbone (do not train it), and train only a linear classifier (the head) on the features it produces. It trains very fast.

- **LP-FT**: "Linear Probe, then Fine-Tune". The paper argues that if you fine-tune with a random head, the large gradients at the start "damage" the pretrained features. So you should first train a good head and then fine-tune everything together. Many people use this recipe by default.
- **Head initialization**: the values that the new head's parameters start with. There are three choices: random, linear probe, and prototype (slide 7).
- **The "head only" point**: the accuracy before training, with only the starting head.

Everyday analogy

Two students get the same set of practice questions before an exam. Student A memorizes all the answers first (the linear-probe head already fits the training photos well), so A scores high on the first mock test. Student B only knows roughly what each type of question looks like (the prototype head). When the real studying starts, A thinks "I already know this" and does not work hard (a weak learning signal). In the real exam, A does worse than B.

Easy to get wrong

This does not mean that the LP-FT paper is wrong. LP-FT aims to protect the pretrained features, so that the model stays robust on **other kinds of data**. Our task needs the features to change a lot (slide 2: the features must adapt). Also, our linear probe is trained on **the same training photos**, with only 43–45 photos per class. In this setting it becomes a burden.

How it links to the other slides

It follows the "Is the head the problem?" row of slide 4. The next slide gives our fix, and slide 8 explains why it works.

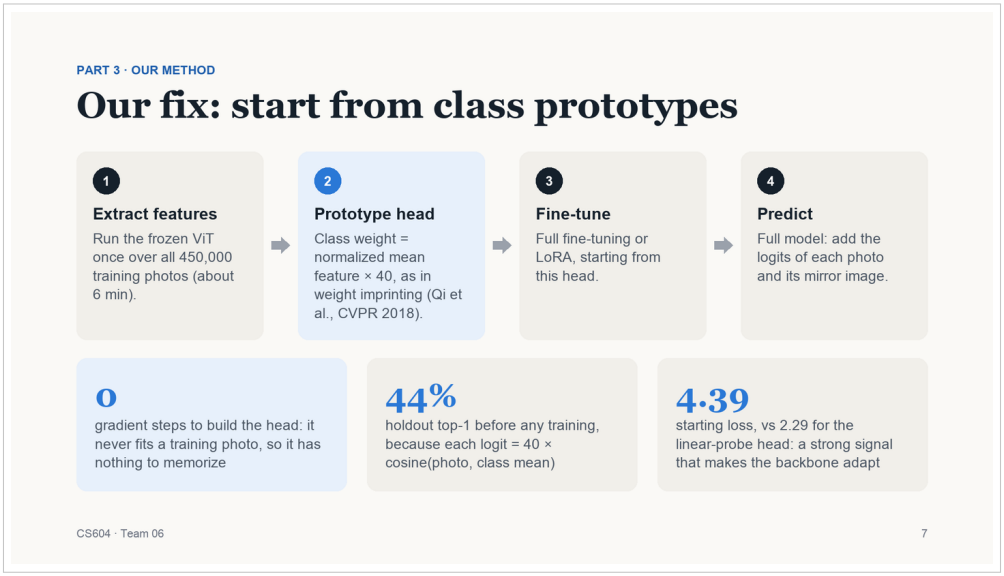
The one sentence to remember

The linear-probe head starts 18 points ahead, but after three epochs it is about 2.5 points behind.

In plain words: the head that starts best finishes worst.

1.7 Slide 7: Our fix: start from class prototypes

Speaker: C



Slide 7

What this slide says

At the top is a 4-step flow:

1. **Extract features**: run the frozen ViT once over all 450,000 training photos (about 6 minutes). Each photo becomes 768 numbers (its features).
2. **Prototype head**: the weight of each class is the "average direction" of the features of its photos (normalize each feature, take the mean, normalize again), times 40. This method is called **weight imprinting** (Qi et al., CVPR 2018).
3. **Fine-tune**: start from this head and run full fine-tuning or LoRA.
4. **Predict**: the full fine-tuning model scores each photo and its mirror image, adds the two scores, and picks the highest class (flip TTA).

Below are three number cards:

- **0**: building the head takes 0 steps of gradient descent. It never "fits" any training photo, so it has nothing to memorize.
- **44%**: the holdout accuracy before any training. This works because each score $= 40\times$ the cosine similarity between the photo's features and the class's average direction.
- **4.39**: the starting loss; the linear-probe head starts at 2.29. A high loss means "there is still a lot to learn", so the backbone gets a strong learning signal.

What problem it solves, and why we need it

The problem with the linear-probe head is that it has already been fitted to the training photos. We need a starting point that is useful but has memorized nothing. It should know roughly what each class looks like (better than random), but none of its parameters should have been tuned on any single training photo.

Key concepts

- **Feature**: the 768 numbers that the backbone turns an image into. Think of them as a "description" of the image.
- **Prototype**: the average of the features of all photos in a class. It stands for the "typical look" of that class.
- **L2 normalization**: scale a list of numbers so that its length is 1. It keeps only the "direction" and ignores the "size".
- **Cosine similarity**: how well two directions agree, from -1 to 1. The closer to 1, the more similar.
- **Temperature (logit scale) of 40**: cosine similarity only ranges from -1 to 1, so the differences between classes are tiny. After softmax, all classes would get almost the same probability. Multiplying by 40 makes the differences bigger. We picked 40 from 5, 10, 15, 20, 30, 40, 60 and 80, choosing the value with the lowest loss on 10,000 training features.
- **Weight imprinting**: we did not invent this method. Our new finding is that, in our setting with many classes and few photos per class, it is a better start than the recommended LP-FT.
- **Flip TTA**: test-time augmentation. The model predicts on the photo and on its mirror image, and combines the two. Chapter 2 explains it in detail.

Everyday analogy

Think of recognizing people. The linear-probe head is like memorizing 45 ID photos one by one. The prototype head is like looking at the 45 photos and remembering "roughly what this person looks like" (an average impression). When a new photo comes in, you compare it with each person's average impression, and the closest one is your answer. The average impression is not a copy of any single photo, so nothing is memorized.

How to compare the three starting losses

These three numbers are the average training loss over the first 100 steps of LoRA training:

- **Random head: 9.28.** Pure guessing (the same score for all 10,000 classes) gives a loss of $\log 10{,}000 \approx 9.21$. The two are almost equal: the random head knows nothing.
- **Prototype head: 4.39.** Much lower than guessing, so it already knows roughly what each class looks like. But it is still far from confident, so there is a lot left to learn, and the gradient signal is strong.
- **Linear-probe head: 2.29.** The lowest, because it was trained on these very training photos and already "knows" them. Its learning signal is the weakest.

Note: training used label smoothing (slide 10), so you cannot turn these losses directly into "the probability of the correct class". Only compare them with each other.

How it links to the other slides

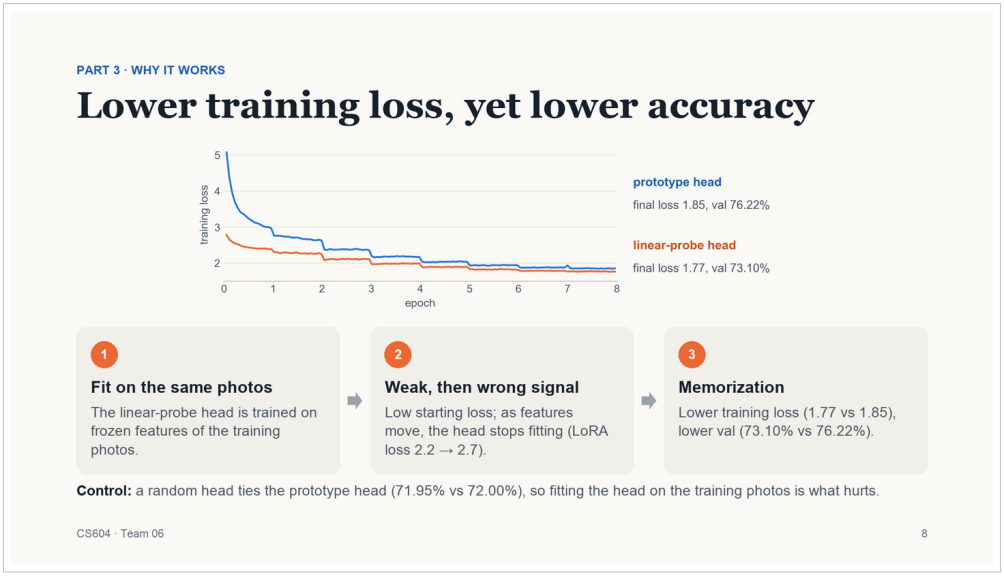
It answers the problem raised on slide 6. The next slide uses the loss curves and a control experiment to show why it works.

The one sentence to remember

The head never fits any training photo, so it has nothing to memorize, yet it already reaches 44% before any training.
In plain words: the prototype head is useful from the start, but it has memorized nothing.

1.8 Slide 8: Lower training loss, yet lower accuracy (why it works)

Speaker: C



Slide 8

What this slide says

At the top are the loss curves of two training runs. Both are full fine-tuning on all data for 8 epochs. The only difference is how the head starts. The orange line (linear-probe head) always has the lower training loss and ends at 1.77; the blue line (prototype head) ends at 1.85. But on the CodaBench validation set, orange scores 73.10% and blue scores 76.22%. **Lower training loss but lower accuracy: that is memorization.**

The three steps in the middle explain why:

1. **Fit on the same photos**: the linear-probe head was trained on the frozen features of the training photos.
2. **Weak, then wrong signal**: its starting loss is low, so the backbone gets only a weak learning signal. And as soon as the backbone starts to change, the features change too, and the fitted head no longer matches them. That is why, with the linear-probe head, LoRA's training loss went *up* in the first epoch (the slide says 2.2 → 2.7; the exact numbers in the report are 2.22 → 2.69).
3. **Memorization**: in the end, the training loss is lower (1.77 vs 1.85), and the validation score is lower too (73.10% vs 76.22%).

The control experiment at the bottom: a random head ties the prototype head (71.95% vs 72.00%). So **the harm comes from fitting the head on the training photos**.

What problem it solves, and why we need it

Saying "the prototype head is better" is not enough. We need to explain **why**, and use an experiment to rule out other explanations. This is what the instructor means by "how you interpreted your results".

Key concepts

- **Training loss vs generalization**: the training loss only shows how well the model "knows" the training photos. It does not show how well the model does on new photos.

- **Learning signal** (the size of the gradient): a larger loss usually means larger gradients, so the backbone changes more. Our task needs the backbone to change (slide 2).
- **Control experiment**: an experiment that changes only one factor, to confirm a cause. The random head was never fitted to the training photos, and neither was the prototype head. If these two tie while the linear-probe head is worse, the problem must be that the head was fitted to the training photos.

Key point

The prototype head is not magic. Its value is that it does not start from zero like the random head (it learns faster: after epoch 1 it leads the random head by 5 points, 63.3% vs 58.4%), and it has not memorized the answers like the linear-probe head.

How to read the figure

Both curves drop suddenly at each epoch boundary and stay fairly flat within an epoch. The report reads this as a sign that the model is memorizing the 45 photos per class: after one full pass over all the photos, it "recognizes" them on the next pass. Chapter 4 has the details.

How it links to the other slides

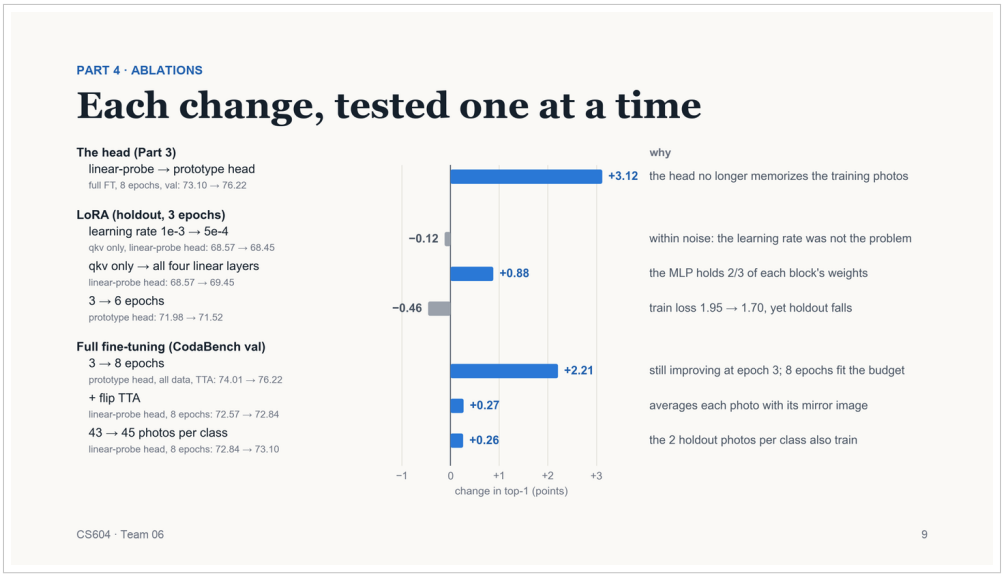
It explains why the method of slide 7 works. The next slide tests every setting of our final recipe in the same "one change at a time" way.

The one sentence to remember

Lower training loss but lower accuracy: that is memorization. A random head ties the prototype head, so the harm comes from fitting the head on the training photos.
In plain words: a head trained on the training photos makes the model memorize instead of learn.

1.9 Slide 9: Each change, tested one at a time (ablations)

Speaker: C



Slide 9

What this slide says

A horizontal bar chart. Each row changes only one setting and keeps everything else the same. The length of the bar shows how many points the accuracy changed. The right column gives the reason.

What changed	Before → after	Change	Reason
Head: linear probe → prototype (full fine-tuning, 8 epochs, validation)	73.10 → 76.22	+3.12	The head no longer memorizes the training photos
LoRA learning rate $10^{-3} \rightarrow 5 \times 10^{-4}$ (holdout)	68.57 → 68.45	−0.12	Within noise, so no real change
LoRA on qkv only → on all four linear layers	68.57 → 69.45	+0.88	The MLP holds 2/3 of each block's parameters
LoRA 3 epochs → 6 epochs	71.98 → 71.52	−0.46	Training loss falls from 1.95 to 1.70, but the holdout score falls too: overfitting
Full fine-tuning 3 → 8 epochs (validation)	74.01 → 76.22	+2.21	Still improving at epoch 3, and 8 epochs fit the budget
Flip TTA (validation)	72.57 → 72.84	+0.27	Judge the photo and its mirror image together
43 → 45 photos per class (validation)	72.84 → 73.10	+0.26	2 more photos per class

What problem it solves, and why we need it

It explains **why each setting of the final recipe is there**, instead of "it felt right" or "someone else used it". This is exactly the interpretation and iteration that the instructor wants to see.

Key concepts

- **Ablation**: change one setting at a time and measure its own effect.
- **Noise**: if you run the same setting again, the score moves a little. Our holdout has 20,000 images, so differences of about 0.3 points can be noise. So -0.12 means "no change", and $+0.27$ and $+0.26$ are at the edge of the noise; treat them with care.
- **Full fine-tuning and LoRA react to longer training in opposite ways**: full fine-tuning gains 2.2 points from 3 to 8 epochs, but LoRA loses 0.46 points from 3 to 6 epochs. Our guess (not yet tested): 87% of the parameters that LoRA trains belong to the head (7.69 million of 8.87 million). LoRA's recipe gives the head a learning rate 10 times higher than full fine-tuning does, a weight decay 500 times smaller, and no drop path. So the head memorizes most easily.

How it links to the other slides

These results set up the two final models on slides 10 and 11: full fine-tuning with 8 epochs, flip TTA and all 45 photos per class; LoRA on all four linear layers, for 3 epochs.

The one sentence to remember

Every row changes one thing and keeps the rest fixed; the right column gives the reason. In plain words: one change per row, with its effect and its reason.

1.10 Slide 10: Full fine-tuning trains every weight (final model 1)

Speaker: D



Slide 10

What this slide says

At the top is the diagram of full fine-tuning. From left to right: the image → cut into patches → 197 tokens → 12 Transformer blocks → the final normalization → the [CLS] feature → the head → 10,000 scores. Blue means trained and grey means frozen. Here everything is blue, because every parameter is trained. Below are three cards:

- **Model**: all 85.8 million parameters of ViT-B/16 are trained; the head maps 768 → 10,000 and has

7.69 million parameters; the head starts from the prototypes $\times$ 40; all 45 photos per class, at 224 pixels; random crop (25%–100% of the area) + horizontal flip.

- **Training**: cross-entropy loss with label smoothing 0.1; AdamW with weight decay 0.05 and gradient clipping at 1.0; learning rate 10^{-4}, multiplied by 0.75 for each block toward the input; drop path 0.1; bf16; 256 images per batch; 8 epochs: 1 warmup epoch, then a cosine curve down to 0.
- **Cost and result**: 1.03 GPU-hours; peak memory 18.9 GB; 14,056 steps; with flip TTA, 14.95 ms per image and 1,919 images per second; validation 76.22%, test 76.09%.

What problem it solves, and why we need it

It describes the final model clearly: which loss, how many parameters, which learning rate, and how much compute. The instructor and classmates often ask "How exactly did you train it?" This slide is the answer.

Key concepts (why we chose each setting)

- **Label smoothing** 0.1: we do not ask the model to be 100% sure of the right answer. This reduces overconfidence and memorization.
- **AdamW**: a common optimizer. Each parameter gets its own step size. The "W" means that weight decay is applied separately from the gradient update.
- **Weight decay** 0.05: at every step, pull every parameter a tiny bit toward 0. This stops the parameters from growing too large, and it helps against overfitting.
- **Gradient clipping** 1.0: if a gradient is too large, shrink it, so that no single step goes too far.
- **Layer-wise learning-rate decay** 0.75: the closer a layer is to the input, the smaller its learning rate. The first block gets only about 3% of the top learning rate ($0.75^{12} \approx 0.03$). The reason: the lower layers learn general features like edges and textures, which need little change. The top layers are more task-specific, so they can change more.
- **Drop path** 0.1: during training, randomly "skip" some blocks, so the model does not depend too much on any one part. It is a form of regularization (a way to prevent overfitting).
- **Warmup + cosine**: the learning rate first rises slowly from 0 to its maximum, then falls slowly to 0 along a cosine curve. Starting slowly keeps the large early gradients from damaging the pretrained features. Ending slowly lets the model settle down.
- **bf16**: compute with 16-bit numbers. It saves memory and runs faster.

Chapter 2, in the section on how training works, explains each of these in more detail, with examples.

How it links to the other slides

The recipe comes from the ablations on slide 9. The next slide is the other final model, LoRA. Slide 12 puts all the numbers of both models together.

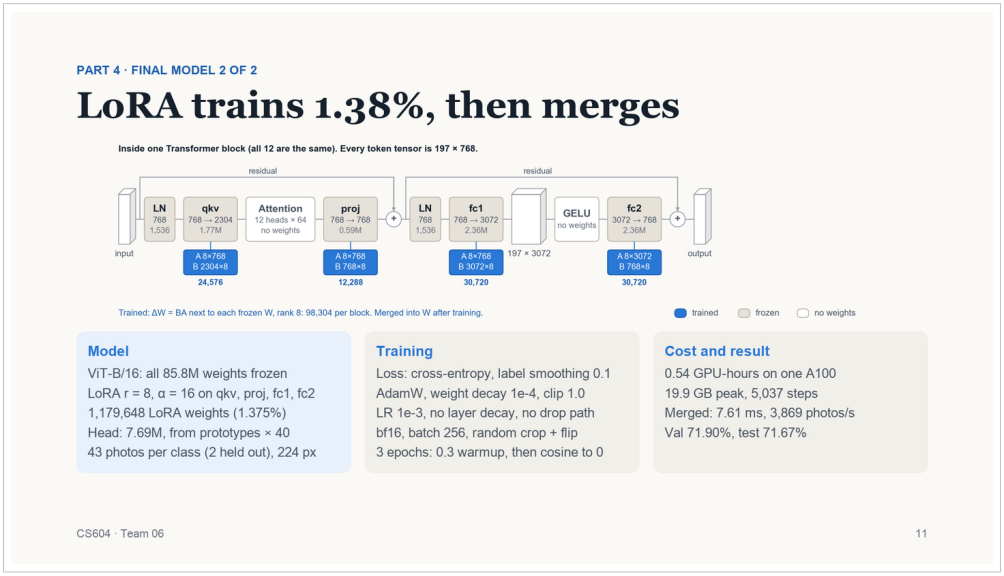
The one sentence to remember

We train all 85.8 million weights plus the new 7.69 million head, which starts from the class prototypes, with a learning rate that shrinks by 0.75 per block toward the input.

In plain words: train everything, start the head from prototypes, and change the lower layers less.

1.11 Slide 11: LoRA trains 1.38%, then merges (final model 2)

Speaker: D



Slide 11

What this slide says

At the top is the inside of **one** Transformer block (all 12 blocks are the same): LayerNorm → qkv → attention → proj → residual add → LayerNorm → fc1 → GELU → fc2 → residual add. All the original grey weights are frozen. Next to each of the four linear layers (qkv, proj, fc1 and fc2) sits a pair of small blue matrices, A and B. Only these are trained. Below are three cards:

- **Model**: all 85.8 million parameters of ViT-B/16 are frozen; LoRA with rank r = 8 and α = 16 on qkv, proj, fc1 and fc2; 1,179,648 LoRA parameters in total (1.375% of the backbone); the head has 7.69 million parameters, starts from the prototypes $\times$ 40, and is also trained; 43 photos per class (2 kept for the holdout), at 224 pixels.
- **Training**: the same cross-entropy loss with label smoothing 0.1; AdamW with weight decay 10^{-4} and gradient clipping at 1.0; learning rate 10^{-3}, with no layer-wise decay and no drop path; bf16; 256 images per batch; random crop + flip; 3 epochs: warmup for the first 0.3 epoch, then a cosine decrease.
- **Cost and result**: 0.54 GPU-hours; peak memory 19.9 GB; 5,037 steps; after merging, 7.61 ms per image and 3,869 images per second; validation 71.90%, test 71.67%.

What problem it solves, and why we need it

With full fine-tuning, every task needs its own full copy of the model (374 MB). With many tasks, storing and deploying them becomes expensive. LoRA trains very few parameters, so each task needs only a small file. After training it can be merged, so inference is as fast as with the original model. The course also asked us to submit a parameter-efficient model.

Key concepts

- **Freeze**: a frozen parameter is not trained and keeps its pretrained value.

- **Low-rank update**: instead of changing the big matrix W directly, learn a "change" $\Delta W = BA$. A is $8\times$ (input size) and B is (output size) $\times 8$, so both are very "thin".
- **Rank r = 8**: the size in the middle. A larger r can express more complex changes, but it also has more parameters.
- α **= 16**: a scaling factor. The actual change is $(\alpha/r) \times BA = 2 \times BA$.
- **B starts at 0**: so at the start of training $BA = 0$, and the model is exactly the pretrained model. Nothing is damaged at the start.
- **Merge**: after training, compute $W' = W + 2BA$, use it in place of W, and throw A and B away. The model then has exactly the original structure, so inference is not slower.

Counting LoRA's parameters

For a linear layer with d_{in} inputs and d_{out} outputs, LoRA has $r \times (d_{\text{in}} + d_{\text{out}})$ parameters:

- qkv ($768 \to 2304$): $8 \times (768 + 2304) = 24{,}576$
- proj ($768 \to 768$): $8 \times (768 + 768) = 12{,}288$
- fc1 ($768 \to 3072$): $8 \times (768 + 3072) = 30{,}720$
- fc2 ($3072 \to 768$): $8 \times (3072 + 768) = 30{,}720$

That is 98,304 per block, and $98{,}304 \times 12 = 1{,}179{,}648$ for all 12 blocks: 1.375% of the backbone's 85.8 million. For comparison: training just the qkv matrix of one block directly would take $768 \times 2304 = 1{,}769{,}472$ parameters. That is more than LoRA uses in the whole model.

Easy to get wrong

LoRA's "1.375%" does not include the head. The head has 7.69 million parameters, so most of what LoRA actually trains is the head (7.69 million of 8.87 million). Also, LoRA does not save memory during training (19.9 GB, more than full fine-tuning). What it saves is **storage** and **optimizer state**.

How it links to the other slides

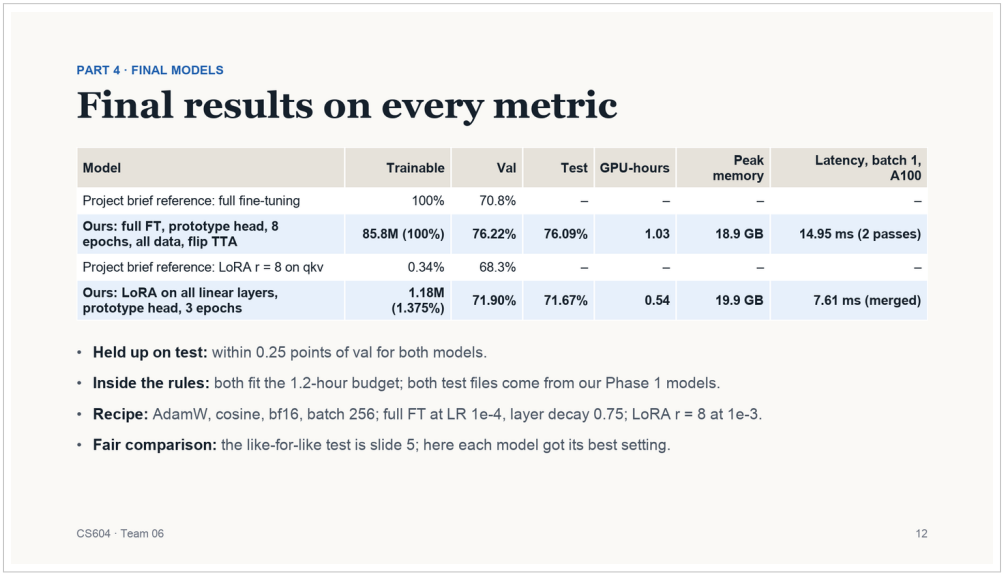
LoRA trains for only 3 epochs because slide 9 showed that 6 epochs are worse. It sits on all four linear layers because slide 9 showed that this adds 0.88 points. The next slide puts all the numbers of both models together.

The one sentence to remember

The backbone stays frozen; rank-8 matrices A and B next to qkv, proj, fc1 and fc2 learn the update, and after training the update is merged, so inference is as fast as the base model.
In plain words: train two thin matrices next to each layer, then fold them back in, so the final model is not slower.

1.12 Slide 12: Final results on every metric

Speaker: D



Slide 12

What this slide says

One summary table with every metric that the project brief asks for: trainable parameters, validation score, test score, GPU-hours, peak memory, and the latency for one image (on an A100).

Model	Trainable	Val	Test	GPU-h	Memory	Latency
Brief reference: full fine-tuning	100%	70.8%	–	–	–	–
Ours: full fine-tuning (prototype head, 8 epochs, all data, flip TTA)	85.8M	76.22%	76.09%	1.03	18.9 GB	14.95 ms
Brief reference: LoRA (qkv only)	0.34%	68.3%	–	–	–	–
Ours: LoRA (four linear layers, prototype head, 3 epochs)	1.18M	71.90%	71.67%	0.54	19.9 GB	7.61 ms

Four lines at the bottom:

1. **Held up on test**: for both models, the test score is within 0.25 points of the validation score.
2. **Inside the rules**: both models fit the 1.2-hour budget, and both test submissions come from the same Phase 1 models.
3. **Recipe**: AdamW, a cosine learning rate, bf16, 256 images per batch; full fine-tuning at learning rate 10^{-4} with layer-wise decay 0.75; LoRA with r = 8 at learning rate 10^{-3}.
4. **Fair comparison**: the like-for-like comparison is on slide 5; here each model uses its own best setting.

What problem it solves, and why we need it

It gives the final answer. It also shows that the results can be trusted (test agrees with validation), that they follow the rules (the budget and the course rules), and that the comparison is fair.

Key concepts

- **+5.4 and +3.6**: $76.22 - 70.8 = 5.42$ and $71.90 - 68.3 = 3.6$. Both are measured against the brief's references.
- **Test agrees with validation**: $76.22 - 76.09 = 0.13$ and $71.90 - 71.67 = 0.23$. This shows that we did not over-tune on the validation set, and that the models behave the same way on new data.
- **14.95 ms (2 passes)**: full fine-tuning uses flip TTA, so each image is processed twice (the original and its mirror image). The latency is about twice that of one pass.
- **7.61 ms (merged)**: LoRA's latency after merging, the same as the original ViT.
- **The Phase 2 rule**: the course requires that the test predictions come from the same model as in Phase 1. We followed this rule.

Easy to get wrong

Do not read 76.22 vs 71.90 on slide 12 as a fair comparison that says "full fine-tuning beats LoRA by 4.3 points". The two models differ in epochs, data and TTA. The fair comparison is on slide 5, under the same conditions: 73.38 vs 71.98 (a gap of 1.4 points).

How it links to the other slides

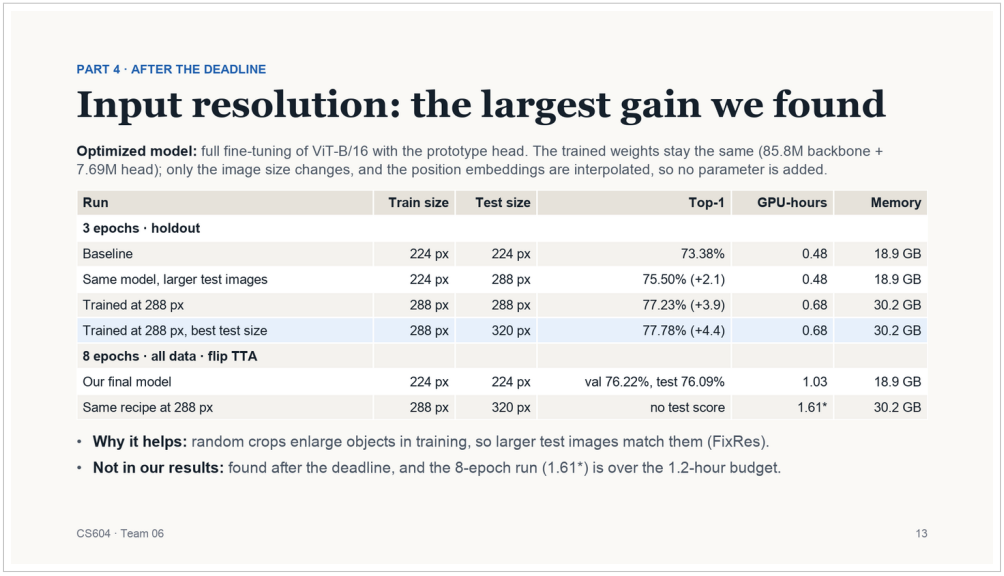
It sums up the two models of slides 10 and 11. The next slide shows one more finding, from after the deadline.

The one sentence to remember

For both models, test is within 0.25 points of val, both fit the budget, and both test files come from our Phase 1 models.
In plain words: the scores held up on the test set, and we followed every rule.

1.13 Slide 13: Input resolution: the largest gain we found (after the deadline)

Speaker: D



Slide 13

What this slide says

An experiment we ran after the deadline: we changed only the size of the input images and measured the effect on accuracy. The top of the slide first says which model we optimized: full fine-tuning of ViT-B/16 with the prototype head. The trained parameters are exactly the same (85.8 million in the backbone + 7.69 million in the head). Only the images get bigger. The position embeddings are interpolated to fit the new size, so no parameter is added.

The table (the upper part is all 3 epochs, on the holdout):

- Baseline: train at 224, test at 224: 73.38% (0.48 GPU-hours, 18.9 GB).
- The same model, with only the test images enlarged to 288: 75.50% (+2.1), with no new training.
- Train at 288, test at 288: 77.23% (+3.9; 0.68 GPU-hours, 30.2 GB).
- Train at 288, test at 320: 77.78% (+4.4), the best.

The lower part (8 epochs, all data, flip TTA) shows our final model (224/224, validation 76.22%, test 76.09%, 1.03 GPU-hours). The same recipe, trained at 288 and tested at 320, has **no test score**. It also needs 1.61 GPU-hours, which is over the budget (the 1.61* in the table).

Two lines at the bottom:

1. **Why it helps**: random crops during training "zoom in" on objects, so larger test images make the objects about the same size as in training (FixRes).
2. **Why it is not in our official results**: we found it after the deadline, and the 8-epoch version is over the 1.2-hour budget.

What problem it solves, and why we need it

It answers the last row of slide 4: "What limits us now?" It also shows honesty. The gain is large, but we ran it after the deadline and only have holdout scores, so we do not present it as an official result.

Key concepts

- **Resolution and the number of tokens**: the image is cut into patches of 16×16 pixels. At 224 pixels that is $14 \times 14 = 196$ patches; with [CLS], 197 tokens. At 288 pixels it is $18 \times 18 = 324$ patches, so 325 tokens. At 320 pixels it is $20 \times 20 = 400$ patches, so 401 tokens.
- **Why compute and memory grow**: going from 197 to 325 tokens makes most of the computation about 1.7 times larger. The attention part grows with the square of the number of tokens, so it grows even more. That is why memory goes from 18.9 GB to 30.2 GB.
- **Interpolating the position embeddings**: the position embeddings form a 14×14 grid. With bigger images the grid becomes 18×18. We "stretch" the old position embeddings onto the new grid by interpolation, like resizing a picture. No new parameters are needed.
- **FixRes** (Touvron et al., 2019): during training, the random crop takes a small part of the image and enlarges it to 224, so objects look bigger than they really are. At test time, the whole image is shrunk to 224, so objects look smaller. If we enlarge the test images a little, the objects get closer to the size the model saw in training.

Everyday analogy

Suppose you always practice recognizing insects with a magnifying glass (the random crop zooms in during training). But in the exam you get a small photo of the whole scene (the test image shrunk to 224). If the exam photo were enlarged a little, you would recognize the insects better.

Easy to get wrong

This result has **no test score**, and it is not one of our official results. If you are asked about it, say clearly that it is a finding from after the deadline, and that we only measured it on the holdout. The 3-epoch version at 288 fits the budget (0.68 hours), but the 8-epoch final recipe at 288 needs 1.61 hours, which is over the budget.

How it links to the other slides

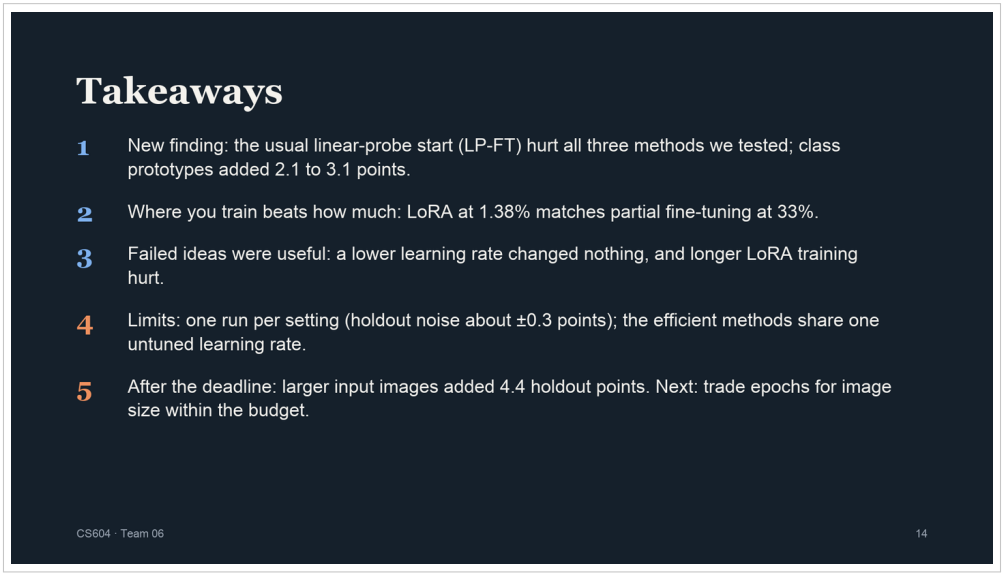
It is the evidence for point 5 on slide 14, the lesson from after the deadline. The next step is to trade a few epochs for larger images, inside the budget.

The one sentence to remember

Training at 288 and testing at 320 gives 77.78%, 4.4 points above the baseline, inside the budget, but it was found after the deadline, so it is not part of our results.
In plain words: bigger images helped a lot, but we found this too late to count it.

1.14 Slide 14: Takeaways

Speaker: D



Slide 14

What this slide says

Five conclusions. The first three (blue numbers) are findings; the last two (orange numbers) are limits and next steps:

1. **New finding**: the usual linear-probe start (LP-FT) hurt all three methods we tested; class prototypes added 2.1 to 3.1 points. (Evidence: slides 6–8)
2. **Where you train beats how much**: LoRA with 1.38% of the parameters matches partial fine-tuning with 33%. (Evidence: slide 5)
3. **Failed ideas were useful**: a lower learning rate changed nothing, and longer LoRA training made it worse. (Evidence: slides 4 and 9)
4. **Limits**: we ran each setting only once (the holdout noise is about ± 0.3 points), and the efficient methods share one learning rate that we did not tune for each method.
5. **After the deadline**: larger input images added 4.4 points on the holdout. Next: trade epochs for image size within the budget. (Evidence: slide 13)

What problem it solves, and why we need it

It leaves the audience with the most important points. Points 4 and 5 answer the instructor's request for lessons and an honest record: it is better to name your limits yourself than to have someone else point them out.

Key concepts

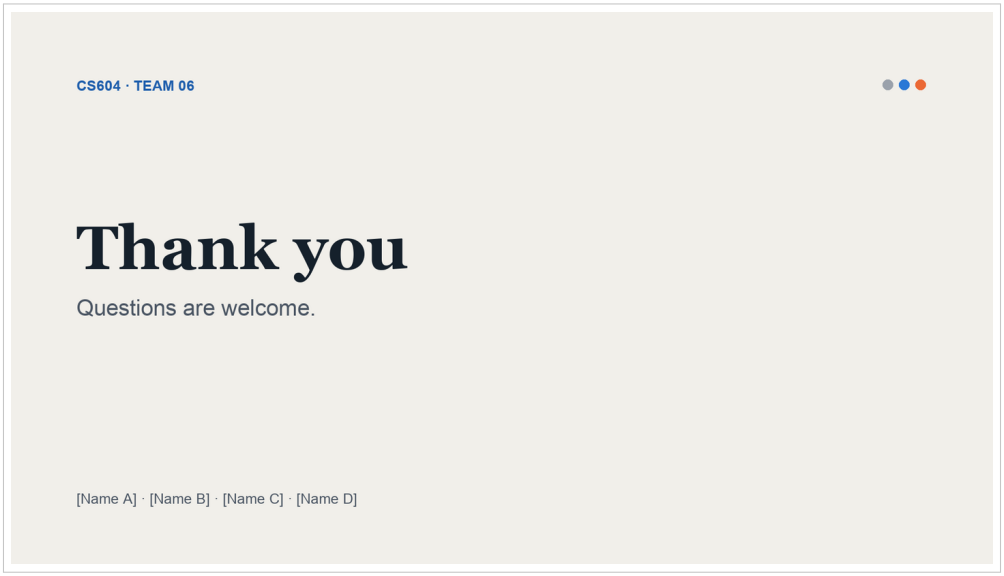
"All three methods" means full fine-tuning, LoRA and AdaptFormer: for these three, we compared the linear-probe head with the prototype head. Where does ± 0.3 come from? The holdout has 20,000 images. At an accuracy of about 72%, the statistical error (the standard error) is about 0.32 points.

The one sentence to remember

The usual linear-probe start hurt all three methods we tried, while a head built from class prototypes added 2 to 3 points.
In plain words: start the head from prototypes, not from a linear probe.

1.15 Slide 15: Thank you (and who answers which questions)

Speaker: D



Slide 15

What this slide says

The closing slide, which invites questions. The speaker notes list **who answers which questions** (this list is not read aloud):

- Why the prototype head beats the linear-probe head, and the evidence → C.
- Why PEFT is below full fine-tuning, whether the final comparison is fair, and why LoRA gets worse with longer training → D.
- Method details, parameter counts, latency, and LoRA vs the Adapter → B.
- The budget, GPU-hours, the backbone, the data and the holdout → A.
- Whether the resolution result is over budget, and our limits → D.

How to use this slide well

- When you answer, give a one-sentence conclusion first, then one number, then the reason.
- If a question is about the structure of a specific method, you can jump to the backup slides 16–20.
- If you do not know, say “We did not test that, but our guess is …”. Do not make things up.

Chapter 5 has more than 30 likely questions, with English answers.

The one sentence to remember

Thank you; we are happy to take questions.

1.16 About the backup slides (slides 16–20)

Slides 16–20 are **backup slides**. We do not present them in the talk. We only show one if someone asks about a specific method. They come from the first version of our slides. They cover five of the seven methods: all except full fine-tuning (slide 10) and LoRA (slide 11).

Every backup slide has the same structure:

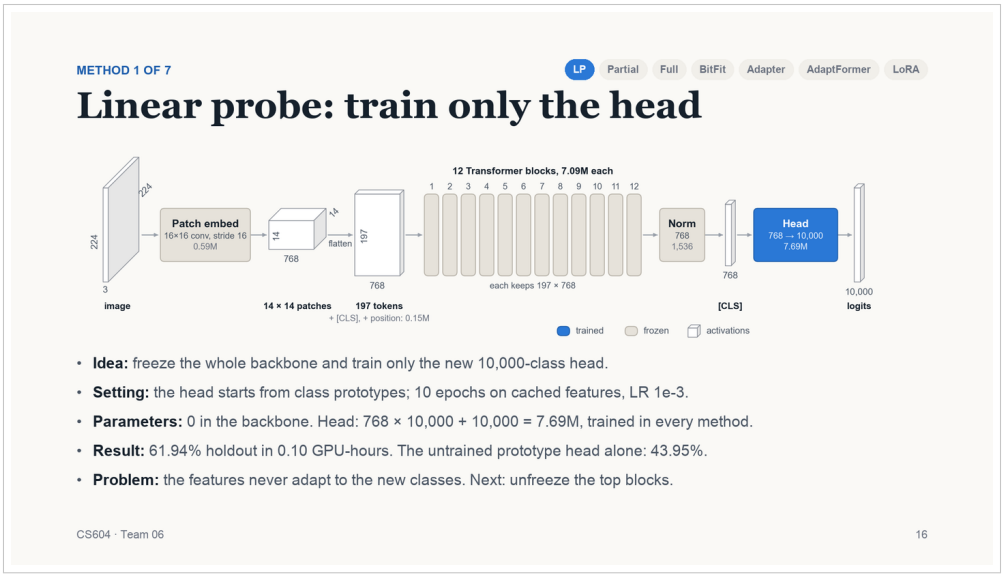
- **A row of "pills" in the top right** (LP, Partial, Full, BitFit, Adapter, AdaptFormer, LoRA): the seven methods, from simple to complex. The current method is solid blue, the ones already covered are light blue, and the ones still to come are grey. Together they form a "ladder" of methods.
- **A diagram**: blue parts are trained, and grey parts are frozen. The first two methods show the whole network; the last three show the inside of **one** Transformer block (all 12 blocks are the same).
- **Five lines**: Idea, Setting, Parameters (how the count is calculated), Result, and Problem → Next (the weakness of this method, and how the next method fixes it).

The five backup slides tell one story

Linear probe: the features never change → partial fine-tuning: only the top changes, the bottom stays the same → full fine-tuning: everything changes, but each task must store a full copy of the model → BitFit: only the biases change, too little capacity → Adapter: add small modules, but they sit in series on the main path and slow down inference → AdaptFormer: move them to a parallel side branch, but they still cannot be merged → LoRA: an update that can be merged.
The "Problem" of each method is exactly what the next method solves.

1.17 Slide 16 (backup): Linear probe: train only the head

Speaker: B (if asked about the linear probe)



Slide 16

What this slide says

- **Idea**: freeze the whole backbone and train only the new 10,000-class head.
- **Setting**: the head starts from the class prototypes; 10 epochs on precomputed ("cached") features, learning rate 10^{-3}.
- **Parameters**: 0 in the backbone. The head has $768 \times 10{,}000 + 10{,}000 = 7.69$M parameters, and every method trains it.
- **Result**: 61.94% on the holdout, using only 0.10 GPU-hours. The untrained prototype head alone gives 43.95%.
- **Problem**: the features never adapt to the new classes. Next: unfreeze the top blocks.

Key concepts

Cached features: because the backbone does not change, the features of each image need to be computed only once and stored. Training the head then uses only these stored numbers, so it is very fast and uses very little memory (3.2 GB).

Why it matters

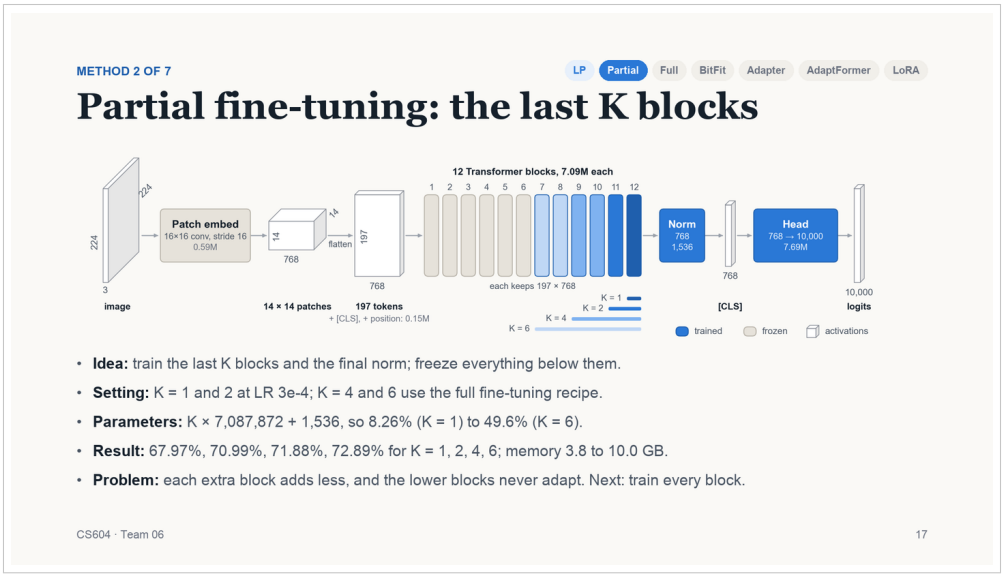
It tells us how far frozen features alone can go: 61.94%. That is more than 11 points below full fine-tuning (73.38%). So this task **must** let the features adapt (as point 1 of slide 2 says).

Easy to get wrong

The "linear probe" on slide 16 is an **adaptation method** (train only the head). The "linear-probe head" on slide 6 is a way to **initialize** the head (use a head trained by a linear probe as the starting point for fine-tuning). Both use the same technique, but for different purposes.

1.18 Slide 17 (backup): Partial fine-tuning: the last K blocks

Speaker: B (if asked about partial fine-tuning)



Slide 17

What this slide says

- **Idea**: train the last K blocks and the final norm layer; freeze everything below them.
- **Setting**: K = 1 and 2 use learning rate 3×10^{-4}; K = 4 and 6 use the full fine-tuning recipe.
- **Parameters**: $K \times 7{,}087{,}872 + 1{,}536$, from 8.26% (K = 1) to 49.6% (K = 6).
- **Result**: 67.97%, 70.99%, 71.88% and 72.89% for K = 1, 2, 4 and 6; memory from 3.8 to 10.0 GB.
- **Problem**: each extra block helps less, and the lower blocks never adapt. Next: train every block.

How to read the figure

Below the diagram are four bars, labeled K = 1, 2, 4 and 6. Each bar shows which blocks that K trains (counting back from block 12).

Key concepts

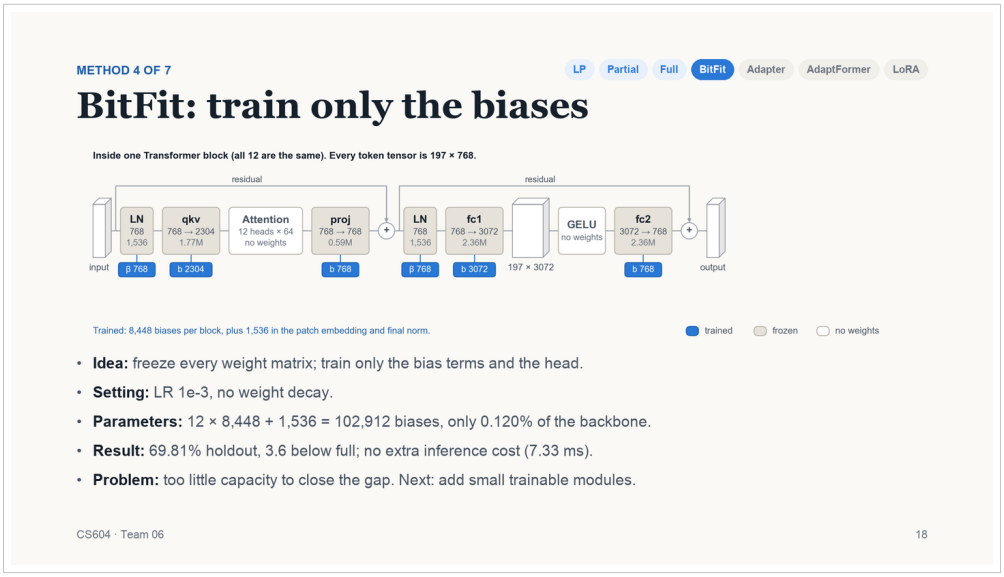
- **Diminishing returns**: the first block adds 6 points (61.94 → 67.97), the second adds 3 points, and after that every 2 blocks add only about 1 point.
- **The cheapest to train**: the gradients only need to flow back to block $12 - K + 1$, so memory is small. Even K = 6 needs only 10 GB and 0.35 GPU-hours.

Easy to get wrong

K = 1 and 2 use a different recipe from K = 4 and 6 (for example, a different learning rate). So the change from K = 2 to K = 4 mixes two factors. The report lists this as one of our limits.

1.19 Slide 18 (backup): BitFit: train only the biases

Speaker: B (if asked about BitFit)



Slide 18

What this slide says

- **Idea**: freeze every weight matrix; train only the bias terms and the head.
- **Setting**: learning rate 10^{-3}, no weight decay.

- **Parameters**: $12 \times 8{,}448 + 1{,}536 = 102{,}912$ biases, only 0.120% of the backbone.
- **Result**: 69.81% on the holdout, 3.6 points below full fine-tuning; no extra cost at inference (7.33 ms).
- **Problem**: too little capacity to close the gap. Next: add small trainable modules.

Key concepts

Bias: each linear layer of a neural network computes $y = Wx + b$. W is the weight matrix, and b is the bias, an amount that shifts the outputs. There are very few biases (one per output), so BitFit trains very few parameters. In the diagram, the small blue tags are the biases. The two LayerNorms have 768 β values each, qkv has 2,304, proj has 768, fc1 has 3,072 and fc2 has 768. Together that is 8,448 per block. On top of that come the biases of the patch embedding and of the final norm layer, 1,536 in total.

Why it matters

It is the most extreme example of "small changes in every block". With only 0.12% of the parameters, it scores almost 2 points higher than training the whole last block (8.26% of the parameters, 67.97%). This is strong evidence for the point of slide 5: where you train matters more than how much.

1.20 Slide 19 (backup): Adapter: a bottleneck after each MLP

Speaker: B (if asked about the Adapter)



Slide 19

What this slide says

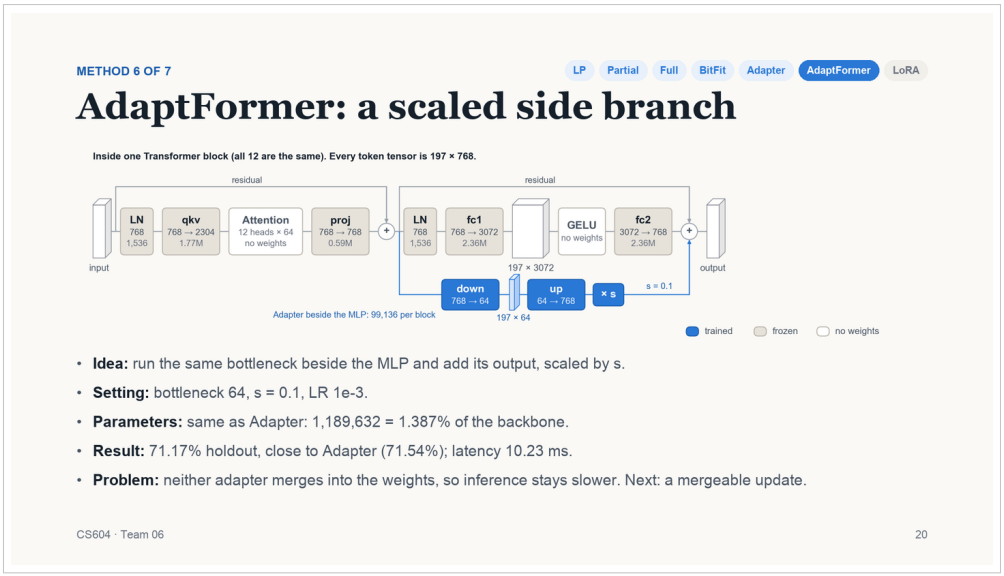
- **Idea**: add a small "bottleneck" module in series after each MLP: $768 \to 64 \to 768$.
- **Setting**: bottleneck width 64, scale 1.0, learning rate 10^{-3}; the "up" layer starts at 0.
- **Parameters**: $12 \times (768 \times 64 + 64 + 64 \times 768 + 768) = 1{,}189{,}632$, which is 1.387%.
- **Result**: 71.54% on the holdout; 12.8 GB of memory; latency 9.74 ms, against 7.33 ms for the plain model.
- **Problem**: it sits in series on the main path and rewrites the output of the MLP. Next: a parallel, scaled side branch.

Key concepts

- **Bottleneck**: first "squeeze" down to 64 (compress 768 numbers into 64), apply ReLU, then "expand" back to 768. The middle is narrow, like the neck of a bottle, so there are few parameters.
- **Its own residual connection**: the Adapter's output = its input + the bottleneck's result. Because the up layer starts at 0, the bottleneck's result is 0 at the start, so the model is exactly the original one.
- **Why inference is slower**: the Adapter is an extra layer with a non-linear step (ReLU) in the middle, so it cannot be merged into the original weights the way LoRA can. Every prediction must run it, so each image takes about 2.4 ms longer.

1.21 Slide 20 (backup): AdaptFormer: a scaled side branch

Speaker: B (if asked about AdaptFormer)



Slide 20

What this slide says

- **Idea**: put the same bottleneck module **beside** the MLP (in parallel), multiply its output by s, and add it back.
- **Setting**: bottleneck 64, $s = 0.1$, learning rate 10^{-3}.
- **Parameters**: the same as the Adapter: 1,189,632, which is 1.387%.
- **Result**: 71.17% on the holdout, close to the Adapter (71.54%); latency 10.23 ms.
- **Problem**: neither adapter can be merged into the weights, so inference stays slower. Next: an update that can be merged (LoRA).

Key concepts

- **Series vs parallel**: the Adapter comes after the output of the MLP (in series: the information must pass through it). AdaptFormer opens a small side path that starts where the MLP part starts (right after the attention part), runs next to the MLP, and is added at the end (in parallel: the original MLP output is unchanged).

- **Scale** $s = 0.1$: the output of the side branch is multiplied by 0.1 before it is added, so its effect stays gentle.

Easy to get wrong

The Adapter (71.54%) and AdaptFormer (71.17%) differ by only 0.37 points, which is within noise. They also use different scales (1.0 and 0.1). So we cannot say "series beats parallel". We can only say that, in our setting, the position makes little difference.

The backup slides in one sentence

Of the seven methods, only LoRA has all three: few parameters (1.38%), high accuracy (71.98%, the best of the efficient methods), and no slowdown at inference (it can be merged). The price: during training it uses the most memory and is the slowest.

Chapter 2 Methods and models in depth

This chapter is the "textbook" part of the guide. Chapter 1 went through the slides page by page. This chapter takes a different view. It explains, from zero, **every model and method** that appears on the slides: what it is, why we need it, how it works inside, its pros and cons, and how it differs from the other methods. After this chapter, you can answer from the "why", even when the instructor asks about a principle that is not written on the slides.

The order of this chapter follows the logic of the whole project:

1. First, we meet our model, ViT-B/16 (Section 2.1). A photo goes in and 10,000 scores come out. What happens in between?
2. Next, we look at the most important new part: the classification **head**, and its three possible starting points (Section 2.2). This is where our main finding is.
3. Then we go through the seven **adaptation** methods one by one (Section 2.3). Adaptation means fitting a pretrained model to a new task. LoRA gets the most attention.
4. After that, we cover the details of training and evaluation (Sections 2.4 and 2.5): the loss function, the optimizer, the learning rate and the evaluation metrics.
5. Finally, we cover test-time augmentation and input resolution (Section 2.6), and we end with two summary tables (Section 2.7).

Chapter 4 reads each figure in detail and works through the formulas. Chapter 3 gives short definitions of the terms, so you can look them up at any time.

2.1 Our model: ViT-B/16

2.1.1 The big picture: what goes in and what comes out

Our task is **image classification**. We give the model a photo, and it tells us which of 10,000 classes the photo belongs to. The classes are **fine-grained**, which means they look very similar to each other: for example, different kinds of birds, snakes, moths and plants. The course also hid the class names, so we only know "class 0, class 1, …, class 9,999".

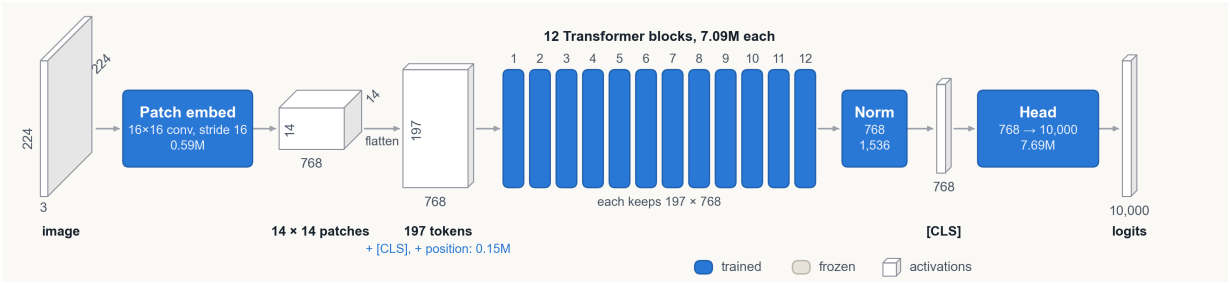
Input and output of the model

Input: one color photo, first resized and cropped to 224×224 pixels. Each pixel has 3 numbers (red, green and blue), so the input is $224 \times 224 \times 3 = 150{,}528$ numbers.
Output: 10,000 scores, called **logits** (raw scores that are not yet probabilities). Each score belongs to one class. The higher the score, the more the model thinks the photo is in that class.
Prediction: the **softmax** function turns the 10,000 scores into probabilities that add up to 1. The class with the highest probability is the model's answer.

One photo going through the model

Suppose the input is a photo of a sparrow. The model outputs 10,000 scores. After softmax, the largest probabilities are: class 3,812 gets 0.71, class 3,815 gets 0.12, class 977 gets 0.05, and so on. The model's top-1 prediction is class 3,812. If the true label is also class 3,812, the photo counts as "correct". (These numbers are made up, only to show the idea.)

The name ViT-B/16 has three parts. **ViT** stands for **Vision Transformer**. **B** stands for **Base**, the medium-size version. **16** means the image is cut into small squares of 16 × 16 pixels. Below, we follow the data step by step and see what happens inside.



The full structure of ViT-B/16 (this figure is from slide 10; blue = the parts trained in full fine-tuning): image → patches → 197 tokens → 12 Transformer blocks → normalization → [CLS] feature → classification head → 10,000 scores

2.1.2 Step 1: cut the image into patches and turn them into vectors

The Transformer was first designed for text. It reads a sequence of "words". To let it read an image, ViT cuts the image into small squares and treats each square as one "word".

- The image is 224 pixels wide and each square is 16 pixels, so each row has $224 \div 16 = 14$ squares. In total there are $14 \times 14 = 196$ squares. These squares are called **patches**.
- Each patch has $16 \times 16 \times 3 = 768$ pixel values. The model uses a linear transform to turn these 768 raw values into a vector of 768 numbers. (In code, this is a 16×16 convolution with stride 16.) This step is called the **patch embedding**. It has $16 \times 16 \times 3 \times 768 + 768 = 590{,}592$ parameters (about 0.59M).
- The vector after the transform is called a **token**. Now the image has become 196 tokens, and each token is 768 numbers.

Everyday analogy

Cut a photo into 196 puzzle pieces. For each piece, write a "description card" with 768 numbers. The card describes the colors, textures and edges in that piece. From now on, the model does not look at raw pixels. It reads these 196 cards.

2.1.3 Step 2: add the [CLS] token and the position embeddings

The [CLS] token: a special token that sums up the whole image

In front of the 196 patch tokens, ViT adds one more special token, called **[CLS]** (short for "class"; it is used for classification). At first it does not belong to any patch. It is just a learnable vector of 768 numbers. Over the 12 layers, it collects information from all the patches. In the end, its output is used as "the summary of

the whole image" and is sent to the classifier. So the total number of tokens is $196 + 1 = 197$, each with 768 numbers. You can write this as a table of 197×768 numbers.

Position embeddings: tell the model where each patch is

The Transformer itself does not know the order or the position of the tokens. If you shuffled the puzzle pieces, it could not tell the difference. So ViT keeps one learnable vector of 768 numbers for each of the 197 positions and adds it to the token in that position. This is the **position embedding**. The [CLS] token and the position embeddings together have $768 + 197 \times 768 = 152{,}064$ parameters (about 0.15M).

Everyday analogy

The [CLS] token is like a team leader. Each of the 196 team members (the patches) looked at one small part of the photo. The leader listens to everyone and then gives the conclusion for the whole team. Position embeddings are like seat numbers. Each description card says "I come from row 3, column 5", so the model knows which piece is at the top and which is at the bottom.

2.1.4 Step 3: 12 Transformer blocks

This is the main body of the model. There are 12 **blocks** (layers) with exactly the same structure, one after another. Each block takes in 197×768 numbers and gives out 197×768 numbers. The shape does not change, but the content "understands" the image better and better. Early layers look more at simple things like edges, colors and textures. Later layers look more at abstract things like object parts and overall shapes.

Inside each block, in order:

1. **LayerNorm (LN)**: it shifts and scales the 768 numbers of each token so that their mean is 0 and their variance is 1. Then it multiplies them by a learnable scale γ and adds a learnable shift β. This keeps the numbers in a good range and makes training more stable. Each LN has $768 + 768 = 1{,}536$ parameters.
2. **Multi-head self-attention**: each token "looks at" all the other tokens and collects useful information from them. It has three parts: the qkv linear layer ($768 \to 2304$; it computes the Q, K and V vectors in one go), the attention calculation itself (no parameters), and the proj linear layer ($768 \to 768$; it combines the results of the 12 heads). We explain it below.
3. **Residual connection**: add the attention output back to its input. Output = input + attention(input).
4. **A second LayerNorm**.
5. **MLP (multi-layer perceptron)**: two linear transforms, applied to each token on its own. fc1 expands 768 numbers to 3072. Then comes the **GELU** activation function, a smooth function that keeps positive numbers and squeezes negative numbers toward 0. Then fc2 brings 3072 back down to 768. Attention is for "talking to each other"; the MLP is for each token to "think on its own".
6. **A second residual connection**: output = input + MLP(input).

The self-attention formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

- Q (**Query**): what each token is "looking for".
- K (**Key**): a label on each token that says "what information I have".

- V (**Value**): the actual content that each token passes on.
- $QK^\top$: the dot product of every Query with every Key. A larger dot product means a better match. All 197 tokens are paired with each other, which gives 197×197 scores.
- $\sqrt{d}$: $d = 64$ is the size of each head. Dividing by $\sqrt{64} = 8$ keeps the scores from getting too large. Otherwise softmax becomes too extreme.
- softmax: turns each row of scores into weights that add up to 1.
- Multiply by V: take a weighted average of the tokens' contents, using these weights. The result is the new token.

Multi-head: ViT-B has 12 heads, each with 64 numbers, and $12 \times 64 = 768$. Different heads can focus on different relations. For example, one head may look at regions with similar colors, and another may look at regions that are connected in shape.

A tiny calculation with two tokens

Let the size be $d = 2$. Token 1 has the Query $q = (1, 0)$. The Keys of the two tokens are $k_1 = (1, 0)$ and $k_2 = (0, 1)$.

- Dot products: $q \cdot k_1 = 1$ and $q \cdot k_2 = 0$.
- Divide by $\sqrt{2} \approx 1.414$: we get 0.707 and 0.
- Softmax: $e^{0.707} \approx 2.03$ and $e^0 = 1$, so the weights are $2.03/3.03 \approx 0.67$ and $1/3.03 \approx 0.33$.
- The new token 1 is $0.67 \times v_1 + 0.33 \times v_2$. It takes in mostly the content of token 1, which matches its question, and a little of token 2.

Everyday analogy: self-attention is like a meeting

197 people are in a meeting. Each person has a question (the Query), wears a name badge that says what they know about (the Key), and has real knowledge in their head (the Value). Based on their own question, each person checks whose badge matches best. They listen more to those people (high weight) and less to the others (low weight). After the meeting, everyone has updated what they know. The 12 heads are like 12 small group meetings on different topics, held at the same time.

Everyday analogy: a residual connection is like editing a draft

A residual connection means that each layer does not rewrite from zero. It edits the draft: output = draft + changes. So even if one layer learns badly, the original information is not lost. Also, during training, the signal can travel back to the earlier layers along this "draft" shortcut, which makes deep networks easier to train.

How many parameters does each block have?

Each linear layer has (input size $\times$ output size) + output size (the biases) parameters. So one block has:

Part	Calculation	Parameters
qkv ($768 \to 2304$)	$768 \times 2304 + 2304$	1,771,776
proj ($768 \to 768$)	$768 \times 768 + 768$	590,592
fc1 ($768 \to 3072$)	$768 \times 3072 + 3072$	2,362,368
fc2 ($3072 \to 768$)	$3072 \times 768 + 768$	2,360,064
Two LayerNorms	$2 \times (768 + 768)$	3,072
Total for one block		**7,087,872**

Key point

The MLP (fc1 + fc2) has 4,722,432 parameters in total. That is **two thirds** of a block. This explains why LoRA gained 0.88 points when we extended it from qkv to the MLP: before that, most of the weights were not adjusted at all.

2.1.5 Step 4: the final norm, the [CLS] feature and the classification head

After the 12 blocks, there is one final LayerNorm (1,536 parameters). Then the model takes only the output of the [CLS] token. It is a vector of 768 numbers, called the **feature** of the image. We write it as z.

Finally, the classification **head** turns this 768-number feature into 10,000 scores:

$$\text{logits} = Wz + b, \qquad W \in \mathbb{R}^{10000 \times 768},\ b \in \mathbb{R}^{10000}$$

Each row of W belongs to one class. You can think of it as the "template" of that class. The more the feature z looks like a row (the larger the dot product), the higher the score of that class. The head has $768 \times 10{,}000 + 10{,}000 = 7{,}690{,}000$ parameters.

2.1.6 How many parameters in total

Part	Parameters	Note
Patch embedding	590,592	turns patches into 768-number vectors
[CLS] + position embeddings	152,064	$768 + 197 \times 768$
12 blocks	85,054,464	$12 \times 7{,}087{,}872$
Final LayerNorm	1,536	
Backbone total	**85,798,656**	about 85.8M
New classification head	**7,690,000**	about 7.69M, 9.0% of the backbone

2.1.7 How the backbone and the head relate

- **Backbone**: everything before the head, that is, the whole "feature extractor" from the image to the 768-number feature. It "understands" the image.
- **Head**: the last linear layer, $768 \to 10{,}000$. It "makes the decision": it decides the class from the feature.

The pretrained model we downloaded came with a 1,000-class head (for the 1,000 classes of ImageNet-1k). It is useless for us, so we replace it with a new 10,000-class head. The backbone is the "experienced veteran"; the head is the "new hire".

A "fixed backbone" is not the same as a "frozen backbone"

The instructor said "please use the recommended backbone and do not switch it casually". This means the **choice of model** stays fixed: every experiment uses the same ViT-B/16. That is different from **freezing**, which means not updating its parameters during training. Full fine-tuning uses this same backbone; it just trains all 85.8M of its parameters. LoRA and the other efficient methods freeze the backbone and train only small new parts. Neither approach "switched the backbone".

2.1.8 Pretraining and transfer learning: why we do not train from scratch

We use vit_base_patch16_224.augreg2_in21k_ft_in1k from the timm library. The name means:

- in21k: first pretrained on ImageNet-21k. This is a large public dataset with more than twenty thousand classes and more than ten million images.
- augreg: pretraining used a set of data augmentation and regularization methods called AugReg. (Augmentation makes varied copies of the training images; regularization helps a model avoid memorizing.)
- ft_in1k: then fine-tuned on ImageNet-1k (1,000 classes).
- patch16_224: patches of 16 pixels and an input size of 224 pixels.

Transfer learning means taking the "ability to see" learned on a large dataset and transferring it to a new task. We have only 45 photos per class. If we trained an 85.8M-parameter model from scratch, it would quickly memorize these 45 photos but would not learn the real patterns. The pretrained model can already recognize edges, textures, shapes and object parts. We only need to adapt it to the new 10,000 classes.

Everyday analogy

A doctor trained in general medicine who moves to dermatology learns much faster than someone who never studied medicine. The basic knowledge (anatomy, pathology) carries over, so they only need to learn what is special about skin. The pretrained model is the "doctor trained in general medicine". Our task is "specialist training in dermatology", with only 45 cases of each disease to learn from.

Why we did not switch the backbone

1. Page 1 of the project brief recommends this model, and the instructor's email says not to switch without a strong reason.
2. With it, we reproduced the three reference results in the brief, all within 0.3 points. This shows that our code pipeline is correct.
3. After the deadline, we also checked two versions that were pretrained only on ImageNet-21k. We compared them like this: freeze the backbone, train only the head (a linear probe), and see whose features are better. The results: augreg_in21k 61.05%, orig_in21k 51.07%, both below the 61.94% of the backbone we use. So switching the backbone would not help.

2.2 The classification head and its three starting points

2.2.1 Why the head matters so much

The head looks like "just the last layer", but in our task it is special:

- **It is large**: 7.69M parameters, 9.0% of the backbone. That is 26 times the reference LoRA in the brief (LoRA only on qkv, 294,912 parameters).
- **It is brand new**: the backbone is pretrained, but the head is new. At the start it knows nothing.
- **It has very little data**: each of its 10,000 rows (the template of one class) can only learn from the 43–45 photos of that class.

So **the values the head starts training from** affect the whole training run. This is the **initialization** problem. We compared three initializations.

How the head scores a class

$$\text{logit}_c = w_c \cdot z + b_c$$

- z: the 768-number feature of the image.
- w_c: row c of the head, also 768 numbers. Think of it as the "template" of class c.
- $w_c \cdot z$: the dot product, which measures how similar the feature and the template are.
- b_c: the bias of class c (one number).

Initialization means choosing the values of w_c and b_c before training starts.

2.2.2 Start 1: random initialization

What it is

Fill the head with small random numbers. This is what the timm library does by default for a new linear layer.

How it works, and the problem

A random head gives about the same score to every class, so each of the 10,000 classes gets a probability close to $1/10{,}000$. The cross-entropy loss is then about $\ln 10{,}000 \approx 9.21$ (Section 2.4 explains the loss). We measured an average loss of 9.28 over the first 100 steps of LoRA training, almost exactly this value.

A large loss means large gradients. At the start of training, large and messy gradients flow from the head back into the backbone. They may "damage" the good features learned in pretraining. This is exactly the worry of the LP-FT paper.

Pros and cons

Pros: the simplest choice, with no extra computation. Cons: the worst starting point (accuracy close to 0), so early training is wasted; in theory, it may distort the features.

2.2.3 Start 2: the linear-probe head (LP) and LP-FT

What it is

First freeze the backbone, compute the features of all training images and store them. Then train only the head on these fixed features, so it learns to classify. Finally, use this trained head as the starting point for

fine-tuning. This is called **linear-probe** initialization. The two-step recipe "first a linear probe, then fine-tune everything" is called **LP-FT** (Linear Probing then Fine-Tuning). It comes from a paper by Kumar et al. at ICLR 2022, and it is a common, recommended recipe.

Why people recommend it

The LP-FT paper argues that a random head produces large, messy gradients early in fine-tuning. These gradients **distort** the pretrained features. This especially hurts how well the model works on data from a "new domain". If the head is trained first, the gradients during fine-tuning are small, and the features are protected.

How it works

1. Use the frozen backbone to compute a feature for each of the 450,000 training images (about 6 minutes).
2. Start from the prototype head (Start 3 below) and train the head on these cached features for 10 epochs (AdamW, learning rate 10^{-3}, batch 4,096, cosine decay, label smoothing 0.1). This takes only a few seconds, and holdout accuracy reaches about 62%.
3. Start fine-tuning (full fine-tuning or LoRA) from this head.

What happened in our task

The LP head starts very well (62.1%, 18 points above the prototype head's 44.0%), but after training it is worse:

- Full fine-tuning, 3 epochs (holdout): LP 70.88%, prototype 73.38%.
- LoRA, 3 epochs (holdout): LP 69.45%, prototype 72.00%.
- AdaptFormer, 3 epochs (holdout): LP 69.05%, prototype 71.17%.
- Final recipe (full fine-tuning, 8 epochs, CodaBench validation set): LP 73.10%, prototype 76.22%, a gap of 3.12 points.

It was the same for all three methods we tested: the LP head lowered the final result by 2.1–3.1 points.

Why the LP head hurts here: three steps

Step 1: it was fitted on the same training photos. The features used to train the LP head come from the same photos that fine-tuning uses later. So when fine-tuning starts, the head already classifies these photos quite well. The training loss is only 2.29 (the prototype head starts at 4.39, the random head at 9.28).

Step 2: the signal is first weak, then wrong. A small loss means small gradients. The backbone gets a weak learning signal, and it comes mostly from the few photos that the LP head happens to get wrong. Once the backbone's features start to change, the fitted head no longer matches the new features. In LoRA training with the LP head, the training loss even **rose** from 2.22 to 2.69 during the first epoch, before any photo had been seen a second time.

Step 3: the result is memorization. The two final-recipe models differ only in the head's starting point. The LP model always has the lower training loss (1.77 at the end, against 1.85 for the prototype model), yet a much lower validation accuracy (73.10% against 76.22%). Lower training loss but lower real accuracy: this is **memorization**. The model remembered the training photos instead of learning the patterns.

Everyday analogy

The LP head is like a student who memorized the answers to a workbook in advance. The first time through the same workbook, the student scores very high (a 62% start), so they feel there is nothing left to learn (small gradients). But then the teacher changes how the material is taught (the backbone's features change), and the memorized answers no longer fit (the loss goes up). In the end, the student knows the workbook even better (lower training loss), but does worse on new questions (the validation set).

Pros and cons, and when to use it

Pros: a high starting accuracy. LP-FT really does help when there is a lot of training data and the features must be protected for new domains. Cons: in our setting, it leads to memorization and a worse final result. Our setting has only 45 photos per class, very many classes, and a head fitted on the same photos.

2.2.4 Start 3: the prototype head (weight imprinting)

What it is

Do not train the head; compute it directly. The template w_c of each class is the average direction of the features of all training photos in that class. This average direction is called the class **prototype**. Then multiply it by a temperature of 40. This method is called **weight imprinting**, from a paper by Qi et al. at CVPR 2018. **We did not invent the method.** Our new finding is that, in our setting, using it instead of the LP-FT starting point raised all three methods by 2.1–3.1 points.

How it works (step by step)

1. Use the frozen backbone to compute a 768-number feature for each of the 450,000 training photos (about 6 minutes, counted in the GPU time).
2. Divide each feature by its own length, so it becomes a vector of length 1 (L2 normalization). Only its "direction" is kept.
3. Add up the direction vectors of the same class and normalize again. This gives the prototype μ_c of that class.
4. Set row c of the head to $w_c = \mu_c \times \dfrac{40}{\text{mean feature length}}$, and set the bias to $b_c = 0$.
5. Then each score is about $40 \times \cos(z, \mu_c)$: the cosine similarity between the photo's feature and the class's average direction, times 40.

The whole process uses **no gradient descent step at all**. The head has never "fitted" any training photo, so there is nothing for it to memorize.

Prototypes and cosine similarity

$$\mu_c = \frac{\sum_{i:\, y_i = c} \hat{z}_i}{\left\| \sum_{i:\, y_i = c} \hat{z}_i \right\|}, \qquad \hat{z}_i = \frac{z_i}{\|z_i\|}, \qquad \text{logit}_c \approx \tau \cdot \cos(z, \mu_c)$$

- z_i: the feature of training photo i; y_i is its class.
- $\hat{z}_i$: the normalized feature, with length 1. $\|\cdot\|$ means the length of a vector.
- $\sum_{i:\, y_i = c}$: add up all photos that belong to class c.
- $\cos(z, \mu_c)$: the cosine of the angle between two vectors. It ranges from -1 to 1. The closer it

is to 1, the more similar they are.

- $\tau = 40$: the **temperature**, also called the **logit scale**. We tried 5, 10, 15, 20, 30, 40, 60 and 80, computed the cross-entropy on 10,000 training features, and 40 was the best.

A small 2-D example

Suppose features have only 2 numbers. The "sparrow" class has two photos, with features $(2, 0)$ and $(1, 1)$.

1. Normalize: $(2, 0) \to (1, 0)$; $(1, 1) \to (0.707, 0.707)$.
2. Add them: $(1.707, 0.707)$, with length ≈ 1.848. Normalize again to get the prototype $\mu_{\text{sparrow}} \approx (0.924, 0.383)$.
3. A new photo arrives with the feature $(3, 1)$, which normalizes to $(0.949, 0.316)$. Its cosine with the sparrow prototype is $0.924 \times 0.949 + 0.383 \times 0.316 \approx 0.998$.
4. Suppose another class, "swallow", has the prototype $(0, 1)$. The cosine with it is 0.316.

What the temperature does: without it, the two scores are 0.998 and 0.316. After softmax, the probabilities are only 0.66 and 0.34, which is too "unsure". After multiplying by 5, the scores are 4.99 and 1.58, and the sparrow probability becomes 0.97. The real task has 10,000 classes, and many classes have similar cosines, so we need a larger temperature (40) to make the correct class stand out.

Everyday analogy

Make an "average face" for each person: stack their 45 photos and take the average. To recognize someone, compare the new photo with each person's average face. Whoever it looks most like is the answer. This does not need to remember any specific photo, so it cannot "memorize". But the average face already holds the person's main features, so the starting point is decent (44%).

Why it works

- **It encodes the structure of the classes**, so the model learns faster. After one epoch of LoRA training, the prototype head is at 63.3%, and the random head only at 58.4%.
- **It has not fitted any training photo**, so the model does not start out "feeling it already knows everything". The starting loss is 4.39, higher than the LP head's 2.29, so the backbone gets a strong and meaningful learning signal.

2.2.5 The key control experiment: the random head ties the prototype head

If the prototype head were only "better than random", the LP head's problem might just be that it is "not good enough". To tell these apart, we ran a **control** experiment. LoRA trained for 3 epochs ends at 71.95% with a random head and 72.00% with the prototype head, almost the same. (Both runs were on an RTX PRO 6000. The prototype run, repeated on the A100, gave 71.98%.)

What the control experiment shows

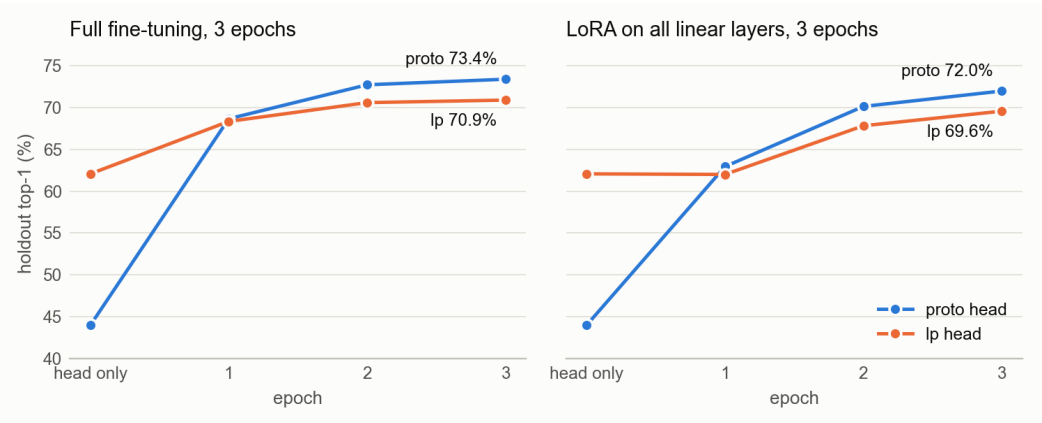
The random head, the worst starting point, ends up as good as the prototype head. Only the LP head is clearly worse. So the problem is not "a starting point that is not good enough". The problem is that **the head was fitted beforehand on the same training photos**. The prototype head is not magic. Its value is that it starts at a sensible point, learns fast, and has not been fitted. This also partly answers the worry of the LP-FT paper. In our task, the "feature distortion" caused by a random head did no

harm, because our 10,000 fine-grained classes need the features to change anyway.

2.2.6 The three starting points side by side

	Random head	LP head (LP-FT)	Prototype head (imprinting)
How you get it	random numbers	trained for 10 epochs on cached features	average direction of each class's features × 40
Accuracy before training	close to 0 (random guessing)	62.1%	44.0%
Starting training loss*	9.28	2.29	4.39
LoRA, 3 epochs (holdout)	71.95%	69.45%	72.00%
Full fine-tuning, 3 epochs (holdout)	–	70.88%	73.38%
Final recipe (validation set)	–	73.10%	76.22%
Main pro	simple	high start	learns fast, no memorization
Main con	poor start, messy early gradients	fitted on the same photos, which leads to memorization	needs one pass to compute features (about 6 minutes)

* The starting training loss is the average training loss over the first 100 steps of LoRA training. It includes the effects of label smoothing and data augmentation, so only compare the three numbers with each other. Do not convert them into "the probability of the correct class". 9.28 is about random guessing ($\ln 10{,}000 = 9.21$). 4.39 means the head roughly knows what each class looks like but is still very unsure. 2.29 means the head has already been fitted on these training photos.



The figure from slide 6: holdout accuracy over the epochs. The LP head (orange) starts high and ends low; the prototype head (blue) starts low and ends high. The last points of this LoRA curve are 69.6% against 72.0% (values recorded after each epoch). Chapter 4 reads this figure in detail.

2.3 The seven adaptation methods

Adaptation means how we make a pretrained model fit our new task. The methods differ in **which parameters they train**. We compared seven methods, and they fall into two groups:

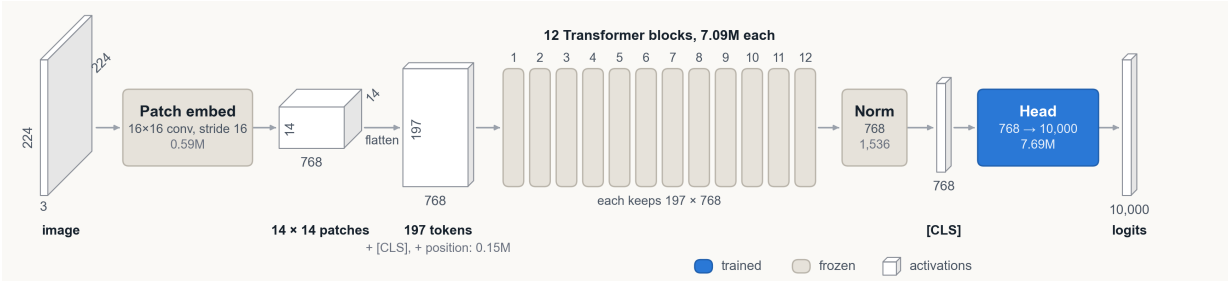
Group	Methods
Group 1: train whole blocks, starting from the top	linear probe (trains only the head), partial fine-tuning (trains the last K blocks), full fine-tuning (trains all parameters)
Group 2: make small changes inside every block	BitFit (trains only the biases), Adapter (a small module in series), AdaptFormer (a small module in parallel), LoRA (a low-rank update)

The methods in the second group are often called **PEFT** (Parameter-Efficient Fine-Tuning). They train very few parameters but come close to full fine-tuning.

The rule for a fair comparison: one protocol

So that the seven methods can be compared directly, they all use exactly the same setup (the **protocol**): the prototype head, 3 epochs of training, the same 20,000-photo holdout and the same kind of GPU (A100). Every method also trains the head (7.69M parameters). So, following the brief, the "trainable parameters" below **do not count the head**. They only count what is trained inside the backbone.

2.3.1 Method 1: Linear probe



Linear probe: the whole backbone is frozen (gray); only the final head is trained (blue).

What it is

Freeze the whole backbone and do not change a single parameter in it. Train only the head. The backbone is used only as a fixed "feature extractor".

Why we need it

It is the cheapest baseline. With very little computing, it tells us "how good the pretrained features are on their own". We also used it to compare the features of different backbones (see Section 2.1).

How it works (step by step)

1. Use the frozen backbone to compute the features of all training photos once, and store them.
2. Start the head from the prototype head and train it on the cached features for 10 epochs (learning rate 10^{-3}).
3. The backbone does not change, so no gradients need to flow into it. This makes it very fast and very light on memory.

Example

Everyday analogy

You hire an experienced photographer (the backbone) to look at your photos and tell you what is in them. You only train a new assistant (the head) to label the photos, based on the photographer's descriptions. The photographer's eye does not change at all, even though they have never carefully told these 10,000 kinds of living things apart.

Data: holdout 61.94%, using only 0.10 GPU-hours, 3.2 GB of training memory, and a latency of 6.96 ms per photo. For comparison, the prototype head with no training at all (**zero-shot**) gets 43.95%.

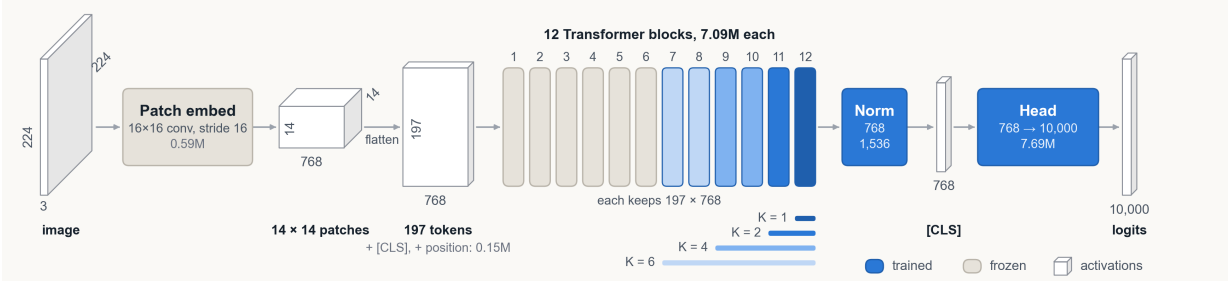
Pros and cons, and when to use it

Pros: the fastest and cheapest method. The backbone does not change, so one backbone can serve many small heads. Cons: the features never adapt to the new classes, so accuracy hits a ceiling (about 62%, more than 11 points below full fine-tuning). When to use it: for a quick baseline, to check feature quality, or when there is very little computing power.

How it differs from the other methods

Compared with the zero-shot prototype head, the linear probe trains the head and gains about 18 points. Compared with the other six methods, it is the only one that does not change the backbone's features. So its result can be seen as "the ceiling for frozen features".

2.3.2 Method 2: Partial fine-tuning (the last K blocks)



Partial fine-tuning: only the last K blocks, the final LayerNorm and the head are trained. The bars below show which blocks are trained for K = 1, 2, 4 and 6.

What it is

Unfreeze the last K Transformer blocks (and the final LayerNorm), and train them together with the head. The lower $12 - K$ blocks stay frozen. We tried K = 1, 2, 4 and 6.

Why we need it

The linear probe changes no features, and full fine-tuning changes all of them. Partial fine-tuning sits in between. The intuition: the lower layers learn general things like edges and textures, which need few changes. The top layers are closer to the specific task, so they need to adapt most to the new classes.

How it works (step by step)

1. Freeze the first $12 - K$ blocks and the patch embedding.
2. Train the last K blocks, the final LayerNorm and the head.

3. Gradients only need to flow from the head down to block $12-K+1$, not further. This saves memory and computing.

Trainable parameters $= K \times 7{,}087{,}872 + 1{,}536$ (the 1,536 is the final LayerNorm).

Parameters and results

K	Trainable parameters	Share of the backbone	Holdout
1	7,089,408	8.26%	67.97%
2	14,177,280	16.5%	70.99%
4	28,353,024	33.0%	71.88%
6	42,528,768	49.6%	72.89%

Add the linear probe (K = 0, 61.94%) and full fine-tuning (K = 12, 73.38%), and you can see a pattern. The first block brings about 6 points, the second about 3, and after that every two more blocks add only about 1 point. This is called **diminishing returns**. Training memory grows from 3.8 GB (K = 1) to 10.0 GB (K = 6), about 1.5 GB more for each extra trained block. Note: K = 1 and 2 use a learning rate of 3×10^{-4}, while K = 4 and 6 use the full fine-tuning recipe. So the change from K = 2 to K = 4 mixes two factors.

Everyday analogy

A 12-floor office building is being converted for a new use. Partial fine-tuning renovates only the top few floors and leaves the lower floors as they are. The more floors you renovate, the better the result. But the lower you go, the smaller the improvement.

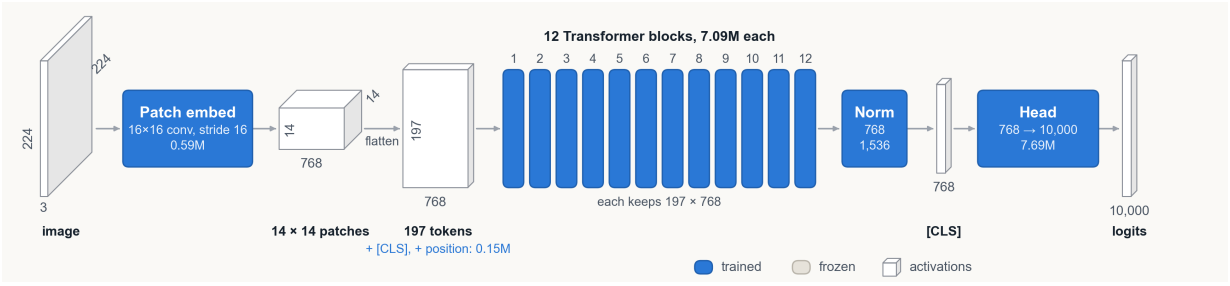
Pros and cons, and when to use it

Pros: the cheapest to train (at most 10 GB of memory and 0.35 GPU-hours), with no extra cost at inference. Training half of the blocks is only 0.5 points below full fine-tuning. Cons: the lower layers never change; there are many trainable parameters (8%–50%), so each task must store a large set of weights; and the parameter efficiency is low. When to use it: when memory is tight but storage is not a problem.

How it differs from the other methods

It makes "big changes in a few layers", while BitFit, Adapter, AdaptFormer and LoRA make "small changes in every layer". The contrast is striking. BitFit trains only 0.12% and beats K = 1 (8.26%). LoRA trains 1.38% and matches K = 4 (33%). This is our conclusion: "**where you train matters more than how much you train**".

2.3.3 Method 3: Full fine-tuning



Full fine-tuning: every part is blue, so everything is trained.

What it is

Train every parameter of the model: the 85.8M in the backbone plus the 7.69M in the head. This is the method of our best-ranked final model.

Why we need it

It has the most freedom: the model can adjust every layer as needed. So it is usually the most accurate, and it is often used as the "upper bound" for the other methods. Our 10,000 fine-grained classes need the features to change in every layer, so it suits our task best.

How it works (step by step)

1. All parameters are trainable.
2. Use a **small learning rate**, 10^{-4}, so the model does not wash away what it learned in pretraining.
3. Use **layer-wise learning-rate decay** of 0.75. The closer a layer is to the input, the smaller its learning rate (the first block gets only about 3%). So the general features in the lower layers change little, and the top layers change more.
4. Use regularization such as **drop path** 0.1 and **weight decay** 0.05 to prevent memorization.
5. Section 2.4 explains these details.

Example

Everyday analogy

The whole team takes training for a new line of business, and everyone may adjust how they work. Senior staff (the lower layers) adjust only a little. Staff who work closest to the customers (the top layers) adjust more.

Data: under the shared protocol, holdout 73.38%, the highest of the seven methods; 18.9 GB of training memory; a training speed of 1,139 images per second; 0.48 GPU-hours. Each task must store a full copy of the model, about 374 MB. The final model (8 epochs, all 45 photos per class, flip TTA) gets 76.22% on the CodaBench validation set and 76.09% on the test set, 5.4 points above the brief's reference (70.8%).

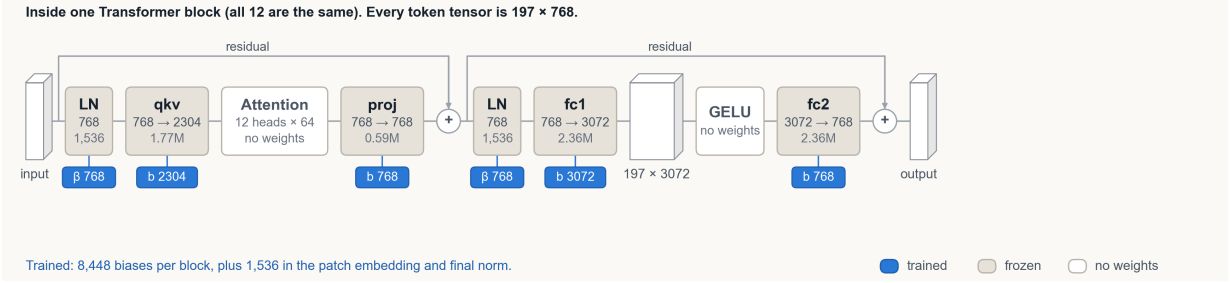
Pros and cons, and when to use it

Pros: the most accurate, and it keeps improving with longer training (going from 3 to 8 epochs added 2.21 points). Cons: large training memory; each task must store a whole model with 85.8M parameters; if the learning rate is too large, it can damage the pretrained features. When to use it: when there is enough data overall (we have 450,000 photos) and you only care about the best accuracy on one task.

How it differs from the other methods

It is the upper bound of all the methods: under the same protocol, no PEFT method beats it. The best PEFT method, LoRA, reaches 98% of its accuracy (71.98% against 73.38%) while training only 1.38% of the parameters.

2.3.4 Method 4: BitFit (train only the biases)



BitFit: a close-up of one Transformer block. All weight matrices are frozen; only the bias vectors, shown as small blue tags, are trained.

What it is

Freeze all weight matrices and train only the **bias** vectors, plus the head. It comes from a 2022 paper by Ben Zaken et al.

Why we need it

To find out "the fewest parameters we need to change". Biases are a tiny share of the parameters, but they appear in every layer. Changing them is like making a small shift in every layer.

How it works (step by step)

A linear layer computes $y = Wx + b$. BitFit changes only b and leaves W alone. b shifts the whole output, which is like adjusting the "threshold" of each neuron.

1. The biases in each block: the β of the two LayerNorms (768 each), qkv (2304), proj (768), fc1 (3072) and fc2 (768). In total: $768 + 768 + 2304 + 768 + 3072 + 768 = 8{,}448$.
2. 12 blocks give $12 \times 8{,}448 = 101{,}376$. Add the bias of the patch embedding (768) and the β of the final LayerNorm (768), and the total is **102,912**, only 0.12% of the backbone.
3. Training uses a learning rate of 10^{-3} and no weight decay.

Example

Everyday analogy

A restaurant chain moves into a market with different tastes. BitFit does not change any recipe (the weight matrices). It only lets each dish adjust its amount of salt (the biases). Each change is tiny, but every dish changes, so the overall taste can shift a lot.

Data: holdout 69.81%; 12.7 GB of training memory; a latency of 7.33 ms per photo, with no extra inference cost. LoRA is about 10 points above the linear probe. BitFit, with about 1/11 of LoRA's parameters, gets 7.9 of those 10 points.

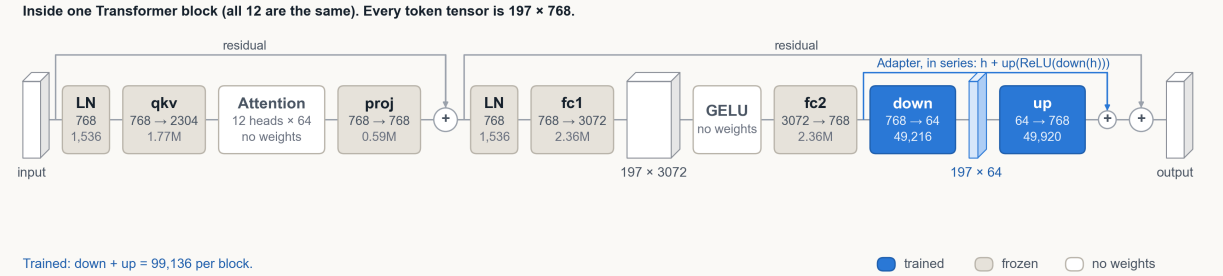
Pros and cons, and when to use it

Pros: very few parameters, so each task only stores about 100,000 numbers; no extra inference cost at all. Cons: limited power, 2.2 points below LoRA. Training memory is not small (12.7 GB), because the gradients still have to flow through all 12 blocks. When to use it: when you must store a separate model for very many tasks and storage is extremely tight.

How it differs from the other methods

Compare it with partial fine-tuning at K = 1. BitFit trains only 0.12% of the parameters, yet it is more accurate than K = 1, which trains a whole block (8.26%): 69.81% against 67.97%. This is the most direct evidence that "small changes in every layer beat a big change in one layer".

2.3.5 Method 5: Adapter (a bottleneck module in series)



Adapter: a small bottleneck module (blue) is added in series after the MLP of each block: $768 \to 64 \to 768$, with its own skip connection.

What it is

After the MLP of each block, insert a very small module **in series**, called an **adapter**. It first compresses 768 numbers down to 64, applies a ReLU activation, then expands back to 768, and adds the result to the original output. Its shape is like an hourglass, so it is called a **bottleneck**. It comes from a 2019 paper by Houlsby et al. All original backbone parameters are frozen; only the adapters and the head are trained.

Why we need it

It gives each layer a bit of "new ability" without touching the pretrained weights, so the model can adapt to a new task. Each task only needs to store its small adapters, so one backbone can serve many tasks.

How it works (step by step)

Let h be the output of the MLP (768 numbers). The adapter computes:

$$h' = h + W_{\text{up}}\,\text{ReLU}(W_{\text{down}}\,h)$$

1. W_{down}: a linear layer from 768 to 64 (compress), with $768 \times 64 + 64$ parameters.
2. ReLU: negative numbers become 0, and positive numbers stay. This adds non-linearity.
3. W_{up}: a linear layer from 64 to 768 (expand), with $64 \times 768 + 768$ parameters. It starts at all zeros. So at the start of training, the adapter outputs 0 and the model is exactly the pretrained model.
4. Add the result back to h (the adapter's own skip connection), then continue to the block's residual addition.

Each block has $2 \times 768 \times 64 + 64 + 768 = 99{,}136$ parameters. The 12 blocks together have **1,189,632**, which is 1.387%.

Example

Everyday analogy

Each department of a company (each block) keeps its old workflow. But a small "translator" sits at the exit of each department. The translator first boils the department's report down to key points (768 → 64), then rewrites it as a full report for the new business (64 → 768). On day one, the translator

changes nothing ($W_{up} = 0$), and slowly learns what to change.

Data: holdout 71.54%; 12.8 GB of training memory; a training speed of 1,509 images per second, about 1.7 times faster than LoRA. But the latency is 9.74 ms per photo, about 2.4 ms slower than the 7.33 ms of the original model.

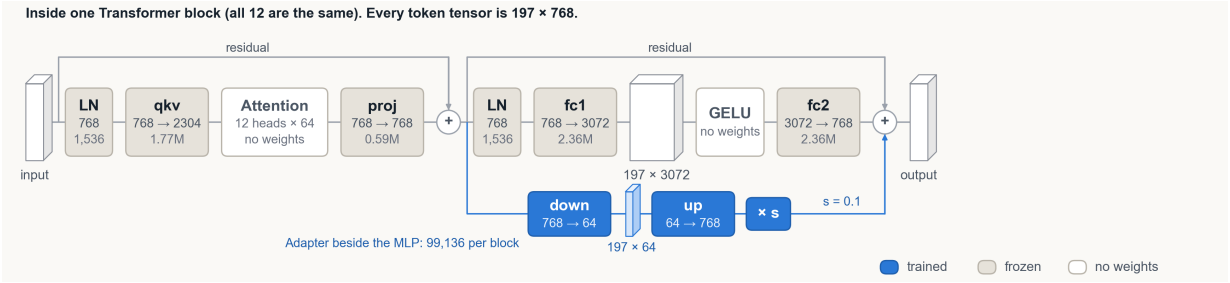
Pros and cons, and when to use it

Pros: accuracy close to LoRA (only 0.44 points lower, within noise); fast training, and 36% less memory than LoRA. Cons: slower inference. The adapter is an extra layer on the main path, and it has a ReLU inside, so it cannot be merged into the original weights. Every prediction must compute it again. When to use it: when training resources are tight and inference speed is not critical.

How it differs from the other methods

Compared with AdaptFormer, it has almost the same structure. The difference is "in series after the MLP" versus "in parallel next to the MLP". Compared with LoRA, it has about the same number of parameters (1.19M against 1.18M), but LoRA can be merged after training, so LoRA's inference is not slower.

2.3.6 Method 6: AdaptFormer (a bottleneck branch in parallel)



AdaptFormer: the same bottleneck module, but placed next to the MLP as a parallel branch. Its output is multiplied by $s = 0.1$ and then added.

What it is

It uses the same bottleneck module as the Adapter ($768 \to 64 \to 768$). But instead of sitting in series after the MLP, it sits **in parallel** next to the MLP. It gets the same input as the MLP, and each computes its own result. The module's output is multiplied by a small factor $s = 0.1$ and added in. It comes from a 2022 paper by Chen et al.

Why we need it

An adapter in series directly rewrites the MLP's output. A parallel branch keeps the original MLP output unchanged and only adds a small "correction" on the side. This is gentler.

How it works (step by step)

Let x' be the result after the attention part (that is, the input to the MLP part):

$$x_{\text{next}} = x' + \text{MLP}(\text{LN}(x')) + s \cdot W_{\text{up}}\,\text{ReLU}(W_{\text{down}}\,x')$$

1. Main path: x' goes through LayerNorm and the MLP. This part is frozen.
2. Side path: x' goes through W_{down} ($768 \to 64$), ReLU and W_{up} ($64 \to 768$), and is then multiplied by $s = 0.1$. The side path also uses dropout 0.1, and W_{up} starts at 0.

3. The three parts are added together to give the input of the next layer.

The parameters are exactly the same as the Adapter's: 1,189,632 (1.387%).

Example

Everyday analogy

The department's report goes out as usual (the main path does not change). Next to it sits an advisor who reads the same material and writes a short note with extra comments. The note only counts for 10% of the final report ($s = 0.1$).

Data: holdout 71.17% (69.05% with the LP head, so the prototype head adds 2.1 points); 12.8 GB of training memory; a latency of 10.23 ms per photo, also slower than the original model.

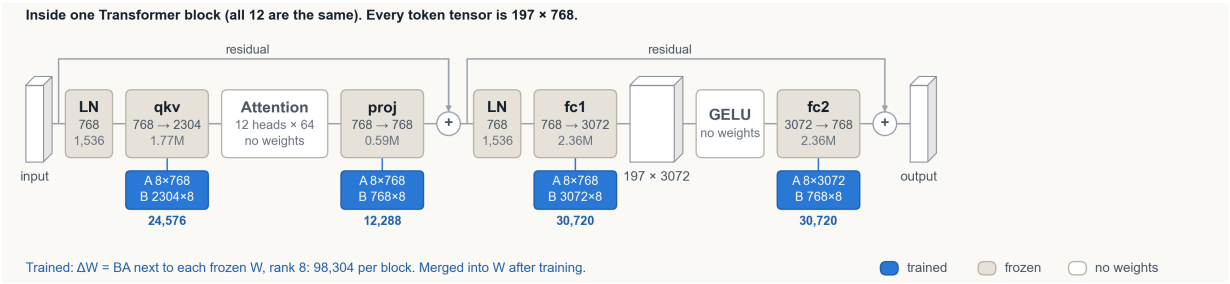
Pros and cons, and when to use it

Pros: it does not change the original MLP output, so it is more stable; fast training and little memory. Cons: like the Adapter, it cannot be merged, so inference is about 30%–40% slower.

How it differs from the other methods

Compared with the Adapter: 71.17% against 71.54%, a gap of 0.37 points. That is about the size of our noise level (about ± 0.3). The two also use different scales (1.0 for the Adapter, 0.1 for AdaptFormer), so this comparison changes more than one factor. Our conclusion: in our task, series or parallel makes little difference.

2.3.7 Method 7: LoRA (a low-rank update that can be merged)



LoRA: a close-up of one block. Next to each frozen linear layer (qkv, proj, fc1, fc2) sits a pair of small matrices, A and B (blue), that learn the update $\Delta W = BA$. After training, the update is merged into W.

LoRA is our final parameter-efficient model, and it is the method people ask about most, so we explain it in the most detail. It comes from a paper by Hu et al. at ICLR 2022. Its full name is **Low-Rank Adaptation**.

What it is

Freeze the original weight matrix W_0. Next to it, add two very "thin" matrices, A and B. Their product BA is the change to W_0. During training, only A and B (and the head) are learned. After training, BA is added into W_0, and the model goes back to its original structure.

Why we need it

- Full fine-tuning changes and stores 85.8M parameters for every task. That is too heavy.
- LoRA assumes that the "change" needed for a new task is actually simple and can be described with very few parameters. In math terms, the change is **low-rank**.

- Unlike an adapter, LoRA's change is linear, so after training it can be merged into the original weights. **Inference is not slower at all.**

First, what does "low-rank" mean?

A $d_{\text{out}} \times d_{\text{in}}$ matrix has $d_{\text{out}} \times d_{\text{in}}$ numbers. If it can be written as the product of two thin matrices, B ($d_{\text{out}} \times r$) and A ($r \times d_{\text{in}}$), it needs only $r \times (d_{\text{in}} + d_{\text{out}})$ numbers. r is called the **rank**. The smaller r is, the fewer numbers we need.

A small example with rank 1

$$B = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \quad A = \begin{pmatrix} 1 & 0 & -1 \end{pmatrix}, \quad BA = \begin{pmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 3 & 0 & -3 \end{pmatrix}$$

This 3×3 matrix has 9 numbers, but it is really decided by only $3 + 3 = 6$ numbers: every row is a multiple of the same pattern $(1, 0, -1)$. That is what "low-rank" means: the change covers the whole matrix, but it has only a few "directions".

Now take the real qkv layer. The original matrix has $768 \times 2304 = 1{,}769{,}472$ numbers. LoRA with $r = 8$ needs only $8 \times (768 + 2304) = 24{,}576$ numbers, about 1.4% of the original.

How it works (step by step)

1. **Freeze**: none of the backbone's original parameters are updated.
2. **Insert**: next to the four linear layers (qkv, proj, fc1, fc2) in each of the 12 blocks, add a pair of matrices. A is $r \times d_{\text{in}}$ and starts random. B is $d_{\text{out}} \times r$ and **starts at 0**. We use $r = 8$.
3. **Forward pass**: $y = W_0 x + \frac{\alpha}{r} BAx$. Because $B = 0$ at the start, the extra term is 0. So at the start of training, the model is **exactly the same** as the pretrained model, and it is not damaged right away.
4. **Scaling**: $\alpha = 16$ and $r = 8$, so the extra term is multiplied by $\alpha/r = 2$. With this scaling, we do not need to retune the learning rate when we change r.
5. **Training**: only A, B and the head are updated. We use a learning rate of 10^{-3}, weight decay 10^{-4}, and 3 epochs.
6. **Merging**: after training, compute $W = W_0 + \frac{\alpha}{r} BA$, then delete A and B. The model goes back to its original structure.

The LoRA formula and merging

$$y = W_0 x + \frac{\alpha}{r} BAx = \left(W_0 + \frac{\alpha}{r} BA \right) x = Wx$$

- x: the input of this layer (size d_{in}); y: the output (size d_{out}).
- W_0: the frozen pretrained weights; $\Delta W = BA$: the learned change.
- Ax: first squeeze the input down to only $r = 8$ numbers. $B(Ax)$: then expand it back to d_{out} numbers.
- The middle step of the equation shows that the two matrix multiplications can first be merged into one matrix W. After merging, only one matrix multiplication is needed, so it is as fast as the original model.

How we get 1,179,648 parameters

Each linear layer adds $r \times (d_{\text{in}} + d_{\text{out}})$ parameters:

Layer	Calculation ($r = 8$)	Parameters
qkv ($768 \to 2304$)	$8 \times (768 + 2304) = 8 \times 3{,}072$	24,576
proj ($768 \to 768$)	$8 \times (768 + 768) = 8 \times 1{,}536$	12,288
fc1 ($768 \to 3072$)	$8 \times (768 + 3072) = 8 \times 3{,}840$	30,720
fc2 ($3072 \to 768$)	$8 \times (3072 + 768) = 8 \times 3{,}840$	30,720
One block		98,304
12 blocks	$12 \times 98{,}304$	**1,179,648 (1.375%)**

The reference LoRA in the brief is only on qkv: $12 \times 24{,}576 = 294{,}912$ parameters, which is 0.344%.

Example

Everyday analogy

You want to turn a very thick textbook (W_0) into a new edition. Full fine-tuning rewrites the whole book page by page. LoRA leaves the book alone and sticks a few sticky notes (A, B) next to each chapter, saying "change this chapter like this". There are few sticky notes, but every chapter has some, so the meaning of the whole book changes. When the book is printed, the notes are printed straight into the text (merging). Readers get a normal book and do not need to flip between the text and the notes, so their reading speed stays the same.

We tried three changes to LoRA:

- Halve the learning rate ($10^{-3} \to 5 \times 10^{-4}$): 68.57% $\to$ 68.45%, a change of -0.12, within noise. So the learning rate was not the problem.
- Extend LoRA from qkv only to all four linear layers: 68.57% $\to$ 69.45%, a change of $+0.88$. The MLP holds two thirds of each block's weights, and before this change they were not adjusted at all. (Both of these tests used the LP head.)
- Train for 6 epochs instead of 3 (prototype head): 71.98% $\to$ 71.52%, a change of -0.46. The training loss fell from 1.95 to 1.70, while the holdout accuracy fell too. That is overfitting, so the final model uses 3 epochs.

The final LoRA model: holdout 71.98%, CodaBench validation set 71.90%, test set 71.67%, which is 3.6 points above the reference LoRA (68.3%); 0.54 GPU-hours; a latency of 7.61 ms per photo after merging.

Why LoRA needs more training memory than full fine-tuning

Intuition says that fewer trained parameters should mean less memory. But LoRA actually uses 19.9 GB, more than the 18.9 GB of full fine-tuning. Here is why:

- During training, most of the memory does not go to the parameters themselves. It goes to the **intermediate results** (called **activations**) that are saved for the backward pass.
- To compute the gradient of A, the model must save the input x of every linear layer. That is as much as full fine-tuning saves. LoRA also saves the intermediate results of 96 extra small matrix multiplications (12 blocks $\times$ 4 layers $\times$ 2 matrices).
- What full fine-tuning adds is mainly the optimizer state: the gradients of 85.8M parameters and Adam's two running averages. That is about 12 bytes per parameter, only about 1 GB in total.

So: **training memory depends on how far back the gradients must flow and how many intermediate results must be saved, not on how many parameters are trained**. Partial fine-tuning with K = 4 only sends gradients back through 4 blocks, so it uses only 7.0 GB. LoRA is also the slowest to train (888 images

per second).

Pros and cons, and when to use it

Pros: few parameters (1.38%); each task's checkpoint is only 35.5 MB (30.8 MB of it is the head); it can be merged, so inference has zero extra cost; and it is our most accurate PEFT method. Cons: in our setup, its training memory and speed are no better than full fine-tuning; it is about 1.4 points below full fine-tuning; and it overfits with longer training. When to use it: when one backbone must serve many tasks, each task must store little, and inference must not get slower.

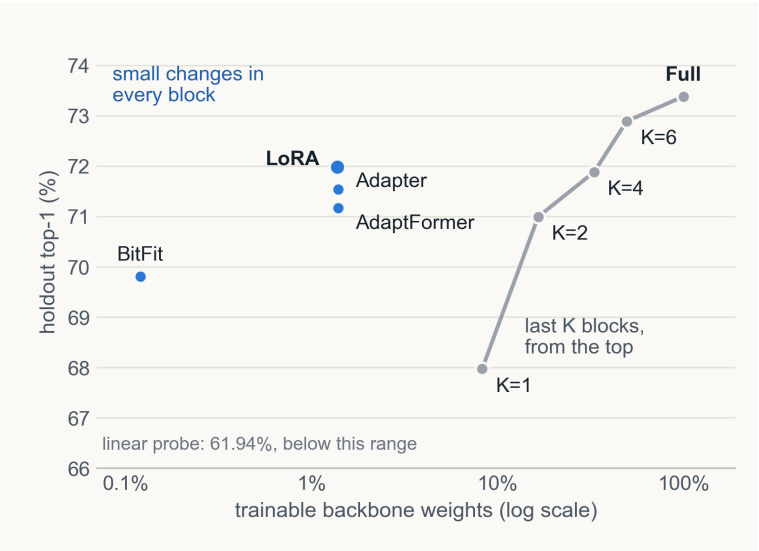
Why LoRA gets worse with longer training: an untested guess

LoRA has 8.87M trainable parameters in total, and 7.69M of them (87%) are the head. In our LoRA recipe, the head's learning rate is 10^{-3} (only 10^{-4} in full fine-tuning), its weight decay is only 10^{-4} (0.05 in full fine-tuning), and there is no drop path (0.1 in full fine-tuning). So our guess is this: with longer training, it is mainly the head that memorizes the 45 photos, the same kind of problem as with the LP head. How to test it: train LoRA for 6 epochs and lower only the head's learning rate to 10^{-4}. We have prepared the notebook (methods/10_lora_head_lr.ipynb), but we have **not run it yet**, so this is only a guess.

How it differs from the other methods

- Compared with Adapter and AdaptFormer: about the same number of parameters, and 0.44 and 0.81 points more accurate (the gap to the Adapter is within noise). The key difference is that LoRA can be merged, so its inference is not slower.
- Compared with BitFit: BitFit only shifts the outputs, while LoRA can change the "directions" of the weight matrices themselves. So LoRA is more powerful, by 2.2 points.
- Compared with full fine-tuning: it reaches 98% of the accuracy while training only 1.38% of the parameters. But in our task, its training is not cheaper.

2.3.8 All seven together: where you train matters more than how much



The figure from slide 5. The x-axis is the share of backbone parameters that are trained (log scale); the y-axis is holdout accuracy. The gray line is "train whole blocks from the top"; the blue points are "small changes in every block".

The three most important conclusions of this section

1. Where you train matters more than how much you train. The gray line (partial fine-tuning) climbs slowly to the right: training 8% gives only 68%, and training 50% gives only 72.9%. The blue points (small changes in every block) all sit above and to the left of the gray line. LoRA with 1.38% matches K = 4 with 33%, and BitFit with 0.12% beats K = 1 with 8%. Changing every layer a little works better than changing a few layers a lot.

2. Training memory follows "how far the gradients must flow". Methods that train only the top few layers use the least memory (K = 4 needs only 7.0 GB). When every layer has trainable parameters, the gradients must flow all the way to the first layer, so memory is large (about 12.8 GB for BitFit, Adapter and AdaptFormer, and 19.9 GB for LoRA).

3. Whether a method can be merged decides the inference speed. LoRA and BitFit do not change the structure after training, so inference is not slower. The two adapters add layers that cannot be merged, so each prediction is 2–3 ms slower (about 30%–40%).

2.4 How training works

The earlier sections covered "which parameters to train". This section covers "how to train them". We use our two final models as the running example: the full fine-tuning model (1019_1319_1_full) and the LoRA model (1019_1319_3_lora).

2.4.1 What happens in one training step

1. **Take a batch of photos**: 256 photos at a time (the **batch size** is 256), together with their labels.
2. **Data augmentation**: each photo is randomly cropped and randomly flipped left to right (see "Data augmentation" later in this section).
3. **Forward pass**: the photos go through the model, which gives 10,000 scores (logits) for each photo.
4. **Compute the loss**: compare the scores with the true labels. The result is one number that says "how wrong" the model is. This number is the **loss**.
5. **Backward pass**: starting from the loss, compute the **gradient** of each trainable parameter, that is, "which way to change this parameter so that the loss goes down".
6. **Gradient clipping**: if all the gradients together are too large, shrink them in proportion (see "Gradient clipping" later in this section).
7. **Update the parameters**: the optimizer, AdamW, moves each parameter a small step in the direction that lowers the loss, using the current learning rate.

One round of this is called a **step**. Using all the training photos once is called an **epoch**.

How many steps and epochs

Full fine-tuning: 450,000 photos ÷ 256 ≈ 1,757 steps per epoch, so 8 epochs give **14,056 steps** in total.
LoRA: 430,000 photos (43 per class) ÷ 256 ≈ 1,679 steps per epoch, so 3 epochs give **5,037 steps** in total.

2.4.2 Logits, softmax and the cross-entropy loss

Softmax: turning scores into probabilities

$$p_c = \frac{e^{z_c}}{\sum_{j=1}^{K} e^{z_j}}$$

- z_c: the score (logit) of class c; $K = 10{,}000$ is the number of classes.
- e^z: the exponential function. It turns any score into a positive number, and a slightly higher score gives a much larger result.
- The denominator adds up all the classes, so that all the p_c add up to 1.

The cross-entropy loss

$$L = -\ln p_y$$

y is the correct class, and p_y is the probability that the model gives to the correct class. The closer p_y is to 1, the closer the loss is to 0. The smaller p_y is, the larger the loss. It measures "how sure the model is about the correct answer".

A simplified example with only 3 classes (no label smoothing)

The model gives the three classes the scores $(2.0,\ 1.0,\ 0.1)$.

- $e^{2.0} \approx 7.39$, $e^{1.0} \approx 2.72$, $e^{0.1} \approx 1.11$; the sum is ≈ 11.21.
- Probabilities: $7.39/11.21 \approx 0.66$, $2.72/11.21 \approx 0.24$, $1.11/11.21 \approx 0.10$.
- If the correct answer is class 1: $L = -\ln 0.66 \approx 0.42$ (a small loss, a good answer).
- If the correct answer is class 3: $L = -\ln 0.10 \approx 2.31$ (a large loss, a wrong answer).

If the model is purely guessing, every one of the 10,000 classes has the probability $1/10{,}000$, and the loss is $\ln 10{,}000 \approx 9.21$. This is where the random head's starting loss (9.28) comes from.

Training losses can only be compared with each other

The starting losses in Section 2.2 (LP head 2.29, prototype head 4.39, random head 9.28) are the **average training losses** over the first 100 steps of LoRA training. They include the effects of label smoothing and data augmentation. So do not convert them into "the probability of the correct class". Only compare them with each other. 9.28 is about random guessing ($\ln 10{,}000 = 9.21$). 4.39 means the head roughly knows what each class looks like but is still very unsure, so the gradients are strong. 2.29 means the head has already been fitted on these training photos, so the gradients are weak.

2.4.3 Label smoothing 0.1: do not let the model be too sure

Plain cross-entropy wants the probability of the correct class to be 1 and all other classes to be 0. **Label smoothing** makes this target a bit "softer". The target for the correct class is $1 - \varepsilon + \varepsilon/K$, and the target for each other class is ε/K. We use $\varepsilon = 0.1$ and $K = 10{,}000$, so the target for the correct class is 0.90001, and the target for each other class is 0.00001.

Cross-entropy with label smoothing

$$L = (1 - \varepsilon)\,(-\ln p_y) + \varepsilon \cdot \frac{1}{K} \sum_{k=1}^{K} (-\ln p_k)$$

The first term: most of the weight (0.9) still asks for a high probability on the correct class. The second term: a small part of the weight (0.1) asks the model to keep a little probability on every class, and not to push the others down to 0.

Continuing the 3-class example

Let $\varepsilon = 0.1$ and $K = 3$, and let the correct class be class 1. The target becomes $(0.933,\ 0.033,\ 0.033)$. The loss is $0.933 \times 0.42 + 0.033 \times 1.42 + 0.033 \times 2.31 \approx 0.52$. That is a bit larger than the 0.42 without smoothing, because the loss no longer only rewards "being very sure about the correct answer". It also punishes "pushing the other classes down too hard".

Why we use it: we have only 45 photos per class, so the model easily becomes too sure about the training photos, that is, it memorizes them. Label smoothing leaves the model a little room. It is a mild form of **regularization** (any method that helps prevent overfitting). It also explains why the training loss never drops to 0: even when every answer is right, the smoothed loss has a lower limit.

2.4.4 Backpropagation and gradients: how the model knows what to change

The gradient tells us: if a parameter grows a little, does the loss go up or down, and how fast? If we move the parameter a small step against the gradient, the loss goes down. This is called **gradient descent**.

Everyday analogy

You are on a hillside in thick fog, and you want to reach the lowest point of the valley (the lowest loss). You cannot see far, so you feel with your feet which way the ground slopes down (the gradient). Then you take a small step that way (the learning rate decides the step size), one step at a time.

Backpropagation starts from the loss and uses the chain rule to compute the gradients layer by layer, going backwards. Only **trainable** parameters need gradients. But to compute the gradients of parameters in the lower layers, the gradient signal must pass through all the layers above them. This is why "how far the gradients must flow" decides the training memory.

2.4.5 The AdamW optimizer and weight decay

What AdamW does

The simplest gradient descent is "parameter = parameter − learning rate × gradient". Adam is smarter:

- It remembers the "average direction" of the gradients (the momentum, written m). This makes the updates smoother, so one or two noisy steps do not push it off course.
- It remembers the average size of the squared gradients (written v), and it sets the step size for each parameter. Parameters that always have large gradients take smaller steps. Parameters whose gradients are always small take larger steps.

Weight decay

At every step, pull each parameter a little toward 0: parameter ← parameter − learning rate $\times\lambda\times$ parameter. This keeps the parameters from growing too large. The model stays "simpler" and memorizes less easily. The "W" in AdamW means that this term is done separately from the Adam update (**decoupled weight decay**). Our settings: full fine-tuning uses $\lambda = 0.05$ (the biases, the normalization parameters, the [CLS] token and the position embeddings get no decay); LoRA uses $\lambda = 10^{-4}$.

AdamW in a simplified form

$$\theta \leftarrow \theta - \eta \left(\frac{m}{\sqrt{v} + \epsilon} + \lambda\, \theta \right)$$

θ: a parameter; η: the learning rate; m: the moving average of the gradient; v: the moving average of the squared gradient; ϵ: a tiny number that prevents division by 0; λ: the weight decay factor. The first term in the brackets means "follow the gradient"; the second means "pull a little toward 0".

2.4.6 Learning rate: warmup first, then cosine decay

The **learning rate** is how big each step is. Ours changes during training:

1. **Warmup**: at the start, the learning rate rises in a straight line from near 0 to its maximum. At the very start, the model and the new head have not "settled in" with each other yet, and big steps can easily damage the pretrained features. Full fine-tuning warms up for 1 epoch; LoRA warms up for 0.3 epochs.

2. **Cosine decay**: after that, the learning rate goes down slowly to 0 along a cosine curve. Early on, big steps learn fast; later, small steps fine-tune the details.

Cosine decay after warmup

$$\eta(s) = \eta_{\max} \cdot \frac{1}{2}\left(1 + \cos\left(\pi \cdot \frac{s - s_{\text{warmup}}}{S - s_{\text{warmup}}}\right)\right)$$

s: the current step; S: the total number of steps; s_{warmup}: the number of warmup steps; $\eta_{\max}$: the maximum learning rate (10^{-4} for full fine-tuning, 10^{-3} for LoRA). Right after warmup, $\cos 0 = 1$, so the learning rate is at its maximum. At the last step, $\cos \pi = -1$, so the learning rate is 0.

LoRA's learning-rate schedule

LoRA has 5,037 steps in total and 1,679 steps per epoch, so the warmup is $0.3 \times 1{,}679 \approx 503$ steps. From step 0 to step 503, the learning rate rises from near 0 to 10^{-3}. Over the remaining 4,534 steps, it falls along the cosine curve. Halfway through (around step 2,770) it is 5×10^{-4}, and at the last step it is 0.

2.4.7 Layer-wise learning-rate decay 0.75 (full fine-tuning only)

In full fine-tuning, different layers use different learning rates:

$$\text{learning rate of layer } l = 10^{-4} \times 0.75^{\,13-l}$$

The head and the final LayerNorm count as layer 13 and get the full 10^{-4}. The last block (layer 12) gets 0.75 times that. The first block (layer 1) gets $0.75^{12} \approx 0.032$ times, only about 3%. The patch embedding (layer 0) gets $0.75^{13} \approx 0.024$ times.

Why: the lower layers learn general features such as edges, colors and textures. These are useful for the new task too, and changing them too much can hurt. The top layers are closer to the specific classes and need more change. This is the same intuition as partial fine-tuning, but gentler: every layer changes, and the size of the change shrinks from the top to the bottom.

2.4.8 Drop path 0.1 and gradient clipping 1.0

Drop path (stochastic depth)

During training, the attention branch and the MLP branch of each block are sometimes "skipped" completely, and only the residual shortcut is used. Following timm's default, the chance of skipping rises gradually from 0 in the first block to 0.1 in the last block. At test time, every block is used normally.

Everyday analogy

During practice, a sports team rests a few random players each time, and the rest must still play well together. Then on match day (test time), everyone plays, and no one relies too much on one teammate. Drop path stops the model from relying too much on any one layer. It is a regularization method that helps prevent memorization.

Gradient clipping

Treat all the gradients as one very long vector. If its length is above 1.0, shrink it in proportion down to 1.0. This stops one unusual batch from causing one huge step, and it makes training more stable.

2.4.9 bf16 mixed precision and batch size

bf16

Normal decimal numbers are stored with 32 bits (**fp32**). **bf16** (bfloat16) uses only 16 bits. It covers the same range of values as fp32, but with less precision. Doing most of the computing in bf16 halves the memory and runs faster, with almost no effect on accuracy. The A100 supports bf16, so all our training and latency tests use bf16. "Mixed precision" means that most of the computing is in bf16, while a few precision-sensitive parts still use fp32.

Batch size 256

Each step averages the gradient over 256 photos. A bigger batch gives a steadier gradient and better use of the GPU, but it needs more memory. We use 256 on the 80 GB A100.

2.4.10 Data augmentation: more variety from 45 photos

RandomResizedCrop, scale = (0.25, 1)

Each time a training photo is read, a random region is chosen. Its area is 25% to 100% of the original photo, and its position and aspect ratio are also random. This region is then resized to 224 × 224. So the same photo is cropped differently in each epoch.

Horizontal flip

With a 50% chance, the photo is flipped left to right. A bird facing left and a bird facing right are in the same class, so flipping does not change the class.

Why: we have only 45 photos per class. If the model saw exactly the same photos every time, it would memorize them quickly. Augmentation makes every view a little different, which forces the model to learn the "real features". At test time there is no random augmentation: the photo is resized so that its short side is 248 pixels, and then the center 224 × 224 is cut out.

A side effect of random cropping

Random cropping often takes only part of the photo and enlarges it to 224. So during training, objects look **bigger than they really are**. At test time, the model sees the center crop of the whole photo, so objects look smaller in comparison. This "objects are different sizes in training and in testing" effect is the reason why, in Section 2.6, "testing with larger images is more accurate".

2.4.11 The two final models side by side

Setting	Full fine-tuning (1019_1319_1_full)	LoRA (1019_1319_3_lora)
What is trained	backbone 85.8M + head 7.69M	LoRA 1,179,648 + head 7.69M; backbone frozen
Head starting point	prototype head × 40	prototype head × 40
Training data	all 45 photos per class	43 photos per class (2 kept for the holdout)
Loss function	cross-entropy, label smoothing 0.1	same
Optimizer	AdamW, weight decay 0.05, gradient clipping 1.0	AdamW, weight decay 10^{-4}, gradient clipping 1.0
Learning rate	10^{-4}, times 0.75 for each layer further down	10^{-3}, the same for all layers
Regularization	drop path 0.1	no drop path
Training length	8 epochs: 1 warmup epoch + cosine	3 epochs: 0.3 warmup epochs + cosine
Other	bf16, batch 256, random crop + flip	same
Steps and cost	14,056 steps; 1.03 GPU-hours; 18.9 GB	5,037 steps; 0.54 GPU-hours; 19.9 GB
Inference	flip TTA: 14.95 ms per photo, 1,919 photos per second	merged: 7.61 ms per photo, 3,869 photos per second
Results	validation set 76.22%, test set 76.09%	validation set 71.90%, test set 71.67%

2.5 Evaluation: how to tell if a model is good

2.5.1 Accuracy: top-1 and top-5

- **Top-1 accuracy**: the share of photos where the model's highest-scoring class is the correct answer. This is the main metric of the project, and the leaderboard uses it.
- **Top-5 accuracy**: the share of photos where the correct answer is one of the 5 classes with the highest scores. It is always higher than top-1. For example, on the holdout, our LoRA model gets 71.98% top-1 and 89.07% top-5.

The validation set has exactly 5 photos per class, so the classes are perfectly balanced. So top-1 accuracy is also "the average of the accuracies of all the classes".

2.5.2 Three datasets: holdout, validation set, test set

Dataset	Size and labels	Use
Holdout (we set it aside ourselves)	2 per class, 20,000 photos, labels known	daily experiments, comparing methods, tuning settings; can be used at any time
CodaBench validation set (val)	5 per class, 50,000 photos, labels hidden	Phase 1: upload predictions, and the server gives a score
CodaBench test set (test)	10 per class, 100,000 photos, labels hidden	Phase 2: the final result; only 2 submissions in total, and they must use the same model as Phase 1

Why we keep our own holdout: the labels of the validation and test sets are hidden, so they can only be scored by uploading to the server. Uploading every experiment would be slow, and it would also mean "tuning on the exam questions". So we took 2 photos per class out of the training set and did not train on them. They are our own "mock exam". Its results are very close to the server's: full fine-tuning 72.31% against 72.57%, and LoRA 71.98% against 71.90%, within 0.3 points. That is why many comparisons in this guide use holdout numbers.

Always say which dataset a number comes from

The same model gets different numbers on different datasets. For example, LoRA gets 71.98% on the holdout, 71.90% on the validation set and 71.67% on the test set. For full fine-tuning, "73.38%" is a 3-epoch holdout result and "76.22%" is the final model's validation result. These two cannot be compared directly.

2.5.3 Cost metrics: parameters, latency, throughput, GPU-hours, memory

The project asks every method to report four metrics: top-1 accuracy, trainable parameters, inference latency and training GPU-hours. We also report peak memory.

- **Trainable parameters**: the number of parameters updated during training. Following the brief, this does not count the head (for example, LoRA has 1,179,648, or 1.375%).
- **Latency** (batch size 1): how many milliseconds it takes to process one photo. We count only the model's forward pass, in bf16, and take the average of 100 runs. For example: full fine-tuning 7.78 ms, LoRA after merging 7.61 ms, Adapter 9.74 ms.
- **Throughput** (batch size 256): how many photos per second the model can process. Latency asks "how fast is one photo?"; throughput asks "how many photos per second in a large batch?".
- **GPU-hours**: the number of GPUs $\times$ the number of hours used. We use one A100, so this is simply the number of hours of training. The time counted includes: computing features for the prototype head (about 6 minutes), all the training steps, and the holdout evaluation after each epoch.
- **Peak memory**: the most GPU memory that PyTorch used at any point during training.

How the budget is converted

The project allows at most "2 hours $\times$ 48 GB of GPU memory" per training run, that is, $2 \times 48 = 96$ GB·hours, with $\pm 10\%$ allowed. We use an 80 GB A100, so $96 \div 80 = 1.2$ hours (1.32 hours with the extra 10%). The final full fine-tuning model used 1.03 hours and LoRA used 0.54 hours. Both are within the budget.

2.5.4 How to judge whether a result is good

1. **Compare with the reference values**: the brief gives reference results. On the validation set, our final models are 5.4 points above the reference full fine-tuning (76.22% against 70.8%), and 3.6 points above the reference LoRA (71.90% against 68.3%).
2. **Check whether a gap is bigger than the noise**: the holdout has 20,000 photos, so the accuracy itself wobbles a little by chance. We ran each setting only once (seed 0). So gaps smaller than about 0.3 points should be treated as noise, and we cannot say which setting is better.
3. **Check that it fits the budget**: however accurate a result is, it does not count if it goes over the budget.
4. **Check that the test and validation results agree**: for both final models, the test result is within 0.25 points of the validation result. This shows the models did not "overfit" the validation set, so the results can be trusted.
5. **Look for signs of overfitting**: if the training loss goes down but the holdout accuracy also goes down, the model is memorizing the training photos. We saw this sign when LoRA went from 3 to 6 epochs, and in the LP head experiments.

Where the 0.3-point noise comes from

Accuracy can be seen as 20,000 "right or wrong" samples. Its **standard error** is:

$$\text{SE} = \sqrt{\frac{p\,(1-p)}{n}} = \sqrt{\frac{0.72 \times 0.28}{20{,}000}} \approx 0.0032 = 0.32 \text{ points}$$

$p \approx 0.72$ is the accuracy, and $n = 20{,}000$ is the number of holdout photos. In other words, even if the model does not change at all, a different set of holdout photos could move the accuracy up or down by about 0.3 points. So small changes like 0.12 or 0.26 should not be taken seriously, while gaps like 2.5 or 3.1 are far above the noise and can be trusted.

2.6 Test-time augmentation (TTA) and input resolution

2.6.1 Flip TTA (flip test-time augmentation)

What it is

Test-time augmentation: at prediction time, feed the model both the photo and its left-right mirror image. Add up the two sets of 10,000 scores, then pick the class with the highest total.

Why it helps

The model judges a photo and its mirror image a little differently. Adding the two results is like letting two "slightly different views" vote, so some random mistakes cancel out.

The benefit and the cost

On the validation set it adds 0.27 points (72.57% → 72.84%) and changes 4.6% of the predictions. But each photo is processed twice, so the inference time doubles (14.95 ms per photo for the final full fine-tuning model). Only the final full fine-tuning model uses TTA; the final LoRA model does not.

Everyday analogy

In an exam, you read a question once from the front, then look at it again from another angle, and combine both judgments. You are less likely to misread it. The cost is that each question takes twice as long.

2.6.2 Input resolution: the biggest gain, found after the deadline

What it is

Enlarge the input images from 224 pixels to 288 or 320 pixels. The model's structure and parameters stay exactly the same.

Why the model can handle larger images with no new parameters

- The patch size is still 16 pixels, so a larger image gives more patches. 288 pixels gives $18 \times 18 = 324$ patches, plus [CLS], so 325 tokens in total. 320 pixels gives $20 \times 20 = 400$ patches, so 401 tokens in total. (224 pixels gives 197.)
- All the linear layers (qkv, proj, fc1, fc2) work on each token on its own, so the same parameters still work with more tokens.
- The only part tied to the number of patches is the position embedding. It was originally a 14×14

grid. We use **interpolation** to stretch it to 18×18 or 20×20. This is like enlarging a small map, where the missing values are estimated from their neighbors. It adds no parameters.

- The cost is computing. Going from 197 to 325 tokens takes about 1.7 times the computing, and training memory grows from 18.9 GB to 30.2 GB.

Why larger images are more accurate: FixRes

Section 2.4 explained that random cropping in training makes objects look bigger than they are, while the center crop at test time makes objects look smaller in comparison. Touvron et al. pointed out this "object size mismatch between training and testing" in their 2019 FixRes paper. Using slightly larger images at test time makes the object sizes match training again.

Our results (all from 3-epoch holdout experiments)

- Train at 224, test at 224: 73.38% (the baseline).
- The same model, only tested at 288: 75.50% (+2.1), with no retraining at all.
- Train at 288, test at 288: 77.23% (+3.9), using 0.68 GPU-hours, still within the budget, but it needs 30.2 GB of memory.
- Train at 288, test at 320: 77.78% (+4.4), the best combination. This also matches FixRes: a test size a bit larger than the training size works best.

These results are not our official results

The resolution experiments were done **after** the deadline and only have holdout results. Training the final recipe (8 epochs, all data) at 288 pixels takes 1.61 GPU-hours, which is over the 1.2-hour budget, and that model has **no test-set result**. Our official results come only from the two Phase 1 models. On the slides, this is presented as "a lesson learned after the deadline": we studied the adaptation methods early, but we looked at the input itself only late.

2.7 Summary tables

2.7.1 The seven adaptation methods

All the numbers come from the same protocol: prototype head, 3 epochs, holdout, A100. Trainable parameters do not include the head.

Method	What it trains	Share	Holdout	Memory	Latency	Merge?
Linear probe	only the head	0%	61.94%	3.2 GB	6.96 ms	–
Partial, K = 4	the last 4 blocks	33%	71.88%	7.0 GB	7.41 ms	–
Full fine-tuning	all parameters	100%	73.38%	18.9 GB	7.78 ms	–
BitFit	only the biases	0.12%	69.81%	12.7 GB	7.33 ms	yes
Adapter	bottleneck module in series	1.39%	71.54%	12.8 GB	9.74 ms	no
AdaptFormer	bottleneck branch in parallel	1.39%	71.17%	12.8 GB	10.23 ms	no
LoRA	low-rank matrices A, B	1.38%	71.98%	19.9 GB	7.61 ms	yes

("Merge?" only matters for methods that add new parameters. The linear probe, partial fine-tuning and full fine-tuning never change the model's structure, so their inference does not get slower either. BitFit changes the existing biases, so it adds no structure either.)

Method	Main pro	Main con
Linear probe	the fastest and cheapest (0.10 GPU-hours, 3.2 GB)	the features do not change; low accuracy ceiling
Partial fine-tuning	the cheapest to train; no inference cost	the lower layers do not change; many parameters (8%–50%); diminishing returns
Full fine-tuning	the most accurate; improves with longer training	stores a whole model per task; large memory
BitFit	very few parameters (102,912); no inference cost	limited power; 2.2 points below LoRA
Adapter	accuracy close to LoRA; fast training, little memory	cannot be merged; inference about 2.4 ms slower
AdaptFormer	does not change the original MLP output; fast training	cannot be merged; inference about 2.9 ms slower
LoRA	the most accurate PEFT method; can be merged; zero inference cost	training memory and speed are no better than full fine-tuning; overfits with longer training

2.7.2 The three head initializations

	Random head	LP head	Prototype head
Accuracy before training	close to 0	62.1%	44.0%
LoRA, 3 epochs (holdout)	71.95%	69.45%	72.00%
Fitted on the training photos?	no	yes	no
Conclusion	poor start, but it ends level with the prototype head	the highest start, the worst end (memorization)	a sensible start, learns fast; our choice

The four most important conclusions of this chapter

1. The head's starting point matters most. The common LP-FT start made the results worse by 2.1–3.1 points in all three methods we tested. The LP head was fitted on the same training photos, and this leads to memorization. The prototype head (weight imprinting) has not fitted any photo, so it starts at a sensible point and learns fast.

2. Where you train matters more than how much. Small changes inside every block (LoRA 1.38%, BitFit 0.12%) beat big changes in the top few layers (partial fine-tuning, 8%–33%).

3. Training memory follows the gradients; inference speed depends on merging. Fewer parameters do not mean less memory (LoRA uses 19.9 GB). LoRA can be merged, so its inference is not slower; the adapters cannot be merged.

4. Full fine-tuning is the most accurate; LoRA is the best efficient method. Full fine-tuning keeps improving with longer training (8 epochs). LoRA is best at 3 epochs and overfits with longer training (we guess the head is the cause, but we have not tested it yet).

Chapter 3 Key terms (glossary)

This chapter collects the technical terms from the slides, the speaker notes and the report in one place. They are sorted by topic into 5 groups. Each term has three columns:

- **Term**: the name you will hear in the presentation and in questions. The instructor will use these English names.
- **What it really means**: one or two plain sentences. No textbook definitions to learn by heart.
- **Everyday example / in our project**: a comparison from daily life, or where the term shows up in our project and which number goes with it.

On your first read, skim it quickly from top to bottom. When you prepare for the presentation, use it like a dictionary. Chapter 2 explains each method in full. Chapter 4 works through the formulas with examples.

How to use these tables

You do not need to learn everything at once. First read the bold term names in each table. For each one, check that you can explain it in one sentence of your own. If you cannot, read the second and third columns. Before the presentation, you should be able to explain at least these words without thinking: backbone, head, fine-tuning, LoRA, prototype, linear probe, holdout, overfitting, cross-entropy, GPU-hours.

Term	What it really means	Everyday example / in our project

3.1 Data and evaluation

Term	What it really means	Everyday example / in our project
dataset	A large set of images together with their correct answers. We use it to train a model and to check it.	Our data has 10,000 classes. The training set has 450,000 images.
class / label	A class is one kind of thing we want to tell apart. A label is the correct answer for one image: the class it belongs to.	10,000 classes. The class names are hidden; we only see numbers. The training set has labels; the labels of the validation and test sets are hidden.
fine-grained classification	A classification task where the classes differ only a little. For example, telling apart kinds of birds, not telling a bird from a car.	For example, telling apart two birds or moths that look almost the same. At the same time, photos of one class can look very different: one bird or a whole flock, one flower or a whole bush.
training set	The data the model learns from. The model sees both the images and the answers.	450,000 images, exactly 45 per class.
holdout set	A small part of the training set that we keep aside. It is not used for training, only for scoring. The model has never seen it, so its score can be trusted.	2 images per class, 20,000 in total, chosen with a fixed random seed. All our comparisons use it. It stays within about 0.3 points of the CodaBench score.
validation set (val)	Data used to check the model and to choose settings.	The CodaBench val set: 50,000 images (5 per class). The labels are hidden; we see the score only after we upload predictions. Our final full model: 76.22% on val.
test set	Data for the final, one-time check that decides the ranking. You must not use it to tune settings.	100,000 images (10 per class). Our test scores: full fine-tuning 76.09%, LoRA 71.67%.
CodaBench	The online scoring platform the course uses. You upload a prediction file; it computes the accuracy and the ranking.	All our val and test scores come from it.
Phase 1 / Phase 2	The two stages of the competition. Phase 1 scores on val. Phase 2 scores on test, allows only 2 submissions, and must use the same model as Phase 1.	Both our test results come from the Phase 1 models `1019_1319_1_full` and `1019_1319_3_lora`.
leaderboard	The ranking of all teams' scores.	The instructor said the ranking is only a reference. Analysis, iteration, failed experiments and new ideas matter more.
top-1 accuracy	The share of images where the model's first answer is the correct one.	76.22% means: out of 100 images, about 76 are right.
top-5 accuracy	An image counts as correct if the right answer is anywhere in the model's top 5 guesses. It is never lower than top-1.	Like an exam that lets you write 5 possible answers. The report lists it as an extra metric.

Term	What it really means	Everyday example / in our project
overfitting	The model keeps getting better on the training data, but gets worse on data it has not seen.	LoRA trained for 6 epochs instead of 3: the training loss fell from 1.95 to 1.70, but the holdout accuracy dropped from 71.98% to 71.52%.
memorization	The model remembers the training images themselves, instead of learning rules that also work on new images. Overfitting often comes from memorization.	Like learning the answers to practice questions by heart: give a new question, and you are lost. With only 45 images per class, memorizing is easy.
generalization	The ability to do well on new data that the model has never seen. This is the real goal of training.	We only trust scores on the holdout, val and test sets, never the training loss.
noise (random error)	Small random ups and downs. They do not mean the model really got better or worse.	On the holdout, differences under 0.3 points count as noise. For example, the -0.12 from halving the learning rate is noise.
standard error	The typical size of the error when you estimate a number from a limited sample. More samples give a smaller error.	For 72% accuracy measured on 20,000 images: $\sqrt{0.72 \times 0.28/20{,}000} \approx 0.32$ percentage points. That is why we say that about 0.3 points is noise.
seed (random seed)	A number that controls a random process. With the same seed, you get the same random results again.	All our experiments use seed 0, and each setting ran only once. We list this as a limitation.

Term	What it really means	Everyday example / in our project

3.2 Model and architecture

Term	What it really means	Everyday example / in our project
neural network	A big function made of many simple layers of computation. Many adjustable numbers turn the input into the output.	Input: one image. Output: 10,000 scores, one for each class.
parameter / weight	An adjustable number inside the network. Training keeps adjusting these numbers so the answers get more accurate.	ViT-B/16 has 85,798,656 parameters (about 85.8M; M = million).
bias	The constant b that is added in a linear layer $y = Wx + b$. It shifts the whole result up or down a little.	BitFit trains only the biases: 102,912 of them in total.
linear layer (fully connected layer)	It multiplies the input vector by a matrix and adds a bias: $y = Wx + b$. Number of parameters = input size $\times$ output size + output size.	Each block has 4 of them: qkv, proj, fc1, fc2. fc1 is 768 $\rightarrow$ 3,072, with about 2.36M parameters.
activation function	A non-linear "switch" between linear layers. Without it, many linear layers stacked together would still act like one linear layer.	ReLU: negative numbers become 0, positive numbers stay the same. GELU is a smooth version of it. The ViT MLP uses GELU; the Adapter uses ReLU.
backbone	The large part of the network that "understands" the image and extracts features.	Our backbone is ViT-B/16, the one the course recommends. We never changed it.
head	The last layer, right after the backbone. It turns the features into one score for each class.	New head: 768 $\rightarrow$ 10,000, with 7,690,000 parameters, about 9% of the backbone.
feature	The backbone's "number description" of an image: a list of numbers (a vector). Similar images get similar features.	Each image gets one [CLS] feature of 768 numbers.
Transformer	A network design built around "attention". It was first used to translate text, and later also for images.	ViT is simply a Transformer used on images.
ViT / ViT-B/16 (Vision Transformer)	Cut the image into small squares and give them to a Transformer as a sequence of "words". B = Base (medium size); 16 = each square is 16 $\times$ 16 pixels.	We use timm's `vit_base_patch16_224.augreg2_in21k_ft_in1k`: trained first on ImageNet-21k, then on ImageNet-1k.
patch	One of the small squares that the image is cut into.	A 224 $\times$ 224 image becomes 14 $\times$ 14 = 196 patches.
patch embedding	The layer that turns each patch into a vector of 768 numbers (a 16 $\times$ 16 convolution with stride 16).	590,592 parameters (0.59M on the diagram).
token / embedding	A token is the basic unit a Transformer works on. An embedding is the list of numbers that represents it. For images, one token is the vector of one patch.	196 patch tokens + 1 [CLS] token = 197 tokens, each with 768 numbers. That is why the diagram says "197 $\times$ 768".

Term	What it really means	Everyday example / in our project
[CLS] token (class token)	An extra token added at the front. Its job is to collect information about the whole image. At the end, only this token is used for classification.	Like the person in a meeting whose job is to write the summary. The [CLS] vector of the last layer (768 numbers) goes into the head.
position embedding	Position information added to each token, so the model knows where each patch sits in the image.	197×768. At 288 pixels, the 14×14 grid is interpolated to 18×18, with no new parameters.
Transformer block	The repeating unit of a Transformer: attention + MLP, each with its own LayerNorm and residual connection.	ViT-B/16 has 12 identical blocks, each with 7,087,872 parameters (about 7.09M).
self-attention	Each token "looks at" all tokens and gives each one a weight by how related it is. Then it combines their information using those weights.	Like a meeting where everyone first listens to all speakers, then writes notes that focus on what matters most to them.
Q / K / V (query / key / value)	The three kinds of vectors in attention. Q says "what I am looking for"; K says "what I can offer"; V is "my actual content". The better a Q matches a K, the more of that V is used.	The qkv layer computes all three at once: $768 \to 2{,}304 = 3 \times 768$, about 1.77M parameters.
multi-head attention	Split attention into several groups that run side by side. Each group can focus on different relations. The results are joined at the end.	12 heads of 64 numbers each ($12 \times 64 = 768$). Attention itself has no parameters.
MLP (multilayer perceptron)	The two fully connected layers in a block. They make each token bigger, apply an activation function, then make it small again. They work on each token separately.	$768 \to 3{,}072 \to 768$, about 2/3 of a block's parameters.
LayerNorm (LN)	Brings the 768 numbers of each token to a similar scale, which makes training more stable. It has two sets of parameters: a scale and a shift.	Each LN has 1,536 parameters: 768 for the scale and 768 for the shift (β).
residual connection	Adds a layer's input directly to its output: $x + f(x)$. Information and gradients can take a "shortcut", so deep networks are easier to train.	Each block has two "+" signs.
logits	The raw scores from the head, before they become probabilities. The class with the highest score is the prediction.	10,000 logits for each image.
softmax	Turns a set of scores into probabilities that add up to 1. A higher score gets a higher probability, and gaps become bigger.	Scores (2, 1, 0) become probabilities of about (0.67, 0.24, 0.09).
prototype	The "average look" of a class. Average the (normalized) features of all images of that class to get one direction.	Each of the 10,000 classes gets a prototype of 768 numbers. We use them to initialize the head.
L2 normalization	Divide a vector by its own length, so its length becomes 1. Only the "direction" is kept.	(3, 4) has length 5. After normalization it is (0.6, 0.8).
cosine similarity	How close two vectors are in direction: 1 means the same direction, 0 means unrelated, -1 means opposite.	The prototype head's score is about $40 \times \cos$(image feature, class prototype).
temperature / logit scale	A number multiplied with the similarity. It controls how "sharp" the softmax probabilities are. Too small: all probabilities look alike. Too large: the model becomes overconfident.	We picked 40 out of 5, 10, 15, 20, 30, 40, 60 and 80, using the cross-entropy.

3.3 Training

Term	What it really means	Everyday example / in our project
training	Repeat "predict → measure how wrong it is → adjust the parameters" again and again, so the model gets more and more accurate.	Like doing an exercise, checking the answer, correcting yourself, then doing the next one.
forward pass	Compute from the input all the way to the output to get a prediction.	An image goes through the patch embedding, 12 blocks and the head, and comes out as 10,000 scores.
backward pass	Compute backward from the loss to find how each parameter should change (the gradients). For this, the in-between results of the forward pass must be stored.	How far back the gradients must travel decides how much memory you need. Partial fine-tuning with $K = 4$ goes back through only 4 blocks (7.0 GB); LoRA must go back to the first block.
gradient	Information that tells each parameter which way to change, and by how much, to make the loss smaller.	Like the slope under your feet when you walk down a hill: it shows which way is down.
loss function	A number that measures how wrong the predictions are. Smaller is better. Training tries to make it smaller.	Note: a low training loss does not mean a good model. See slide 8.
cross-entropy	The most common loss for classification: $-\log(\text{probability of the correct class})$. The lower the probability of the correct class, the bigger the penalty.	Probability 0.9 → loss 0.11; probability 0.1 → loss 2.30; pure guessing among 10,000 classes → $\ln 10{,}000 \approx 9.21$.
label smoothing	Do not ask the model to be 100% sure. The target becomes 0.9 for the correct class, and the other 0.1 is spread evenly over all classes. This stops overconfidence.	Both our final models use 0.1.
optimizer	The exact rule for updating the parameters from the gradients.	We use AdamW.
AdamW	One of the most common optimizers. It picks a suitable step size for each parameter by itself, and it handles weight decay separately (the W stands for "decoupled weight decay").	Both full fine-tuning and LoRA use it.
learning rate (LR)	How big each change to the parameters is. Too big breaks things; too small learns too slowly.	Full fine-tuning: 1e-4 (= 0.0001). LoRA: 1e-3 (= 0.001).
weight decay	At every step, gently pull the parameters a little toward 0. This keeps them from growing too big and helps against overfitting.	Full fine-tuning: 0.05. LoRA: 1e-4.
warmup	At the start of training, the learning rate rises slowly from very small to the target value. This avoids messing up the pretrained parameters right away.	Full fine-tuning warms up for 1 epoch, LoRA for 0.3 epochs.
cosine schedule	After warmup, the learning rate goes down to 0 along a cosine curve: slowly at first, faster in the middle, and slowly again at the end.	Like a runner who slows down gently before the finish line.
layer-wise LR decay	Layers closer to the input get a smaller learning rate. The lower layers learn general things like edges and textures, so they should not change much.	Multiply by 0.75 for each layer down. The first block gets only about 3% of the head's learning rate ($0.75^{12} \approx 0.032$).

Term	What it really means	Everyday example / in our project
drop path (stochastic depth)	During training, randomly "skip" the computation of some blocks and use only the residual shortcut. The model cannot depend on any single layer. It is a kind of regularization.	Full fine-tuning uses 0.1; LoRA does not use it.
regularization	The general name for all the methods that fight overfitting.	Weight decay, drop path, label smoothing and data augmentation all count.
gradient clipping	If the gradients are too big, scale them down. This stops one step from changing things too much.	The upper limit for the gradient norm is 1.0.
batch / batch size	A group of images that are computed together, and the number of images in that group.	Our batch size is 256.
step	One update of the parameters: after one batch is processed, the parameters change once.	The final full model took 14,056 steps; LoRA took 5,037.
epoch	One full pass over the whole training set.	450,000 images $\div$ 256 $\approx$ 1,757 steps per epoch. Full fine-tuning trains 8 epochs; LoRA trains 3.
data augmentation	During training, make random changes to the images, so the model sees more "versions" and memorizes less.	Like practicing the same question again, but worded differently.
RandomResized-Crop	Randomly cut out a part of the image (for us, 25% to 100% of its area), then resize it to 224 $\times$ 224.	The smaller the cut-out part, the bigger the object becomes. This is the reason behind FixRes later.
horizontal flip	Mirror the image from left to right.	Used randomly during training. Flip TTA at test time uses it too.
bf16 / mixed precision	Compute with a shorter, 16-bit number format. It is faster and uses less memory, and you lose very little accuracy.	All our training uses bf16, so we need a GPU that supports it, such as an A100, L4 or H100.
freeze	Keep some parameters unchanged during training.	LoRA freezes the whole backbone and trains only the A and B matrices and the head.
hyperparameter	A setting that people choose before training. It is not learned by training.	The learning rate, the number of epochs, the LoRA rank r and the temperature 40 are all hyperparameters.

Term	What it really means	Everyday example / in our project

3.4 Fine-tuning methods

Term	What it really means	Everyday example / in our project
pretraining	First train the model on a huge, general dataset, so it learns to "see" in general.	Our backbone was trained first on ImageNet-21k ("21k" means about 21,000 classes), then on ImageNet-1k.
ImageNet-21k / ImageNet-1k	Two famous, large image datasets. 21k means about 21,000 classes; 1k means 1,000 classes.	Our backbone was pretrained on 21k and then fine-tuned on 1k. This is exactly the version the course recommends.
in21k-only checkpoint	A ViT-B/16 trained only on ImageNet-21k, without the extra step on 1k. The course materials also mention it.	After the deadline we checked two of them: their linear probes reach 61.05% and 51.07%, both below the 61.94% of ours. So we did not switch.
AugReg	A recipe for training ViTs with stronger data augmentation and regularization (Steiner et al.).	The "augreg" in our backbone's name refers to it.
transfer learning	Using skills learned on one task for a new task.	Someone who can ride a bicycle learns to ride a motorbike faster.
fine-tuning	Take a pretrained model and keep training it on new data, so it fits the new task.	The core of this project: adapting ViT-B/16 to 10,000 new classes.
adaptation method	A concrete plan for "how to fine-tune": which parameters to train, and whether to add new modules.	We compared 7: linear probe, partial fine-tuning, full fine-tuning, BitFit, Adapter, AdaptFormer, LoRA.
full fine-tuning	Train all the parameters of the backbone.	100%, 85.8M. The most accurate: 73.38% on the holdout; the final model reached 76.22% on val.
partial fine-tuning	Train only the last K blocks (plus the final LayerNorm and the head). Everything below them is frozen.	$K = 1, 2, 4, 6$: holdout 67.97%, 70.99%, 71.88%, 72.89%.
linear probe (LP)	Freeze the whole backbone and train only the head. It "probes" how good the frozen features are on their own.	61.94% on the holdout, using only 0.10 GPU-hours.
zero-shot	Here it means no training at all: predict directly with the prototype head.	43.95% on the holdout (course reference: 43.7%).
LP-FT	A two-step recipe: first train a linear-probe head (LP), then fine-tune the whole model starting from it (FT) (Kumar et al., ICLR 2022).	We did this at first too. Later we found that in our setting it was worse for all three methods we tested, by 2.1 to 3.1 points.
PEFT	Parameter-efficient fine-tuning: adapt to a new task by training only very few parameters. It saves storage and lets one model serve many tasks.	We compared 4: BitFit, Adapter, AdaptFormer, LoRA.
BitFit	Train only all the biases (and the shift parameters of LayerNorm). All other weights are frozen.	With only 0.12% of the parameters, it reaches 69.81%.

Term	What it really means	Everyday example / in our project
Adapter	Add a small module in series after the MLP of each block: 768 $\rightarrow$ 64 $\rightarrow$ 768, with its own residual connection.	1.39% of the parameters and 71.54% on the holdout. But inference must compute two extra layers: the latency is 9.74 ms.
AdaptFormer	Put the same small module in parallel, next to the MLP. Its output is multiplied by a small factor $s = 0.1$ and added back.	71.17% on the holdout, 0.37 below the Adapter, which is close to noise. Latency: 10.23 ms.
bottleneck	First squeeze the size down, then expand it again (768 $\rightarrow$ 64 $\rightarrow$ 768). This does a transformation with very few parameters.	Like first writing a short outline of a book, then filling the book back in from the outline.
LoRA (Low-Rank Adaptation)	Freeze the original weights W, and learn a "low-rank" change $\Delta W = BA$ next to them. After training, merge the change into W.	LoRA on all 4 linear layers: 1,179,648 parameters (1.375%). Final model: 71.90% on val.
rank r / low-rank	A big matrix can be approximated as the product of two "thin" matrices. r is the size in the middle; it decides how many kinds of change can be expressed.	$r = 8$: a 768 $\times$ 768 change needs only 8 $\times$ (768 + 768) = 12,288 numbers.
LoRA alpha (α)	A scaling factor for the LoRA output. The actual multiplier is α/r.	$\alpha = 16$ and $r = 8$, so the multiplier is 2.
merging	After training, add the LoRA change back into the original weights: $W' = W + (\alpha/r)BA$. The model then has exactly the original structure, so inference is not slower.	After merging, LoRA's latency is 7.61 ms. The Adapter has a ReLU in the middle, so it cannot be merged this way.
weight imprinting	Without any training, "imprint" the average feature of each class directly into the weights of the head (Qi et al., CVPR 2018).	Our prototype head is this method. We did not invent the method; what is new is our finding.
head initialization	How the head's weights are set before training starts.	We compared three: random, linear probe and prototype. The linear probe was the worst.
feature distortion	The worry in the LP-FT paper: a random head gives messy gradients at the start, and these can damage the features learned in pretraining.	Our random-head control did not get worse (71.95% vs 72.00%), because our task needs the features to change anyway.

Term	What it really means	Everyday example / in our project

3.5 Experiments and engineering

Term	What it really means	Everyday example / in our project
GPU / A100 (graphics processing unit)	A GPU is good at doing many identical calculations at the same time, so it does most of the work when training neural networks. The A100 is NVIDIA's data-center GPU.	We use an A100 with 80 GB of memory on Colab.
timm	A popular PyTorch library of image models, with many pretrained models and tools.	Our ViT-B/16 weights come from timm. When we change the resolution, timm interpolates the position embeddings.
Colab	Google's online notebook service, where you can rent a GPU when you need one.	We trained on Colab A100s and saved the results to Google Drive.
VRAM / peak memory	The memory on the GPU. It holds the model parameters, the gradients and the in-between results. Peak memory is the highest amount used during training.	Full fine-tuning 18.9 GB, LoRA 19.9 GB, training at 288 pixels 30.2 GB.
GPU-hours	Number of GPUs $\times$ hours used. It measures how much computing one training run takes.	Final full model: 1.03; LoRA: 0.54.
compute budget	The course allows at most 2 hours $\times$ 48 GB per training run. Converted by memory, that is 1.2 hours on an 80 GB A100 ($\pm$10% allowed).	Everything counts toward the budget: extracting features, training, and the holdout evaluation after each epoch.
latency	How long it takes to process one image. It is usually measured with batch size 1 (one image at a time), which is closest to real-time use.	LoRA after merging: 7.61 ms; Adapter: 9.74 ms; the full model with flip TTA: 14.95 ms.
throughput	How many images can be processed per second. During training it is called the training speed; for inference it is measured with a large batch.	Inference: the final LoRA model does 3,869 images/s. Training: full fine-tuning 1,139 images/s, LoRA 888 images/s.
TTA (test-time augmentation)	At test time, make a few versions of the same image and combine the results.	Flip TTA: add the logits of the image and of its mirror image. +0.27 points, but everything is computed twice, so the latency doubles.
ablation	Change one setting at a time and keep everything else the same, to see how much that one setting really matters.	The bar chart on slide 9 is an ablation.
baseline	A reference result to compare against.	The course gave 43.7%, 70.8% and 68.3%. We reproduced them first, all within 0.3 points.
protocol	A fixed set of settings shared by all compared experiments, so their results can be compared directly.	Prototype head, 3 epochs, the same holdout, the same kind of GPU.
resolution / input size	The side length of the input image in pixels. Bigger means more detail, but also more computation.	224 $\rightarrow$ 288: the tokens go from 197 to 325, and the computation grows about 1.7 times.

Term	What it really means	Everyday example / in our project
FixRes	The mismatch between training and test resolution (Touvron et al., 2019): random crops make objects look bigger during training, so slightly bigger test images match better.	The same model trained at 224 scores 2.1 points higher when tested at 288.
interpolation	Estimating the values in between known points.	Stretch the 14×14 position embeddings to 18×18, like enlarging a small picture.
checkpoint	A saved file of the model's parameters.	A full model needs 374 MB per task; LoRA needs only 35.5 MB, of which 30.8 MB is the head.
run	One complete training experiment, with its own name and log.	`1019_1319_1_full` = run 1 of the batch started on October 19 at 13:19; the method is "full".
post-deadline analysis	Experiments done after the competition deadline. They only help us understand the problem; they do not change the results we already submitted.	The resolution experiment and the in21k-only backbone check. Both are on the holdout only, with no test score.

Chapter 4 Figures, formulas and architecture diagrams

This chapter is about "reading the figures". We take apart every figure, table and architecture diagram on the slides. For each one we explain what each part is, how data or information flows, why each part is there, and how the parts relate. Where there are numbers, we work through them with you. The last section collects all the formulas, explains every symbol, and adds a small calculation for each one.

What each method is, why it works, and its pros and cons are in Chapter 2. This chapter only helps you understand the figures and the formulas.

Four questions to ask about any figure

1. What do the x-axis and the y-axis (or each column of a table) show? What are the units?
2. What does each point, each line and each color stand for?
3. Who is compared with whom? Are all the other conditions the same?
4. What is the one-sentence conclusion? How does it connect to the slides before and after it?

If the instructor points at a figure and asks a question, go through these four questions in your head first. Then answer.

4.1 Slide 2: the data examples and the four numbers



The 16 training images on the left of slide 2: each row shows 4 photos of the same class

How to read this figure

There are 16 images in 4 rows and 4 columns. There is only one rule: **each row is one class**.

- Row 1 is one kind of bird: one seen through a telescope, a big flock in the sky, one on a feeder, one

on reeds.

- Row 2 is one kind of snake: on rock, on a road, on gravel, on fallen leaves.
- Row 3 is one kind of moth: on a wall, on a leaf, a close-up, on cloth.
- Row 4 is one kind of plant: a flower, a fruit, the whole bush, a single branch.

What this figure is telling us

With 16 images, it shows why the task is hard:

1. **Photos of the same class look very different.** One bird and a whole flock, one flower and a whole bush share almost nothing at the pixel level. Yet the model must put them into the same class. We call this large within-class variation.
2. **Different classes look very similar.** Many of the 10,000 classes differ only a little, such as two birds that look almost the same. This is what **fine-grained** means.
3. **Only 45 photos per class.** With so few photos that look so different, it is easy for the model to memorize each photo instead of learning the pattern of the class.

Everyday analogy

It is like learning to recognize a person from only 45 photos: some from the front, some from the side, some from far behind, and some group photos. To really know the person, you must find what all the photos have in common. If you only memorize the photos, you will not recognize the person in a new photo.

The four numbers on the right

Each number is followed by a "So:" line, which means "so this is what we do". This slide turns "understanding the problem" directly into "design choices":

Number	The fact it states	So we designed it this way
10,000	Fine-grained classes, and the class names are hidden	The pretrained features were not made for these differences. The features themselves must change; a new head alone is not enough
45	Training photos per class, 450,000 in total	Easy to memorize. Judge only by the score on the 20,000 held-out photos, not by the training loss
1.2 h	Budget per run: 1.2 hours on an A100-80GB (2 h $\times$ 48 GB, $\pm$10%)	Compare every idea inside the same budget, and count all GPU time
ViT-B/16	Trained on ImageNet-21k and then on ImageNet-1k, 85.8M parameters	Use the backbone that the course recommends, and never change it

Calculation: where the 1.2-hour budget comes from

The course rule is "2 hours $\times$ 48 GB of GPU memory". Think of it as a total amount of resources:

$$2\ \text{h} \times 48\ \text{GB} = 96\ \text{GB} \cdot \text{h}.$$

Our A100 has 80 GB of memory. The same amount of resources lasts

$$96\ \text{GB} \cdot \text{h} \div 80\ \text{GB} = 1.2\ \text{h}.$$

The rule allows $\pm 10\%$, so the maximum is $1.2 \times 1.1 = 1.32$ hours. Our two final models used 1.03 and 0.54 GPU-hours, both within 1.2 hours.

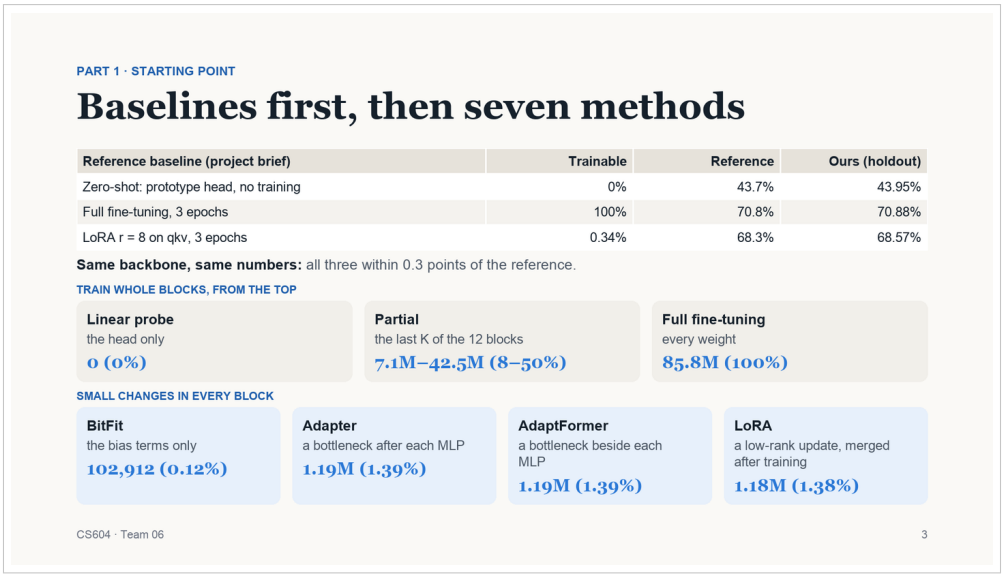
Calculation: the amount of data and the size of the head

- Training set: 10,000 classes $\times$ 45 photos = 450,000 photos.
- Holdout: 10,000 $\times$ 2 = 20,000 photos. The comparison experiments train on the remaining 10,000 $\times$ 43 = 430,000 photos.
- Validation set: 10,000 $\times$ 5 = 50,000 photos. Test set: 10,000 $\times$ 10 = 100,000 photos. The labels of both are hidden.
- New head: $768 \times 10{,}000 + 10{,}000 = 7{,}690{,}000$ parameters. That is $7{,}690{,}000 \div 85{,}798{,}656 \approx 9.0\%$ of the backbone. Each class has 769 numbers in the head (768 weights + 1 bias), but only 45 photos to learn them from.

Easy to get wrong

The "1.2 hours" counts GPU time, in three parts: extracting features with the frozen model at the start (about 6 minutes), the training itself, and the holdout evaluation after each epoch. Unpacking the data and writing the final prediction file do not count.

4.2 Slide 3: the baseline table and the two families of methods



Slide 3

The table at the top: first, reproduce the reference results

The table has four columns: the name of the reference experiment (from the course PDF); Trainable (how much of the backbone is trained); Reference (the score in the PDF); and Ours (holdout) (the score we got with our own code, on the holdout). The three rows compare:

- Zero-shot (prototype head, no training at all): 43.7% vs 43.95%;
- Full fine-tuning for 3 epochs: 70.8% vs 70.88%;
- LoRA ($r = 8$) on qkv only, for 3 epochs: 68.3% vs 68.57%.

All three are within 0.3 points.

Why we do this first

Before trying any new idea, we make sure that our pipeline is right: the data is read correctly, the evaluation is computed correctly, and the training settings match the reference. Only if the starting point can be trusted can we trust the score of every later improvement.

Everyday analogy

Before you weigh something, you check the scale. You put on a weight that you know is 1 kg, and the scale shows 1.00 kg. Now you can trust it for other things. The three reference experiments are our "known weights".

Both are "full fine-tuning for 3 epochs". Why 70.88% here but 73.38% on slide 5?

When we reproduced the reference results (September 28), we still used the common recipe of the time: first train a linear-probe head, then fine-tune (this is LP-FT). The seven-method comparison on slide 5 uses the prototype head instead, which we found later. The only difference between the two runs is the starting point of the head, and it is worth about 2.5 points. That is exactly the story of slides 6 to 8. The same holds for the reference LoRA row: its 68.57% also uses the linear-probe head.

The cards at the bottom: two families of methods

The seven methods form two families:

- **Family 1 (gray cards): train whole blocks, from the top down.** Linear probe (only the head, 0%), partial fine-tuning (the last K blocks, 8% to 50%), full fine-tuning (everything, 100%).
- **Family 2 (blue cards): make a small change in every block.** BitFit (only the biases, 0.12%), Adapter (a bottleneck in series after the MLP, 1.39%), AdaptFormer (a bottleneck in parallel next to the MLP, 1.39%), LoRA (a low-rank change that is merged after training, 1.38%).

The number on each card is "trained parameters (as a share of the backbone)". You can compute all of them yourself.

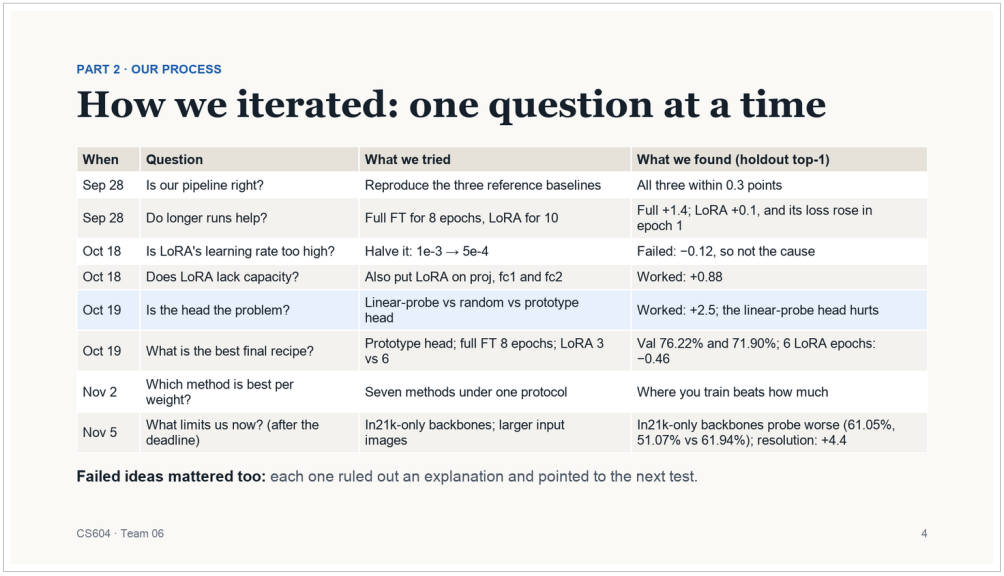
Calculation: parameter shares (the backbone has 85,798,656 parameters)

- Partial fine-tuning with $K = 1$: one block 7,087,872 + the final LayerNorm 1,536 = 7,089,408, which is 8.26%. With $K = 6$: $6 \times 7{,}087{,}872 + 1{,}536 = 42{,}528{,}768$, which is 49.6%.
- BitFit: $102{,}912 \div 85{,}798{,}656 \approx 0.120\%$.
- Adapter and AdaptFormer: $1{,}189{,}632 \div 85{,}798{,}656 \approx 1.387\%$ (the card says 1.39%).
- LoRA (four linear layers): $1{,}179{,}648 \div 85{,}798{,}656 \approx 1.375\%$ (the card says 1.38%).
- Reference LoRA (on qkv only): $12 \times 24{,}576 = 294{,}912$, which is 0.344%, so the table says 0.34%.

Key point

The difference between the two families is *where* the changes go. Family 1 puts the changes into the top few layers of the network. Family 2 spreads a tiny change over all 12 layers. Slide 5 shows with data that, for a similar number of parameters, spreading them over every layer works much better.

4.3 Slide 4: the timeline of our iterations



PART 2 · OUR PROCESS

How we iterated: one question at a time

When	Question	What we tried	What we found (holdout top-1)
Sep 28	Is our pipeline right?	Reproduce the three reference baselines	All three within 0.3 points
Sep 28	Do longer runs help?	Full FT for 8 epochs, LoRA for 10	Full +1.4; LoRA +0.1, and its loss rose in epoch 1
Oct 18	Is LoRA's learning rate too high?	Halve it: 1e-3 → 5e-4	Failed: −0.12, so not the cause
Oct 18	Does LoRA lack capacity?	Also put LoRA on proj, fc1 and fc2	Worked: +0.88
Oct 19	Is the head the problem?	Linear-probe vs random vs prototype head	Worked: +2.5; the linear-probe head hurts
Oct 19	What is the best final recipe?	Prototype head; full FT 8 epochs; LoRA 3 vs 6	Val 76.22% and 71.90%; 6 LoRA epochs: −0.46
Nov 2	Which method is best per weight?	Seven methods under one protocol	Where you train beats how much
Nov 5	What limits us now? (after the deadline)	In21k-only backbones; larger input images	In21k-only backbones probe worse (61.05%, 51.07% vs 61.94%); resolution: +4.4

Failed ideas mattered too: each one ruled out an explanation and pointed to the next test.

CS604 · Team 06 4

Slide 4

How to read the four columns

- **When**: the date. It shows the order of our work.
- **Question**: the one question that this round tries to answer.
- **What we tried**: the experiment for it. We changed only one setting each time.
- **What we found**: the result (holdout top-1; rows marked Val are on the CodaBench validation set), marked Worked or Failed.

The most important thing: cause and effect between the rows

Do not read one row on its own. Read how the answer of each round decides the question of the next round:

1. **September 28.** Is the pipeline right? Reproduce the three reference results → all within 0.3 points, so we can go on.
2. **September 28.** Does training longer help? Full fine-tuning for 8 epochs, LoRA for 10 → full fine-tuning +1.4, LoRA only +0.1. Also, LoRA's training loss went *up* during the first epoch. This is a warning sign.
3. **October 18.** Is LoRA's learning rate too high? Halve it (1e-3 → 5e-4) → −0.12. Failed: the learning rate is not the problem.
4. **October 18.** Does LoRA lack capacity? Add LoRA to proj, fc1 and fc2 as well → +0.88. Worked.
5. **October 19.** Is the head the problem? Compare the linear-probe, random and prototype heads → +2.5, and the linear-probe head turns out to be harmful. This is the biggest finding of the project, so this row is highlighted in blue.
6. **October 19.** What is the final recipe? Prototype head; full fine-tuning for 8 epochs; LoRA with 3 vs 6 epochs → val 76.22% and 71.90%; 6 LoRA epochs give −0.46.
7. **November 2.** Which method gives the most for each parameter? Seven methods with one protocol → "where you train matters more than how much".
8. **November 5 (after the deadline).** What still limits us? Backbones trained only on ImageNet-21k

give weaker linear probes (61.05% and 51.07%; ours gives 61.94%); resolution +4.4.

Everyday analogy

It is like a doctor looking for the cause of an illness. The patient has a fever (LoRA's loss goes up). The doctor first suspects a cold (the learning rate is too high). The medicine does not help (-0.12), so a cold is ruled out. Then the doctor suspects another cause (the head), and a test confirms it ($+2.5$). The test that found nothing was still useful: it stopped you from wasting time in the wrong direction.

Key point

The instructor cares a lot about "how you iterated through experiments" and about "failed experiments". This table shows both on one page: 8 rounds, one question each, with the successes and the failures written down. The sentence under the table sums it up: failed ideas are useful too; each one ruled out an explanation and pointed to the next test.

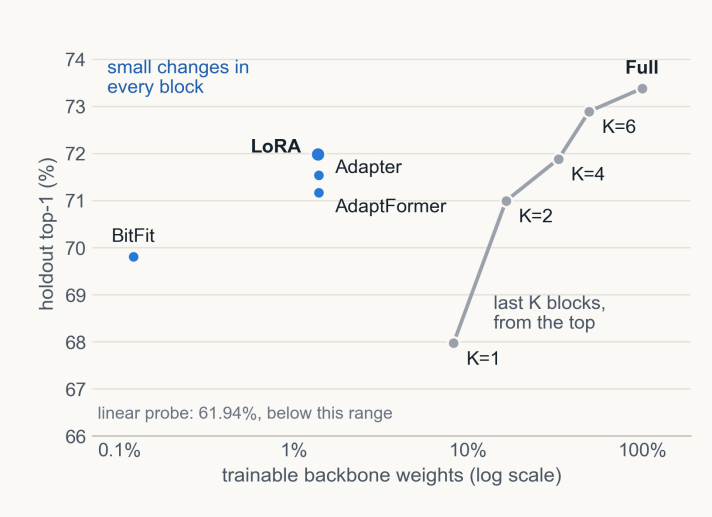
4.4 Slide 5: the seven-method table and the accuracy-vs-parameters plot

The table on the left

All methods use one protocol: prototype head, 3 epochs, the same holdout and the same kind of GPU. So the numbers can be compared directly. The two light-blue rows (full fine-tuning and LoRA) are the two methods we used in the end; they are highlighted on the slide too.

Method	Trained share	Holdout	Training memory	Latency (batch 1)
Linear probe	0%	61.94%	3.2 GB	6.96 ms
Partial fine-tuning, $K = 4$	33%	71.88%	7.0 GB	7.41 ms
Full fine-tuning	100%	73.38%	18.9 GB	7.78 ms
BitFit	0.12%	69.81%	12.7 GB	7.33 ms
Adapter	1.39%	71.54%	12.8 GB	9.74 ms
AdaptFormer	1.39%	71.17%	12.8 GB	10.23 ms
LoRA	1.38%	71.98%	19.9 GB	7.61 ms

How to read the plot on the right



The right plot of slide 5: the x-axis is the share of trained parameters (log scale); the y-axis is holdout accuracy

- **x-axis**: how much of the backbone is trained, on a log scale.
- **y-axis**: holdout top-1 accuracy, from 66% to 74%.
- **Gray line**: "train the last K blocks, from the top down", for $K = 1, 2, 4, 6$, and then up to full fine-tuning (Full).
- **Blue points**: the four methods that "make a small change in every block".
- **Small text at the bottom left**: the linear probe gets 61.94%. That is below the lowest value on the y-axis, so it is not drawn.

What a log scale means

On a normal scale, two neighboring grid lines differ by a fixed amount (for example, +10 each time). On a log scale, they differ by a fixed *factor*: 0.1%, 1%, 10% and 100% are equally far apart, and each step is $\times 10$.
Why use it? BitFit trains 0.12% and full fine-tuning trains 100%, about 833 times more. On a normal scale, 0.12% and 1.38% would be squeezed into a tiny strip at the far left, and you could not tell them apart. On a log scale they are spread out.

The gray line: each extra block gives less

Start from the linear probe (61.94%) and move right along the gray line:

- the last 1 block (8.3%): 67.97%, +6.0 over the linear probe;
- 2 blocks (16.5%): 70.99%, another +3.0;
- 4 blocks (33%): 71.88%, another +0.9;
- 6 blocks (49.6%): 72.89%, another +1.0;
- all 12 blocks (100%): 73.38%, another +0.5.

The further right you go, the less accuracy you get for each extra parameter. This is called **diminishing returns**. Training half of the blocks already gets within 0.5 points of full fine-tuning.

The blue points: far above and to the left of the gray line

The blue points sit above and to the left of the gray line. That means "similar or even higher accuracy with far fewer parameters". Compare each one with a point on the gray line that has a similar accuracy:

Calculation: how many times fewer parameters

- LoRA (1.375%, 71.98%) matches the last 4 blocks (33.0%, 71.88%): $33.0 \div 1.375 = 24$ times fewer parameters.
- Adapter (1.387%, 71.54%) beats the last 2 blocks (16.5%, 70.99%): about 12 times fewer.
- BitFit (0.120%, 69.81%) beats the last 1 block (8.26%, 67.97%): about 69 times fewer.

This is what the report means by "with 12 to 69 times fewer parameters".

Conclusion: **where you train matters more than how much you train.** A small change in every layer works much better than big changes in only the top few layers. A reasonable explanation: frozen lower layers cannot adjust their features to the new classes, while small changes that are spread out can reach every layer.

The memory column: memory follows how far back the gradients go

You might expect that fewer trained parameters means less memory. The table says otherwise:

- LoRA trains only 1.38% but needs 19.9 GB, which is more than full fine-tuning (100%, 18.9 GB);
- training the last 4 blocks (33%) needs only 7.0 GB;
- BitFit (0.12%) and the two adapters (1.39%) need 12.7 to 12.8 GB, which is more than training half of the network ($K = 6$, 10.0 GB).

The reason is the backward pass. To compute the gradients, you need in-between results that were stored during the forward pass (these are called **activations**). If the gradient reaches a layer, the activations of that layer must be kept.

- Partial fine-tuning: the gradients only need to pass through the top K blocks. The blocks below do not need to keep anything, which saves memory.
- BitFit, Adapter, LoRA: every layer has trainable parameters, so the gradients must go all the way back to the first block, and the activations of all layers must be kept.
- But the big weight matrices of BitFit and the two adapters are frozen. No weight gradients are needed for them, so they can keep a bit less. That is why they use about one third less memory than full fine-tuning and also train faster (1,407 to 1,509 images/s; full fine-tuning: 1,139 images/s).
- LoRA is the exception. To compute the gradient of matrix A, it must keep the input of every linear layer, just like full fine-tuning. It must also keep the activations of its own 96 small matrix multiplications (12 blocks $\times$ 4 layers $\times$ 2 matrices). So it needs a bit more memory than full fine-tuning, and it trains the slowest (888 images/s).

Calculation: how much memory each extra block adds

Partial fine-tuning uses 4.1, 7.0 and 10.0 GB for $K = 2, 4, 6$. Each extra block adds about $(10.0 - 4.1) \div (6 - 2) \approx 1.475$ GB, so about 1.5 GB. Continue at this rate up to 12 blocks: $4.1 + 10 \times 1.475 \approx 18.8$ GB. That is almost exactly the 18.9 GB measured for full fine-tuning.
So the memory of full fine-tuning goes mainly into storing activations. The extra optimizer state that it needs compared with LoRA (for each of the 85.8M parameters: its gradient and the two AdamW running averages, 4 bytes each) is only

$$85{,}798{,}656 \times 4 \text{ bytes} \times 3 \approx 1.03 \text{ GB}.$$

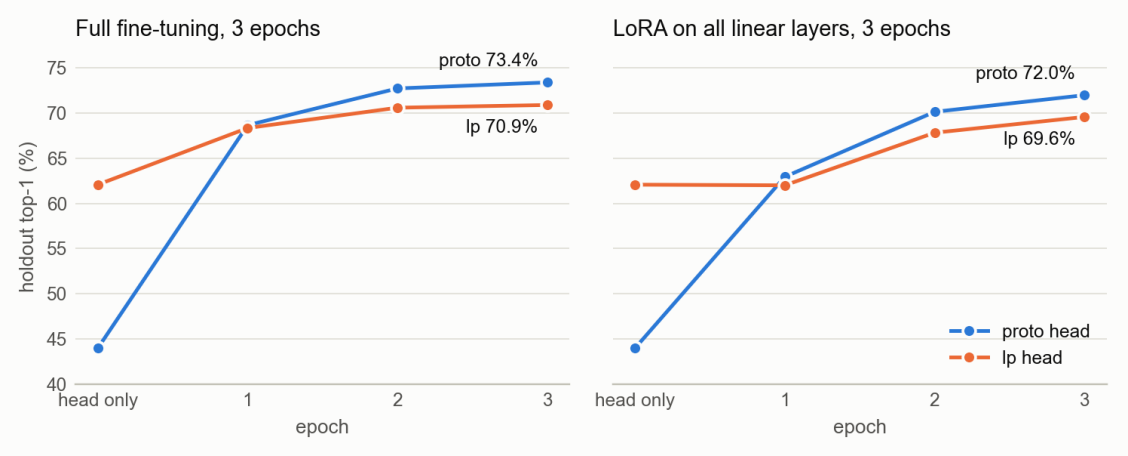
The latency column: only the two adapters are really slower

- The linear probe, partial fine-tuning, full fine-tuning, BitFit and merged LoRA all have exactly the structure of the original ViT-B/16 at inference. Their latency is about 7 ms (6.96 to 7.78 ms). The differences of a few tenths of a millisecond between them are measurement noise: $K = 1$ to 6, which have exactly the same structure, also differ by about 0.5 ms.
- The Adapter (9.74 ms) and AdaptFormer (10.23 ms) take 2 to 3 ms longer. Each block has two extra layers to compute, with a ReLU between them, so they cannot be merged into the original weights.

Three conclusions from this slide

1. Where matters more than how much: LoRA, with 1.38% of the parameters, matches partial fine-tuning with 33%.
2. Memory follows how far back the gradients go, not how many parameters are trained.
3. Full fine-tuning is the most accurate. Among the efficient methods, LoRA is the best, and after merging it adds no latency.

4.5 Slide 6: the head initialization curves



The figure on slide 6: full fine-tuning on the left, LoRA (four linear layers) on the right; orange is the linear-probe head, blue is the prototype head

The axes

- **x-axis**: "head only" at the far left is the accuracy with only the head, before training starts. Then come the accuracies at the end of epochs 1, 2 and 3.
- **y-axis**: holdout top-1 accuracy.
- The two small plots differ only in the method (full fine-tuning or LoRA). The two lines in each plot differ only in how the head starts.

What happens from left to right

1. **Start (head only)**: the linear-probe head has 62.1% and the prototype head 44.0%. The linear probe leads by $62.1 - 44.0 = 18.1$ points. This is because the linear-probe head was trained on the frozen features, while the prototype head had no training at all.
2. **After epoch 1**: in the left plot, the two lines almost meet (about 68%). The right plot is more interesting: the orange line has barely moved (it is still at about 62%), while the blue line is already at about 63%. The orange line standing still matches the fact that, in this setting, the training loss went up during the first epoch.
3. **End of epoch 3**: the blue line has overtaken the orange one and pulled ahead. Full fine-tuning: 73.4% vs 70.9%, a gap of 2.5 points. LoRA: 72.0% vs 69.6%, a gap of 2.4 points.

Key point

This figure in one sentence: **a good start is not a good finish.** The linear-probe head leads by 18 points at the start, but it ends about 2.5 points behind. That is why we judge every setting only by the result after training, on held-out data. We do not judge by the starting point, and we do not judge by the training loss.

Two small details

- The points on the curves are recorded at the end of each epoch during training. The numbers in the tables come from a final evaluation after training. So a few numbers differ by a few tenths. For

example, LoRA with the linear-probe head is 69.6% on the curve, but 69.45% on slide 9 and in the report's table. The difference is within the noise and does not change the conclusion.

- This figure comes from "3 epochs, holdout" experiments. Slide 8 comes from "8 epochs, all data, val" experiments, which are different runs. Two different sets of experiments give the same conclusion, which makes it more convincing.

4.6 Slide 7: the prototype head pipeline and the three numbers

The four steps at the top

1. **Extract features**: run each of the 450,000 training images once through the frozen ViT. Each image gives a feature of 768 numbers. This takes about 6 minutes and counts as GPU time.
2. **Prototype head**: the weight of each class = the normalized average direction of that class's features $\times$ 40. This is the key step, so it is highlighted in blue. The method is called **weight imprinting** (Qi et al., CVPR 2018).
3. **Fine-tune**: start from this head and do full fine-tuning or LoRA.
4. **Predict**: the full model adds the logits of an image and of its mirror image (flip TTA).

The arrows show the data flowing from left to right: features $\rightarrow$ prototypes $\rightarrow$ initialized head $\rightarrow$ training $\rightarrow$ prediction.

How step 2 is computed, step by step

The prototype head formula

Let the feature of training image i be z_i (768 numbers), and let its class be y_i.

1. First normalize each feature to length 1, so only its direction is kept: $\hat{z}_i = z_i / \|z_i\|$.
2. Add up the normalized features of one class, then normalize again. This gives the class prototype (the average direction):
$$\mu_c = \frac{\sum_{i:\, y_i = c} \hat{z}_i}{\left\| \sum_{i:\, y_i = c} \hat{z}_i \right\|}.$$
3. Write the prototypes into the head: the weight of row c is $W_c = \frac{\tau}{\overline{\|z\|}} \mu_c$, and the bias is $b_c = 0$.
4. The score of a new image z for class c is
$$\text{logit}_c = W_c \cdot z = \tau \cdot \frac{\|z\|}{\overline{\|z\|}} \cdot \cos(z, \mu_c) \approx \tau \cos(z, \mu_c).$$

Symbols: $\|z\|$ is the length of a vector; μ_c is the prototype of class c; $\tau = 40$ is the temperature; $\overline{\|z\|}$ is the average length of all features; cos is the cosine similarity.
What it says: the class of an image is the class whose "average direction" is closest to the direction of the image. The head receives the feature z without normalization, so we divide by the average length $\overline{\|z\|}$. That makes the score about "40 $\times$ cosine similarity".

Simplified illustration: 3 classes, features of 2 numbers (label smoothing ignored)

The real case has 768 numbers and 10,000 classes. Here we shrink it to 2 numbers and 3 classes, so you can compute by hand. Each class has two training images:

- Bird: (3, 4) and (4, 3). Both have length 5. Normalized: (0.6, 0.8) and (0.8, 0.6). Sum: (1.4, 1.4). Normalize again: the prototype is $\mu_{\text{bird}} = (0.707, 0.707)$.
- Snake: $(-3, 4)$ and $(0, 5) \to (-0.6, 0.8)$ and $(0, 1) \to$ sum $(-0.6, 1.8) \to$ prototype $(-0.316, 0.949)$.
- Moth: $(5, 0)$ and $(4, -3) \to (1, 0)$ and $(0.8, -0.6) \to$ sum $(1.8, -0.6) \to$ prototype $(0.949, -0.316)$.

A new image has the feature (6, 8). Normalized, it is (0.6, 0.8). Its cosine similarity with the three prototypes:

- Bird: $0.6 \times 0.707 + 0.8 \times 0.707 = 0.990$;
- Snake: $0.6 \times (-0.316) + 0.8 \times 0.949 = 0.569$;
- Moth: $0.6 \times 0.949 + 0.8 \times (-0.316) = 0.316$.

Multiply by the temperature 40: logits = (39.6, 22.8, 12.6). Bird is the largest, so the prediction is "bird". There was no training at all.

What the temperature 40 does

Cosine similarity is only between -1 and 1. If you use it directly as the score, the probabilities after softmax are very "flat", and the model looks very unsure.

The same example with different temperatures (simplified, label smoothing ignored)

- Temperature 1: logits (0.990, 0.569, 0.316) $\to$ probabilities of about (0.46, 0.30, 0.24). Correct, but very unsure.
- Temperature 5: logits (4.95, 2.85, 1.58) $\to$ probabilities of about (0.86, 0.11, 0.03).
- Temperature 40: logits (39.6, 22.8, 12.6) $\to$ the probability of "bird" is almost 1.

In the real case there are 10,000 classes. Many prototypes point in very similar directions, and the cosine of the correct class is only a little higher than the others. If the temperature is too small, the probabilities are too flat and the training signal is too weak. If it is too large, the model becomes extremely confident whenever it is wrong, and the loss gets very large. So we tried 8 candidates (5, 10, 15, 20, 30, 40, 60 and 80) and picked the one with the lowest cross-entropy on 10,000 training features. The result was 40. Note: picking the temperature only chooses one of 8 numbers. It is not gradient training of the head.

The three number cards at the bottom

- **0**: building the head took 0 steps of gradient descent. It never "fitted" any training image, so there is nothing for it to memorize.
- **44%**: before any training, the prototype head already gets 44% on the holdout. Random guessing among 10,000 classes gives only 0.01%, so this is about 4,400 times higher. It shows that the pre-trained features are already good: simply comparing an image with each class's average direction gets many images right.
- **4.39**: the training loss at the very start of training (the average of the first 100 LoRA steps). For comparison: the linear-probe head starts at 2.29, and the random head at 9.28.

How to read 2.29, 4.39 and 9.28

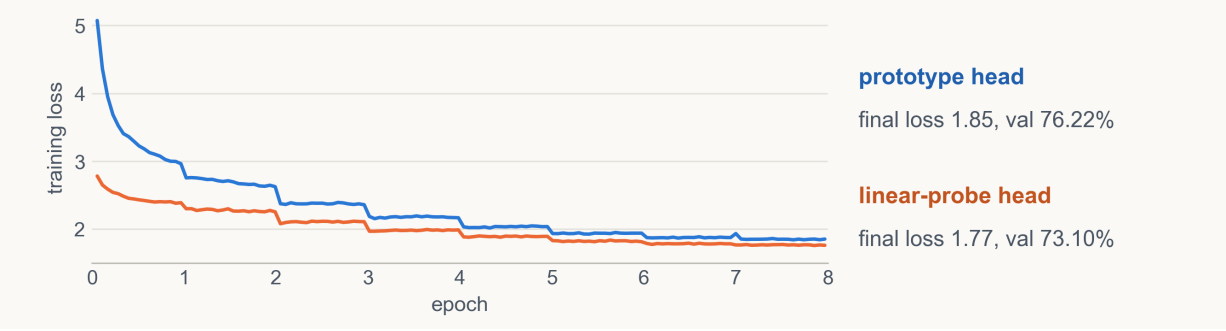
These three numbers are training losses. They include label smoothing 0.1 and the effect of data augmentation, so only compare them with each other. Do not turn them into "the probability of the correct class".

- **9.28 (random head)**: almost the same as $\ln 10{,}000 = 4 \ln 10 \approx 4 \times 2.303 = 9.21$. $\ln 10{,}000$ is exactly the loss of "guessing evenly among 10,000 classes" (it is about the same with or without label smoothing). So the random head is guessing at the start, and the gradients it sends to the backbone are large and messy.
- **4.39 (prototype head)**: it roughly knows what each class looks like, but it is far from confident. The loss is not small, so the gradient signal is strong enough to push the backbone to adapt to the new classes.
- **2.29 (linear-probe head)**: it has already been fitted on these training images, so its loss starts low. The gradient signal is weak, and the backbone gets little push to change.

Easy to get wrong

A high starting training loss is a good thing here: it means there is a lot left to learn. But a loss that is too high (9.28 for the random head) also means messy gradients at the start. The prototype head sits right in between: its starting loss is high enough, and it is not guessing, so it learns fast. After epoch 1 it is 5 points ahead of the random head (63.3% vs 58.4%).

4.7 Slide 8: the training loss curves, the three-step explanation and the control experiment



The figure on slide 8: full fine-tuning, all data, 8 epochs; the two runs differ only in how the head starts

How to read this figure

- **x-axis**: epochs, from 0 to 8. **y-axis**: training loss.
- **Blue line**: the prototype head. It starts above 5 and ends at 1.85; val 76.22%.
- **Orange line**: the linear-probe head. It starts at about 2.8 and ends at 1.77; val 73.10%.
- The two runs differ only in how the head starts. Everything else is the same: full fine-tuning, all 45 photos per class, 8 epochs.

The first surprise: lower loss, yet lower accuracy

The orange line stays below the blue line the whole time: the training loss is lower ($1.77 < 1.85$), yet val

is 3.12 points lower (73.10% vs 76.22%). A low training loss only says that the model answers well on the training images. If it answers badly on images it has not seen, it has remembered the training images themselves, not patterns that also work on new images. This is **memorization**.

The second clue: the "steps"

Look closely at both lines. At every epoch boundary (1, 2, 3, and so on, on the x-axis), the loss suddenly drops a bit. Inside an epoch, it is almost flat. Why?

- Inside an epoch, every image appears for the first time in this round.
- When a new epoch starts, every image is one that the model "has met before".

If the model only learned general patterns, the loss would go down smoothly. A sudden drop at the boundary shows that the model recognizes "I have seen this image before". In other words, it is remembering specific images. This is a typical sign of memorization.

Everyday analogy

A student got the answer key to the practice questions in advance. The practice scores keep rising (the training loss keeps falling), but on new exam questions (val) the student does badly. The linear-probe head is like that answer key: it was trained on exactly these training images.

The three-step explanation at the bottom

1. **Fitted on the same photos**: the linear-probe head was trained on the frozen features of the training images, so it already "knows" these images.
2. **A weak signal, then a wrong one**: the starting loss is low, so the backbone gets a weak gradient signal. Once the features begin to change, the fitted head no longer matches them. Evidence: with LoRA and the linear-probe head, the training loss rose from 2.22 to 2.69 (about 2.7) during the first epoch. At that point, no image had been seen a second time.
3. **Memorization**: in the end, the model fits the 45 images of each class, not features that generalize. The training loss is lower (1.77 vs 1.85), and val is lower (73.10% vs 76.22%).

The control experiment at the very bottom

Suppose the prototype head were good only because the idea of "prototypes" is somehow magic. Then a random head should be clearly worse. The actual result: with LoRA, the random head gets 71.95% and the prototype head gets 72.00%. It is a tie. So the real problem is "fitting the head on the training images", not some magic in the prototypes.

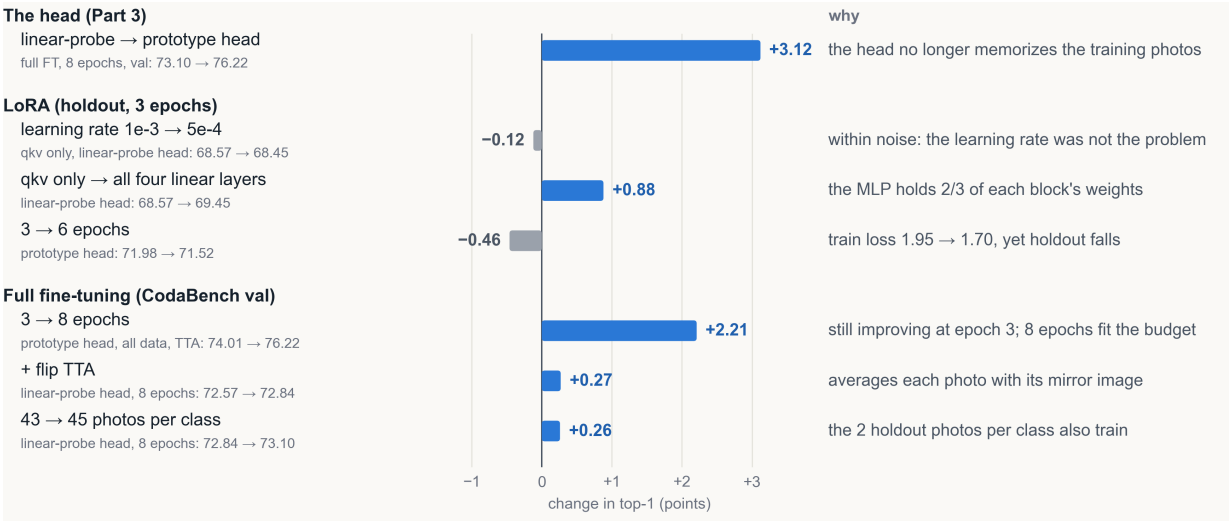
Key point

A **control experiment** is an experiment designed to rule out another explanation. Here, the random head rules out "the prototypes themselves are magic", so the conclusion is more believable.

Easy to get wrong

Do not say "we proved" on this slide. The report says that the explanation is consistent with the loss curves and with the control experiment, but it is not a strict proof of cause and effect. In the presentation it is safer to say: "our explanation is consistent with the loss curves and the control".

4.8 Slide 9: the ablation bar chart



The figure on slide 9: each row changes only one setting; the middle shows the change in accuracy, the right side shows the reason

How to read the three columns

- **Left column**: what was changed. A line of small text below it says in which setting, and from what value to what value.
- **Middle column**: the length of the bar = how many points top-1 changed. Blue is a gain, gray is a drop. The vertical line in the middle is 0.
- **Right column (why)**: why this happened.

The three groups use different data: groups 1 and 3 are on the CodaBench val set, and group 2 is on the holdout. So only compare within a group. Do not compare scores across groups.

Row by row

What changed (setting)	Before → after	Change	Reason
Head: linear probe → prototype (full fine-tuning, 8 epochs, val)	73.10 → 76.22	+3.12	The head no longer memorizes the training images. The biggest single change
LoRA learning rate 1e-3 → 5e-4 (qkv only, linear-probe head)	68.57 → 68.45	−0.12	Within the noise: the learning rate is not the problem
LoRA from qkv to all four linear layers (linear-probe head)	68.57 → 69.45	+0.88	The MLP holds 2/3 of each block's parameters; changing only attention misses most of them
LoRA 3 → 6 epochs (prototype head)	71.98 → 71.52	−0.46	The training loss fell from 1.95 to 1.70, yet the holdout dropped: overfitting, so we stop at 3 epochs
Full fine-tuning 3 → 8 epochs (prototype head, all data, TTA)	74.01 → 76.22	+2.21	Still improving at epoch 3; 8 epochs fit the budget (1.03 GPU-hours)
Add flip TTA (linear-probe head, 8 epochs)	72.57 → 72.84	+0.27	Average each image with its mirror image
43 → 45 photos per class (linear-probe head, 8 epochs)	72.84 → 73.10	+0.26	The 2 held-out photos are used for training too

Calculation: why "the MLP holds 2/3"

In one block, fc1 has $768 \times 3{,}072 + 3{,}072 = 2{,}362{,}368$ parameters, and fc2 has $3{,}072 \times 768 + 768 = 2{,}360{,}064$. Together that is 4,722,432. Divided by the whole block (7,087,872), it is about 66.6%, that is, 2/3. Putting LoRA only on qkv leaves these 2/3 of the weights completely untouched.

Which differences are noise

- -0.12: much smaller than 0.3, so it is noise. That is why we say "the learning rate is not the cause".
- +0.27 and +0.26: only a little bigger than the noise. They point in the expected direction, but do not read too much into them.
- +3.12, +2.21 and +0.88: clearly bigger than the noise. -0.46 is also bigger than the noise, and it comes together with a falling training loss, so we can conclude that it is overfitting.

Key point

The rule of an ablation: change only one setting at a time, and keep everything else the same. Then the change can only come from that one setting. Every row on slide 9 follows this rule, so every setting in our final recipe is backed by data.

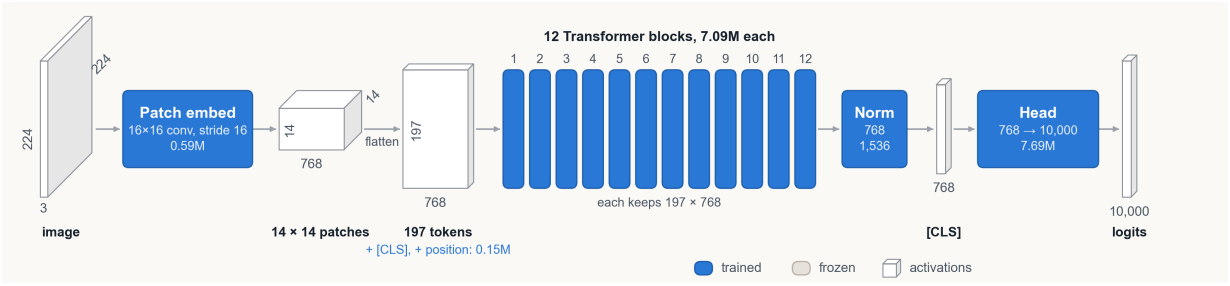
4.9 The architecture diagrams one by one (slides 10, 11, 16 to 20)

These diagrams come in two views:

- **Whole-network view**: from the input image all the way to the output scores (full fine-tuning, linear probe, partial fine-tuning).
- **Inside-a-block view**: one of the 12 blocks, zoomed in (LoRA, BitFit, Adapter, AdaptFormer).

The legend is the same everywhere. **Blue** = the parts this method trains (trained). **Beige-gray** = frozen (frozen). **White box** = computation only, with no parameters (no weights). **White 3D box** = an in-between result (activations, a tensor), with its shape written next to it.

4.9.1 Whole network: full fine-tuning (slide 10)



Slide 10: the whole network for full fine-tuning; every part is blue (trained)

The data flows from left to right:

1. **image (224 × 224 × 3)**: the input image, 224 pixels wide and 224 pixels high, with 3 color channels (red, green, blue).
2. **Patch embed (16 × 16 conv, stride 16, 0.59M)**: a 16 × 16 convolution with stride 16 cuts the image into squares that do not overlap, and turns each square into 768 numbers.
3. **14 × 14 × 768**: $224 \div 16 = 14$, so we get 14 × 14 patches, each with 768 numbers.
4. **flatten → 197 tokens (197 × 768)**: lay the 14 × 14 grid out in one row (196 tokens), and add a [CLS] token in front: 197 tokens in total. Each token also gets its position embedding added. The [CLS] token and the position embeddings have 0.15M parameters together.
5. **12 Transformer blocks (7.09M each)**: 12 identical blocks process the tokens one after another. Each block takes 197 × 768 in and gives 197 × 768 out (the diagram says "each keeps 197 × 768"). The shape stays the same, but the content "understands" the image better and better.
6. **Norm (768, 1,536)**: the final LayerNorm.
7. **[CLS] (768)**: take only the 768-number vector of the [CLS] token. It stands for the whole image.
8. **Head (768 → 10,000, 7.69M)**: one linear layer that outputs 10,000 scores.
9. **logits (10,000)**: the class with the highest score is the prediction.

Calculation: how the 85,798,656 parameters add up

- Patch embedding: $16 \times 16 \times 3 \times 768 + 768 = 590{,}592$ (0.59M on the diagram).

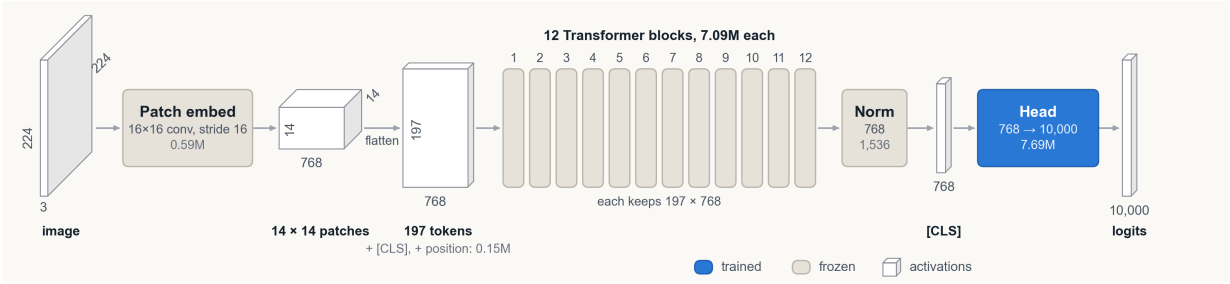
CLS + position embeddings: $768 + 197 \times 768 = 152{,}064$ (0.15M on the diagram).

- One block: qkv $768 \times 2{,}304 + 2{,}304 = 1{,}771{,}776$; proj $768 \times 768 + 768 = 590{,}592$; fc1 $768 \times$

$3{,}072 + 3{,}072 = 2{,}362{,}368$; fc2 $3{,}072 \times 768 + 768 = 2{,}360{,}064$; two LayerNorms $2 \times 1{,}536 = 3{,}072$. Total: 7,087,872 (7.09M on the diagram).

- 12 blocks: $12 \times 7{,}087{,}872 = 85{,}054{,}464$. The final LayerNorm: 1,536.
- Total: $590{,}592 + 152{,}064 + 85{,}054{,}464 + 1{,}536 = 85{,}798{,}656$.
- New head (not counted in the backbone): $768 \times 10{,}000 + 10{,}000 = 7{,}690{,}000$ (7.69M on the diagram).

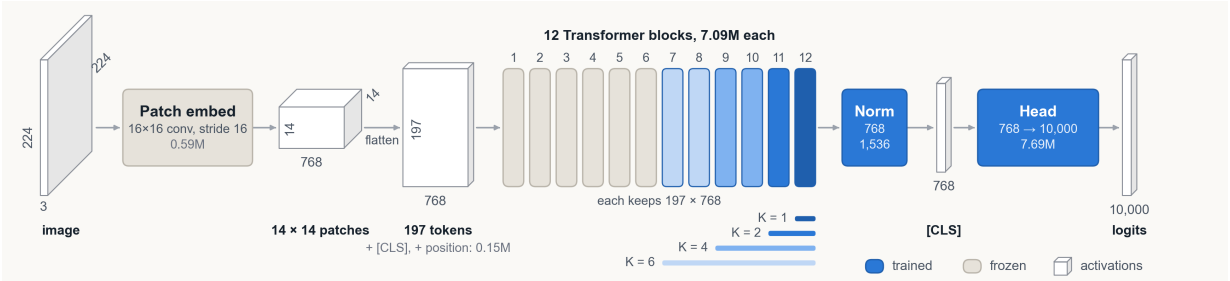
4.9.2 Whole network: linear probe (slide 16)



Slide 16 (backup): the linear probe; only the head is blue

- Everything except the head is beige-gray (frozen). The backbone does not change at all.
- Because the backbone does not change, each image's feature only needs to be computed once. It can be stored and used again and again (cached features). Training the head for 10 epochs on these 768-number features then takes only a few seconds. That makes it the cheapest method: 0.10 GPU-hours (mostly the feature extraction at the start) and 3.2 GB.
- The downside: the features never adjust to the new classes. Its accuracy is capped by how good the frozen features are: 61.94%.

4.9.3 Whole network: partial fine-tuning (slide 17)



Slide 17 (backup): partial fine-tuning; only the last K blocks are trained

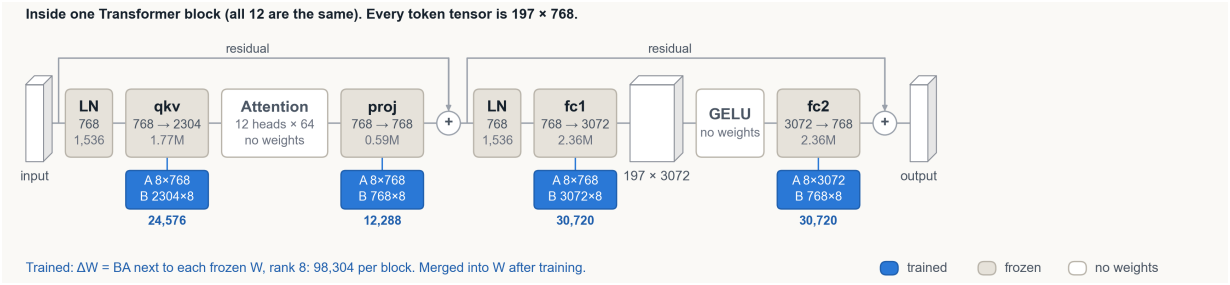
- Blocks 7 to 12 are drawn in different shades of blue. The bars below for $K = 1, 2, 4, 6$ show which blocks each K trains: $K = 1$ trains only block 12, and $K = 6$ trains blocks 7 to 12.
- The final Norm and the head are trained too (blue). The patch embedding and the lower blocks are frozen.
- Parameters: $K \times 7{,}087{,}872 + 1{,}536$ (the final LayerNorm).
- Memory: the gradients only need to pass through the top K blocks, so the memory grows with K, by about 1.5 GB per block (3.8 to 10.0 GB).

4.9.4 The shared base picture inside a block

The diagrams for LoRA, BitFit, Adapter and AdaptFormer all zoom into one block, and they share the same base picture. The note at the top says: all 12 blocks are the same, and every token tensor is 197 × 768. From left to right:

1. **input** (197 × 768).
2. **LN** (768, 1,536 parameters): the first LayerNorm.
3. **qkv** (768 → 2,304, 1.77M): computes Q, K and V at once, 768 numbers each.
4. **Attention** (12 heads × 64, no weights): multi-head attention. It combines the V vectors with weights based on how well Q and K match. This step is only computation and has no parameters, so it is a white box.
5. **proj** (768 → 768, 0.59M): brings the results of the 12 heads back together into 768 numbers.
6. **The first "+"**: the residual connection, which adds the block's input back. The line marked "residual" at the top is this "shortcut".
7. **LN** (768, 1,536): the second LayerNorm.
8. **fc1** (768 → 3,072, 2.36M): expands to 3,072 numbers, giving an in-between result of 197 × 3,072.
9. **GELU** (no weights): the activation function.
10. **fc2** (3,072 → 768, 2.36M): shrinks back to 768 numbers.
11. **The second "+"**: the second residual connection.
12. **output** (197 × 768): goes into the next block.

4.9.5 LoRA (slide 11)



Slide 11: LoRA; each frozen linear layer has a pair of small blue matrices A and B below it

- The four linear layers themselves (qkv, proj, fc1, fc2) are beige-gray (frozen). Each one has a pair of small blue matrices below it: A (8 × input size) and B (output size × 8).
- **How the data flows**: the input x takes two paths at the same time. The main path is the frozen Wx. The side path first uses A to squeeze x down to 8 numbers, then uses B to expand them back to the output size, and multiplies by $\alpha/r = 16/8 = 2$. The two results are added: $y = Wx + 2 \cdot BAx$.
- B starts as all zeros, so at the start of training the side path outputs 0. The model is then exactly the pretrained model, and its good features are not disturbed right away.
- The bottom left of the diagram says: $\Delta W = BA$, rank 8, 98,304 parameters per block; merged into W after training.

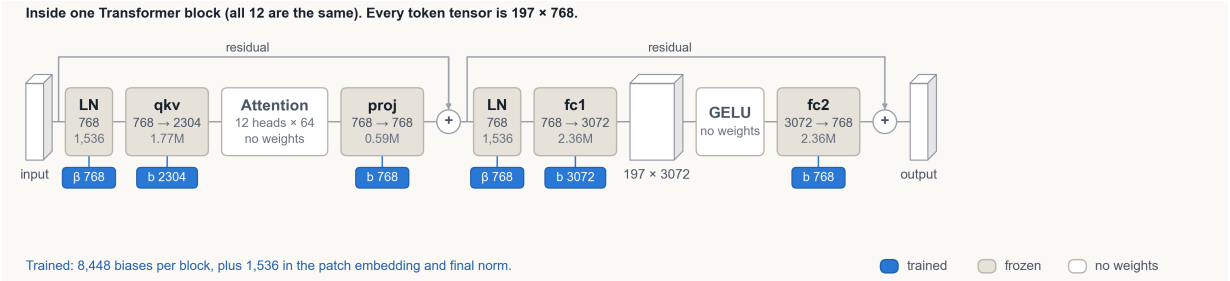
Calculation: LoRA's parameters

The parameters of each pair A, B = $r \times$ (input size + output size), with $r = 8$:

- qkv: $8 \times (768 + 2{,}304) = 24{,}576$; proj: $8 \times (768 + 768) = 12{,}288$;
- fc1: $8 \times (768 + 3{,}072) = 30{,}720$; fc2: $8 \times (3{,}072 + 768) = 30{,}720$.

That is 98,304 per block, and 1,179,648 for all 12 blocks, which is 1.375% of the backbone. For comparison: the proj layer alone has $768 \times 768 = 589{,}824$ weights, and LoRA changes it with only 12,288 numbers, about 2%.

4.9.6 BitFit (slide 18)



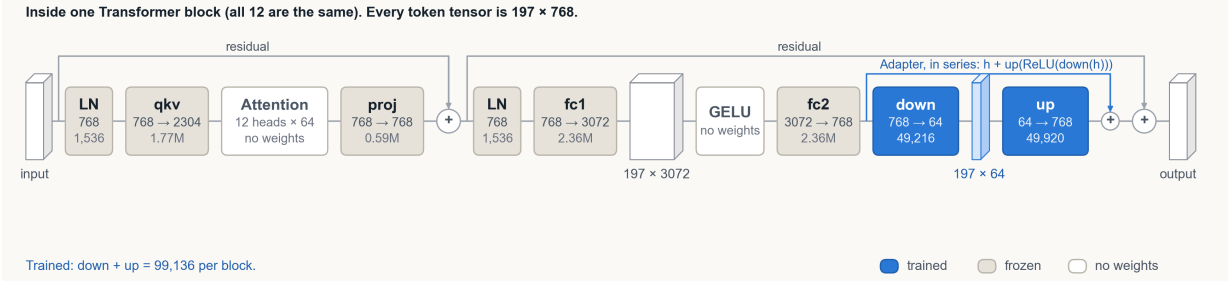
Slide 18 (backup): BitFit; only the small blue bias tags under each layer are trained

- All weight matrices are frozen. Under each layer hangs a small blue tag: the bias of that layer. These are the shift parameter β of the first LN (768), the b of qkv (2,304), the b of proj (768), the β of the second LN (768), the b of fc1 (3,072) and the b of fc2 (768).
- LayerNorm has two sets of parameters: a scale (γ) and a shift (β). BitFit trains only the shift, because it acts like a bias.

Calculation: 8,448 biases per block

$768 + 2{,}304 + 768 + 768 + 3{,}072 + 768 = 8{,}448$. For 12 blocks that is 101,376. Add the bias of the patch embedding (768) and the β of the final LayerNorm (768), 1,536 together: 102,912 in total, which is 0.120%.

4.9.7 Adapter (slide 19)



Slide 19 (backup): the Adapter sits in series after fc2

- After fc2 and before the second residual connection, two blue modules sit in series: down ($768 \to 64$, 49,216 parameters) and up ($64 \to 768$, 49,920 parameters). Between them is a "bottleneck" of 197×64.
- Formula: $h + \text{up}(\text{ReLU}(\text{down}(h)))$, where h is the output of the MLP. In other words, the Adapter

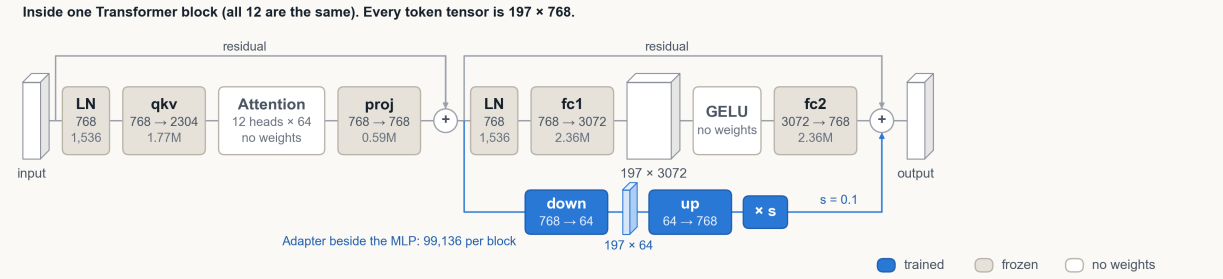
has its own small residual connection (the first blue "+" in the diagram), and only after that comes the main residual connection (the second "+").

- up starts at 0, so at the start of training the Adapter changes nothing.
- Because it is in series, inference has to compute two extra layers for every token, with a ReLU between them, so it cannot be merged. The latency goes from about 7.3 ms to 9.74 ms.

Calculation: 99,136 parameters per block

down: $768 \times 64 + 64 = 49{,}216$; up: $64 \times 768 + 768 = 49{,}920$; together 99,136. For 12 blocks that is 1,189,632, which is 1.387%.

4.9.8 AdaptFormer (slide 20)



Slide 20 (backup): AdaptFormer is a side branch in parallel with the MLP

- The branch starts after the first "+" (that is, at the input of the MLP sub-layer). It goes through down ($768 \to 64$), a 197×64 bottleneck and up ($64 \to 768$). Then it is multiplied by a small factor $s = 0.1$, and at the second "+" it is added back to the main path together with the output of the MLP.
- Formula: $x_{l+1} = x' + \mathrm{MLP}(\mathrm{LN}(x')) + s \cdot \mathrm{up}(\mathrm{ReLU}(\mathrm{down}(x')))$, where x' is the result after the attention sub-layer.
- The difference from the Adapter: the Adapter is "in series and changes the MLP output". AdaptFormer is "in parallel and adds a small correction on the side", so the original MLP output stays unchanged. $s = 0.1$ keeps the correction gentle at first. The number of parameters is the same: 99,136 per block.
- Accuracy: 71.17%, vs 71.54% for the Adapter. The gap is 0.37, close to the noise. The two also use different scaling factors (1.0 vs 0.1). So we cannot say "parallel is better" or "series is better".
- It cannot be merged either: its latency is 10.23 ms.

4.9.9 Why LoRA can be merged but the Adapter cannot

A small 2×2 example

Let the frozen weights be $W = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$. Let LoRA's $A = (1 \;\; 1)$ (size 1×2; here $r = 1$) and $B = \begin{pmatrix} 0.5 \\ 0 \end{pmatrix}$ (size 2×1). The scaling is $\alpha / r = 2$, and the input is $x = (1, 2)$.

- **Without merging**: the main path gives $Wx = (1, 2)$. The side path gives $Ax = 1 + 2 = 3$, then $B \times 3 = (1.5, 0)$, and times 2 that is $(3, 0)$. Adding the two paths gives $(4, 2)$.

- **With merging**:

$$\Delta W = 2BA = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}, \qquad W' = W + \Delta W = \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}, \qquad W'x = (2+2,\ 0+2) = (4, 2).$$

Both ways give exactly the same result. But after merging there is only one matrix multiplication, and the structure is exactly that of the original model, so the latency does not increase.
The Adapter has a ReLU between down and up (negative numbers become 0). That step is not a matrix multiplication, so "down, ReLU, up" cannot be multiplied together into one matrix. That is why it cannot be merged.

4.9.10 All seven diagrams side by side

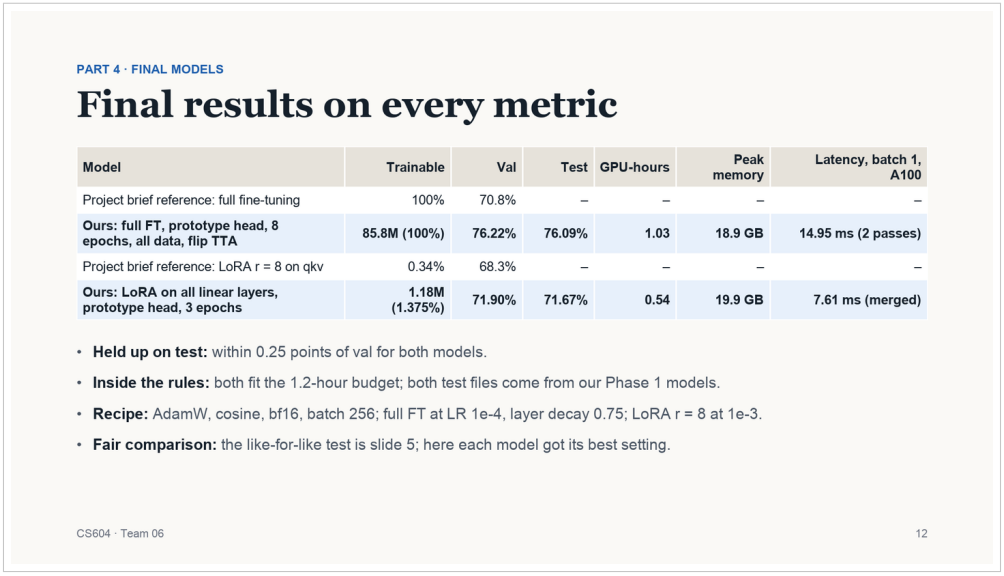
After all seven diagrams, one table puts their differences together. The "Where" column answers "where is the blue in the diagram?" The "Inference" column answers "after training, is the deployed model the same as the original ViT-B/16?"

Method	Trained part (blue)	Where	Inference (batch-1 latency)	Trained share
Linear probe	Only the head	The very top of the network	Same structure, 6.96 ms	0%
Partial fine-tuning	The last K blocks and the final LN	The top K layers	Same structure, about 7.4 ms	8.26% to 49.6%
Full fine-tuning	All parameters	Every layer	Same structure, 7.78 ms	100%
BitFit	All biases and the β of LN	Every layer, but only a little	Same structure, 7.33 ms	0.12%
Adapter	down and up in series in each block	Every layer, after the MLP (in series)	Two extra layers, cannot be merged, 9.74 ms	1.39%
AdaptFormer	A side branch next to each block's MLP	Every layer, next to the MLP (in parallel)	One extra branch, cannot be merged, 10.23 ms	1.39%
LoRA	A and B next to the 4 linear layers	Every layer, next to the linear layers	Same structure after merging, 7.61 ms	1.38%

One sentence to remember

Family 1 (linear probe, partial fine-tuning, full fine-tuning) trains whole blocks, from the top down. Family 2 (BitFit, Adapter, AdaptFormer, LoRA) changes a little in every layer. In family 2, BitFit and merged LoRA add no inference latency, while the two adapters do. And LoRA has the highest accuracy in family 2 (71.98%).

4.10 Slide 12: the final results table



PART 4 · FINAL MODELS

Final results on every metric

Model	Trainable	Val	Test	GPU-hours	Peak memory	Latency, batch 1, A100
Project brief reference: full fine-tuning	100%	70.8%	–	–	–	–
Ours: full FT, prototype head, 8 epochs, all data, flip TTA	**85.8M (100%)**	**76.22%**	**76.09%**	**1.03**	**18.9 GB**	**14.95 ms (2 passes)**
Project brief reference: LoRA r = 8 on qkv	0.34%	68.3%	–	–	–	–
Ours: LoRA on all linear layers, prototype head, 3 epochs	**1.18M (1.375%)**	**71.90%**	**71.67%**	**0.54**	**19.9 GB**	**7.61 ms (merged)**

- **Held up on test:** within 0.25 points of val for both models.
- **Inside the rules:** both fit the 1.2-hour budget; both test files come from our Phase 1 models.
- **Recipe:** AdamW, cosine, bf16, batch 256; full FT at LR 1e-4, layer decay 0.75; LoRA r = 8 at 1e-3.
- **Fair comparison:** the like-for-like test is slide 5; here each model got its best setting.

CS604 · Team 06 12

Slide 12

What each column means

- **Model**: two rows are the course references (gray, with only the val score and the trained share). Two rows are our final models (blue, in bold).
- **Trainable**: the number of trained backbone parameters and their share. Full fine-tuning: 85.8M (100%). LoRA: 1.18M (1.375%). Both also train the 7.69M head, which is not counted in the share, following the course reference.
- **Val / Test**: top-1 on the CodaBench validation set and test set.
- **GPU-hours**: the GPU time of one training run.
- **Peak memory**: the highest GPU memory used during training.
- **Latency, batch 1, A100**: the time to process one image at a time on an A100.

Calculation: how far above the reference, and the gap between test and val

- Full fine-tuning: $76.22 - 70.8 = 5.42 \approx +5.4$. LoRA: $71.90 - 68.3 = 3.60 \approx +3.6$.
- Test minus val: full fine-tuning $76.09 - 76.22 = -0.13$; LoRA $71.67 - 71.90 = -0.23$. Both are within 0.25. The models did not over-adapt to val, and they are just as stable on 100,000 completely new images.
- Budget: 1.03 and 0.54 GPU-hours, both within 1.2 hours.

What the two notes in brackets mean

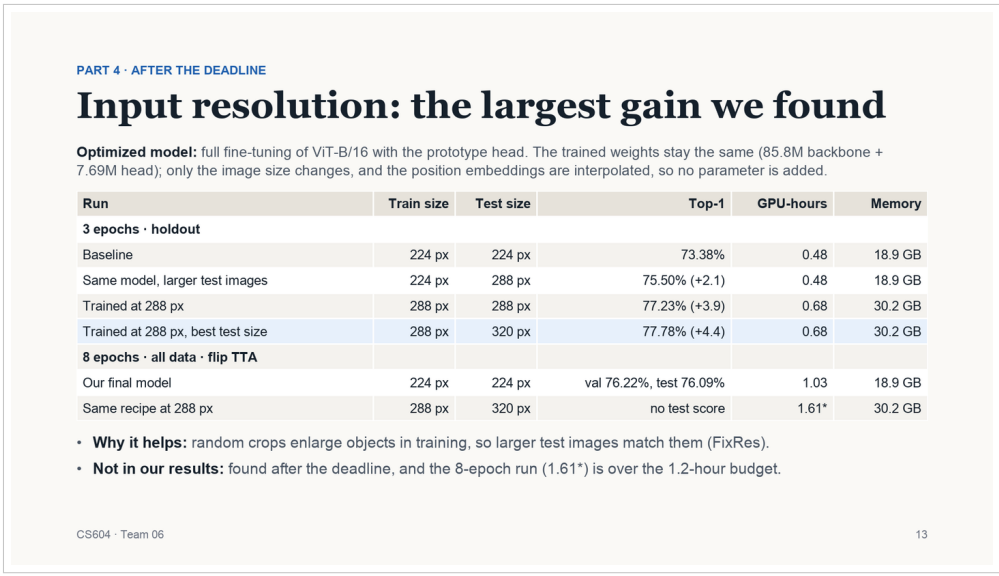
- **14.95 ms (2 passes)**: the full model uses flip TTA, so each image is computed twice (the original and the mirror image), and the latency roughly doubles. The same structure without TTA takes 7.35 ms.
- **7.61 ms (merged)**: after merging, LoRA is a normal ViT-B/16, with the same latency as the original model.

The fair comparison is on slide 5, not on this slide

This slide shows each method in its own best setup. Full fine-tuning used 8 epochs, all 45 photos per class and flip TTA. LoRA used 3 epochs, 43 photos per class and no TTA. So the gap between 76.22 and 71.90 is not caused by the method alone. The fair comparison with one protocol is on slide 5: 73.38% vs 71.98%.

The "Inside the rules" point at the bottom of the slide matters too: both models stay within the 1.2-hour budget, and both test files come from the same Phase 1 models, as the course requires.

4.11 Slide 13: the resolution table



PART 4 · AFTER THE DEADLINE

Input resolution: the largest gain we found

Optimized model: full fine-tuning of ViT-B/16 with the prototype head. The trained weights stay the same (85.8M backbone + 7.69M head); only the image size changes, and the position embeddings are interpolated, so no parameter is added.

Run	Train size	Test size	Top-1	GPU-hours	Memory
3 epochs · holdout					
Baseline	224 px	224 px	73.38%	0.48	18.9 GB
Same model, larger test images	224 px	288 px	75.50% (+2.1)	0.48	18.9 GB
Trained at 288 px	288 px	288 px	77.23% (+3.9)	0.68	30.2 GB
Trained at 288 px, best test size	288 px	320 px	77.78% (+4.4)	0.68	30.2 GB
8 epochs · all data · flip TTA					
Our final model	224 px	224 px	val 76.22%, test 76.09%	1.03	18.9 GB
Same recipe at 288 px	288 px	320 px	no test score	1.61*	30.2 GB

- **Why it helps:** random crops enlarge objects in training, so larger test images match them (FixRes).
- **Not in our results:** found after the deadline, and the 8-epoch run (1.61*) is over the 1.2-hour budget.

CS604 · Team 06 13

Slide 13

How to read the table

The sentence above the table first makes one thing clear: the optimized model is "full fine-tuning + prototype head". The trained parameters are exactly the same (85.8M backbone + 7.69M head); only the size of the input images changes. The position embeddings are interpolated to fit, so no parameters are added.

The columns are: Run (which experiment), Train size (the image side length in training), Test size (the side length at test time), Top-1, GPU-hours and Memory.

- **Top half (3 epochs, holdout)**:
  - Baseline: train at 224, test at 224: 73.38%;
  - The same model, with only the test images enlarged to 288: 75.50% (+2.1), with no extra training at all;
  - Train at 288, test at 288: 77.23% (+3.9), 0.68 GPU-hours, 30.2 GB;
  - Train at 288, test at 320: 77.78% (+4.4), highlighted in blue. This is the best test size.
- **Bottom half (8 epochs, all data, flip TTA)**:
  - Our final model: train at 224, test at 224, val 76.22%, test 76.09%, 1.03 GPU-hours;
  - The same recipe, trained at 288 and tested at 320: no test score, 1.61* GPU-hours (the star marks that it is over the 1.2-hour budget).

Calculation: the number of tokens and the amount of computation

Number of tokens = $(\text{side length} \div 16)^2 + 1$ (the extra 1 is [CLS]):

$$224: \ 14^2 + 1 = 197, \qquad 288: \ 18^2 + 1 = 325, \qquad 320: \ 20^2 + 1 = 401.$$

How does the computation change? Most of ViT's computation is in the linear layers (qkv, proj, fc1, fc2), which grow in proportion to the number of tokens: $325 \div 197 \approx 1.65$ times. The attention step, where "each token looks at all tokens", grows with the square of the number of tokens: $(325 \div 197)^2 \approx 2.7$ times. But it is only a small part of ViT-B's total computation. Together this gives about 1.7 times, which is what the report means by "about 1.7 times more expensive". The memory grows too: 18.9 GB $\rightarrow$ 30.2 GB.

How the position embeddings fit the new size

During pretraining, the position embeddings were learned for a 14×14 grid. At 288 pixels, the grid becomes 18×18. timm interpolates the original 14×14 "position map" to 18×18, like enlarging a photo (20×20 at 320 pixels). This way every new position gets a sensible position vector, and no parameters are added.

Why larger test images help: FixRes

In training we use RandomResizedCrop: randomly cut out 25% to 100% of the image's area, then resize it to 224×224. The smaller the cut-out part, the more the object is enlarged.

An extreme example

Suppose you cut out 25% of the area, which is half of the side length, and resize it to 224×224. Then the object in the training image is enlarged 2 times. At test time we do not cut out small parts. We shrink the whole image and take the middle (resize to 248, then cut out the central 224). So objects in test images "look smaller than in training". If you enlarge the test image to 288, the objects get bigger and closer to the size the model is used to from training, so the accuracy goes up.

This is called a mismatch between the training and test resolutions. The FixRes paper (Touvron et al., 2019) studies it. After training at 288, the best test size also moves up, to 320, for the same reason.

The full test-size sweep (from the report's resolution table)

The slide shows only a few key points. The report tested the two 3-epoch models at 6 test sizes (holdout top-1, in %; bold marks the best value in each row):

Train size \ test size	224	256	288	320	352	384
224 pixels	73.38	75.21	**75.50**	75.28	74.33	72.94
288 pixels	73.10	75.74	77.23	**77.78**	77.63	77.41

How to read this table:

- **Each row first goes up, then down.** A test image slightly bigger than the training image is best (the model trained at 224 is best at 288; the model trained at 288 is best at 320). If you go bigger still, objects look bigger than anything the model saw in training, so the accuracy drops again (the model trained at 224 gets only 72.94% at 384).
- **The bigger the test image, the clearer the advantage of training at 288.** At the same test size, the model trained at 288 beats the one trained at 224: by 0.5 points at 256, 1.7 at 288, 2.5 at 320 and 4.5

at 384. Bigger images have more detail, and the model trained on big images has learned to use that detail.

- **One small exception.** Tested at 224, the model trained at 288 (73.10%) is slightly below the one trained at 224 (73.38%): it is already used to bigger inputs. This difference is within the noise.

Why the last row is not one of our results

- **Found too late**: the resolution experiments are an analysis done after the competition deadline.
- **Over budget**: the 8-epoch recipe at 288 pixels needs 1.61 GPU-hours, more than 1.2 hours (even with the 10% it is only 1.32).
- **No test score**: no more submissions were possible after the deadline.

So it only appears as a “lesson and next step”. The 3-epoch experiment at 288 pixels in the top half used only 0.68 GPU-hours, within the budget, but it too was tested only on the holdout and has no test score.

Key point

The lesson of this slide: we studied “how to fine-tune” early, but “what the input looks like” late. The +4.4 points from resolution are bigger than the gain from any single fine-tuning setting. The next step: within the budget, trade “a few fewer epochs” for “bigger images”.

4.12 All the formulas

Every formula below is explained in the same order: first the formula, then what each symbol means and what the formula says, and finally a small calculation. If a symbol is unclear, read the "what it says" sentence first.

4.12.1 softmax: turning scores into probabilities

softmax

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

- z_k: the score (logit) of class k. K: the number of classes, 10,000 for us.
- e^{z_k}: the exponential makes every score positive and makes the gaps bigger.
- The denominator: the sum of the exponentials of all classes, so that all the p_k add up to 1.

What it says: a higher score means a higher probability. Even a slightly higher score becomes a clearly higher probability after the exponential.

Example

The scores of three classes are $z = (2, 1, 0)$: $e^2 = 7.389$, $e^1 = 2.718$, $e^0 = 1$, and the sum is 11.107. The probabilities are $(0.665, 0.245, 0.090)$, and they add up to 1.

4.12.2 Cross-entropy: how wrong is the answer?

Cross-entropy

$$L = -\log p_y$$

y is the correct class, p_y is the probability that the model gives to the correct class, and log is the natural logarithm.
What it says: the closer the probability of the correct class is to 1, the closer the loss is to 0. The closer it is to 0, the bigger the loss, and it grows very fast.

Example

Continuing the example: if class 1 is correct, $L = -\ln 0.665 = 0.41$. If class 3 is correct, $L = -\ln 0.090 = 2.41$. If the probability is spread evenly over 10,000 classes (0.0001 each), $L = -\ln 0.0001 = \ln 10{,}000 = 9.21$. This is the loss of "pure guessing".

4.12.3 Label smoothing: do not be too sure

Label smoothing

$$q_k = (1 - \varepsilon)\,\mathbf{1}[k = y] + \frac{\varepsilon}{K}, \qquad L = -\sum_{k=1}^{K} q_k \log p_k$$

- q_k: the probability that class k "should get" in the training target. $\varepsilon = 0.1$: the share that is taken out and spread evenly over all classes.

- $\mathbf{1}[k = y]$: equals 1 if k is the correct class, and 0 otherwise.

Our numbers: $K = 10{,}000$, so the target for the correct class is $0.9 + 0.1/10{,}000 = 0.90001$, and every other class gets 0.00001.
What it says: do not ask the model to be 100% sure of the correct answer. Leave a little room. This prevents overconfidence and reduces memorization.

A 3-class example ($\varepsilon = 0.1$)

The target is $q = (0.933, 0.033, 0.033)$. With the prediction $p = (0.665, 0.245, 0.090)$:

$$L = -(0.933 \ln 0.665 + 0.033 \ln 0.245 + 0.033 \ln 0.090) \approx 0.381 + 0.047 + 0.080 \approx 0.51,$$

while the plain cross-entropy is 0.41. If the model gave the correct class 100% probability, the other classes would get 0. Since $\log 0$ is minus infinity, the loss would become infinite. So with smoothing, the best prediction is "very confident, but not absolutely sure".

4.12.4 Accuracy, and how big the noise is

Top-1, top-5 and the standard error

$$\text{top-1} = \frac{\text{images whose first answer is correct}}{\text{all images}}$$

$$\text{top-5} = \frac{\text{images with the correct answer in the top 5}}{\text{all images}}$$

$$\text{standard error} = \sqrt{\frac{p\,(1-p)}{N}}$$

p is the accuracy and N is the number of test images. The standard error shows how much the accuracy moves randomly just because we tested on these N images and not on others.

Example

- 5 images, where the model ranks the correct answer 1st, 1st, 3rd, 1st and 8th: top-1 = 3/5 = 60%, top-5 = 4/5 = 80%.
- Our holdout: $N = 20{,}000$ and $p \approx 0.72$: $\sqrt{0.72 \times 0.28/20{,}000} \approx 0.0032$, that is, about 0.32 percentage points. So differences under 0.3 points on the holdout cannot be taken seriously.

4.12.5 L2 normalization and cosine similarity

Formula

$$\hat{z} = \frac{z}{\|z\|}, \quad \|z\| = \sqrt{z_1^2 + z_2^2 + \cdots + z_d^2}, \qquad \cos(a, b) = \frac{a \cdot b}{\|a\|\,\|b\|} = \frac{\sum_i a_i b_i}{\|a\|\,\|b\|}$$

$\|z\|$ is the length of a vector. $a \cdot b$ is the dot product (multiply the numbers in matching positions, then add them up).
What it says: normalizing removes the length and keeps only the direction. Cosine similarity compares only directions: 1 is the same direction, 0 is unrelated, and -1 is exactly the opposite direction.

Example

$z = (3, 4)$: its length is $\sqrt{9+16} = 5$, and normalized it is (0.6, 0.8). $\cos\big((0.6, 0.8), (0.707, 0.707)\big) = 0.990$: very close in direction. $\cos\big((1, 0), (0, 1)\big) = 0$: at right angles. $\cos\big((1, 0), (-1, 0)\big) = -1$: opposite directions.

4.12.6 The score of the prototype head

Formula

$$\text{logit}_c \approx \tau \cos(z, \mu_c), \qquad \tau = 40$$

μ_c is the prototype of class c (the normalized average direction), and z is the feature of one image. The full derivation and the 3-class example are in the "Slide 7" section of this chapter.

Example

If an image has a cosine of 0.990 with one class's prototype, its score is $40 \times 0.990 = 39.6$. If it has a cosine of 0.569 with another class, the score is 22.8. The scores differ by 16.8, so after softmax almost all of the probability goes to the first class.

4.12.7 Self-attention

Self-attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

- Q, K, V: three sets of vectors that the qkv layer computes from the same tokens (197 tokens, 64 numbers per head).
- $QK^\top$: the Q of every token is multiplied (dot product) with the K of every token. This gives a 197 $\times$ 197 "relatedness table".
- $\sqrt{d}$: divide by $\sqrt{64} = 8$, so the dot products do not get too big and make the softmax too sharp.
- softmax: turns each row into weights that add up to 1. Multiplying by V at the end takes a weighted average of the content of all tokens.

What it says: the new representation of each token is a weighted average of the content of all tokens, and more related tokens get bigger weights. The 197 $\times$ 197 table also explains why the attention computation grows with the square of the number of tokens.

A small example with 2 tokens and 2 numbers each

Token 1 has $q = (1, 0)$. The two tokens have k = (1, 0) and (0, 1), and v = (10, 0) and (0, 10). The dot products are 1 and 0. Divided by $\sqrt{2}$ they are 0.707 and 0. Softmax: $e^{0.707} = 2.03$ and $e^0 = 1$, so the weights are 0.67 and 0.33. Output $= 0.67 \times (10, 0) + 0.33 \times (0, 10) = (6.7, 3.3)$. Token 1 mainly "listens" to token 1, which matches it, but it also takes in some of token 2's information.

4.12.8 LoRA: the forward pass, the number of parameters, and merging

LoRA

$$y = W_0 x + \frac{\alpha}{r} BAx, \qquad \text{parameters} = r\,(d_{\text{in}} + d_{\text{out}}), \qquad W' = W_0 + \frac{\alpha}{r} BA$$

- W_0: the frozen original weights, with shape $d_{\text{out}} \times d_{\text{in}}$.
- A ($r \times d_{\text{in}}$) and B ($d_{\text{out}} \times r$): the two thin matrices that are trained. B starts at 0.
- $r = 8$: the rank. $\alpha = 16$: the scaling; the actual multiplier is $\alpha/r = 2$.
- W': the new weights after merging at the end of training. It has the same shape as W_0.

What it says: a change to a big matrix can be written as the product of two thin matrices, which needs far fewer parameters. After training it is added back, so inference is as fast as with the original model.

Example

fc1: $d_{\text{in}} = 768$ and $d_{\text{out}} = 3{,}072$. LoRA has $8 \times (768 + 3{,}072) = 30{,}720$ parameters, while the whole weight matrix of fc1 has $768 \times 3{,}072 = 2{,}359{,}296$ numbers. LoRA uses only about 1.3% of that. For a numerical example of merging, see the subsection "Why LoRA can be merged but the Adapter cannot" above.

4.12.9 Adapter and AdaptFormer

Formula

$$\text{Adapter:} \quad h \leftarrow h + W_{\text{up}}\,\text{ReLU}(W_{\text{down}}\,h)$$

$$\text{AdaptFormer:} \quad x_{l+1} = x' + \text{MLP}(\text{LN}(x')) + s \cdot W_{\text{up}}\,\text{ReLU}(W_{\text{down}}\,x')$$

- W_{down}: $768 \to 64$. W_{up}: $64 \to 768$, which starts at 0.
- ReLU: negative numbers become 0, positive numbers stay the same.
- h: the output of the MLP. x': the result after the attention sub-layer. $s = 0.1$: AdaptFormer's scaling factor.

What it says: a small "compress → non-linearity → restore" module learns a small correction and adds it back to the main path. The Adapter changes the output of the MLP (in series). AdaptFormer adds a branch next to the MLP (in parallel).

Example

ReLU(-2) = 0 and ReLU(3) = 3. Suppose the branch outputs 5.0 at some position. AdaptFormer multiplies it by $s = 0.1$ and adds only 0.5, which is a gentle correction. The Adapter has no such factor and adds the 5.0 directly.

4.12.10 Layer-wise learning-rate decay

Layer-wise LR decay

$$\text{lr}_l = \text{lr} \times 0.75^{\,13-l}$$

- l: the layer number. The bottom (patch embedding, [CLS], position embeddings) is layer 0. The 12

blocks are layers 1 to 12. The final LayerNorm and the head are layer 13.

- lr = 1e-4: the base learning rate of full fine-tuning. 0.75: the factor for each layer down.

What it says: the closer a layer is to the input, the smaller its learning rate. The head at the top learns fastest, and the bottom layers barely move. This protects the general features learned in pretraining. Only full fine-tuning uses it; LoRA does not.

Example

Layer	Factor	Learning rate
Layer 13 (head, final LN)	$0.75^0 = 1$	1.0e-4
Block 12	$0.75^1 = 0.75$	7.5e-5
Block 6	$0.75^7 \approx 0.133$	1.3e-5
Block 1	$0.75^{12} \approx 0.032$	3.2e-6 (about 3%)
Layer 0 (patch embedding, etc.)	$0.75^{13} \approx 0.024$	2.4e-6

4.12.11 Warmup plus cosine schedule

How the learning rate changes with the step

$$f(s) = \begin{cases} \dfrac{s+1}{S_w}, & s < S_w \\ \dfrac{1}{2}\left(1 + \cos\dfrac{\pi\,(s - S_w)}{S - S_w}\right), & s \geq S_w \end{cases} \qquad \mathrm{lr}(s) = \mathrm{lr} \times f(s)$$

s is the current step, S is the total number of steps, and S_w is the number of warmup steps.
What it says: during the first S_w steps, the learning rate rises in a straight line from near 0 to its maximum. After that, it goes down to 0 along a cosine curve.

The final full model: $S = 14{,}056$ (8 epochs of 1,757 steps), $S_w = 1{,}757$ (1 epoch)

- Step 878 (halfway through warmup): $f = 879/1{,}757 \approx 0.5$, so the learning rate is about 5e-5.
- Step 1,757 (the end of warmup): $f = \frac{1}{2}(1 + \cos 0) = 1$, so the learning rate is 1e-4, its maximum.
- About step 7,907 (halfway through the cosine part): $\cos(\pi/2) = 0$, so $f = 0.5$ and the learning rate is 5e-5.
- The last step: $\cos\pi = -1$, so $f \approx 0$ and the learning rate is close to 0.

LoRA has 3 epochs of 1,679 steps, 5,037 steps in total, with 0.3 epochs of warmup (about 503 steps). The shape is the same.

4.12.12 One step of AdamW (simplified)

AdamW

$$m \leftarrow \beta_1 m + (1 - \beta_1)\, g, \qquad v \leftarrow \beta_2 v + (1 - \beta_2)\, g^2$$

$$\theta \leftarrow \theta - \mathrm{lr} \cdot \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon} - \mathrm{lr} \cdot \lambda\, \theta$$

- θ: one parameter. g: its gradient.

- m: a running average of the gradient. It shows the "direction", with a little momentum. v: a running average of the squared gradient. It shows "how big this parameter's gradient usually is".
- $\hat{m}$, $\hat{v}$: corrected versions for the first few steps. β_1, β_2: the "memory length" of the averages (the PyTorch defaults are 0.9 and 0.999). ϵ: a very small number that prevents dividing by 0.
- λ: the weight decay factor. The last term pulls the parameter toward 0, and it is handled separately from the gradient. This is the "W" in AdamW (decoupled weight decay).

What it says: the step size of each parameter adjusts itself. After dividing by $\sqrt{\hat{v}}$, a parameter whose gradients are always large does not take steps that are too big, and a parameter whose gradients are small does not take steps that are too small.

The first step: $\theta = 0.5$, $g = 0.2$, lr = 1e-4, $\lambda = 0.05$

After the correction, $\hat{m} = 0.2$ and $\hat{v} = 0.04$, so $\hat{m}/\sqrt{\hat{v}} = 0.2/0.2 = 1$.

$$\theta \leftarrow 0.5 - 0.0001 \times 1 - 0.0001 \times 0.05 \times 0.5 = 0.5 - 0.0001 - 0.0000025 = 0.4998975.$$

If the gradient were 2.0 (10 times bigger), the first change would still be about 0.0001. Adam does not care much about the overall size of the gradient; the step size is mainly set by the learning rate.

4.12.13 GPU-hours and the budget

Formula

$$\text{GPU-hours} = \text{number of GPUs} \times \text{hours}$$

$$\text{budget} = \frac{2\text{ h} \times 48\text{ GB}}{80\text{ GB}} = 1.2\text{ h} \quad (\pm 10\%, \text{ at most } 1.32\text{ h})$$

Example

We use only 1 A100, so the GPU-hours equal the hours. A quick check: the final full model used 1.03, within the budget; LoRA used 0.54, within the budget; 288 pixels with 3 epochs used 0.68, within the budget; the 8-epoch recipe at 288 pixels would need 1.61, over the budget.

4.12.14 The share of parameters

Formula

$$\text{share} = \frac{\text{trained backbone parameters}}{85{,}798{,}656} \times 100\%$$

Following the course reference, the 7.69M head is not counted in the share (the course's LoRA figure of 0.34% does not count the head either).

Example

LoRA: $1{,}179{,}648 \div 85{,}798{,}656 \approx 1.375\%$. BitFit: $102{,}912 \div 85{,}798{,}656 \approx 0.120\%$. If you count the head too, LoRA really trains $1{,}179{,}648 + 7{,}690{,}000 = 8{,}869{,}648$ parameters, and the head is about 87% of them.

Chapter 5 Presentation Q&A

This chapter is a question bank. We collected 37 questions that the instructor is likely to ask. Each question has four parts:

- **The question**: how the instructor will probably ask it.
- **The explanation**: what the question is really asking, the idea behind it, and the numbers you need. If you understand this part, you can answer even when the question is asked in a different way.
- **An answer you can say**: 2 to 5 sentences of spoken English that you can use directly.
- **Possible follow-up questions**: what you may be asked next, and how to answer.

The "Speaker" line at the start of each question says who should answer it. The table below shows how A, B, C and D share the topics.

5.1 How to answer questions

Three steps to a good answer

1. **Answer first**: your first sentence answers the question directly. Do not start with background.
2. **Then give one key number**, such as "+2.5 points" or "1.03 GPU-hours". A number makes the answer convincing.
3. **Then give the reason**, in one or two sentences.

If you do not know the answer, or we did not run that experiment, say honestly "we did not test that". Then give your guess and say how you would test it. The instructor's email said that a clear, honest process matters more than chasing a high score.

Who answers which questions

Speaker	Topics	Slides
A	Budget, GPU-hours, the base model (the backbone), data, hold-out, integrity	Slides 2, 4, 12
B	Method details, numbers of weights, latency, LoRA vs. Adapter, the experiment process	Slides 3, 5, 10, 11 and backups 16–20
C	The prototype head vs. the linear-probe head, and the evidence	Slides 6–8
D	The gap between PEFT and full fine-tuning, fairness, limits, resolution, the training recipe	Slides 10–14

Useful English phrases

Phrase	When to use it
Good question. In short, …	Give the answer first, then explain
The key number here is …	Bring in one key number
You can see this on slide …	Take the instructor back to a slide
We did not test that, but our guess is …, and we would test it by …	An experiment we did not run: answer honestly
That is a fair point, and it is one of our limits.	The instructor points out a weakness: admit it first
The difference is within our noise of about 0.3 points.	When a gap is very small
Just to make sure I understand: are you asking about …?	You did not hear or did not understand the question
My teammate worked on this part, so let me hand it over.	Pass the question to the teammate in charge

Things you must not do

- **Do not make up numbers.** If you are not sure, give a rough number, such as "about 2 points", and point to the slide.
- **Do not say that the 288 px model has a test score.** It was only evaluated on the holdout. It

has no test score, and it is not one of our official results.

- **Do not guess what other teams did.**
- **Do not mix up the three kinds of scores**: holdout (our own 20,000 held-out photos), validation (the CodaBench validation set) and test (the CodaBench test set). When you give a number, say which one it is.

5.2 Basic concepts

Q1. What is transfer learning, and what does fine-tuning mean in your project?

Speaker: A

Transfer learning means: first learn general skills on a large dataset, then reuse them for a new task. Our ViT-B/16 was trained first on ImageNet-21k and then on ImageNet-1k. So it already knows how to see general things in images, such as edges, textures and shapes.

Fine-tuning means we take this trained model and keep training it for a short time on our new data. Our data has 10,000 classes, with 45 photos per class. There are two main ways to fine-tune:

- **Full fine-tuning**: all 85.8 million weights (the model's adjustable numbers, also called parameters) are updated.
- **Parameter-efficient fine-tuning (PEFT)**: most weights stay frozen (fixed), and only a small part is trained.

We tried both kinds and compared 7 methods in total.

Everyday analogy

Think of a person who already knows how to draw. To learn to draw one kind of bird, they do not start by learning how to hold a pencil. They just practice the new subject a few more times.

Answer you can say

Transfer learning means we start from a model that already learned general visual features on large datasets, here ImageNet-21k and then ImageNet-1k. Fine-tuning means we keep training this model on our own task: 10,000 classes with 45 photos each. We compared full fine-tuning, which updates all 85.8 million weights, with parameter-efficient methods that update only a small part.

Q2. What is the difference between the backbone and the head?

Speaker: B

The **backbone** is the large main part of the network. It looks at the image and extracts features. Our backbone is ViT-B/16, with 85.8 million weights. It turns an image into a feature vector of 768 numbers.

The **head** is the last layer. It turns these 768 numbers into one score for each class. Our head is a linear layer from 768 to 10,000. Its number of weights is $768 \times 10{,}000 + 10{,}000 = 7{,}690{,}000$, or 7.69 million.

The original head of the pretrained model was made for the 1,000 classes of ImageNet, so we cannot use it. We must replace it with a new 10,000-class head. This new head knows nothing at first. How we initialize it (how we set its starting values) turned out to be the most important finding of our project (see slides 6 to 8).

Everyday analogy

The backbone is like your eyes and the part of your brain that sees. The head is the last step, where you say the name of what you see. For a new task, you can keep using your eyes, but you must learn the new names.

Answer you can say

The backbone is the big pretrained network that turns an image into a feature vector: for ViT-B/16, that is 85.8 million weights and a feature of 768 numbers. The head is the last layer that turns this feature into one score per class. Our head is new, 768 to 10,000, with 7.69 million weights, because the pretrained head was made for ImageNet's 1,000 classes. How this new head starts turned out to be our most important design choice.

Possible follow-up questions

Follow-up: Why is the head so large?
Answer: Because there are 10,000 classes. Each class needs its own 768 weights plus a bias, so 7.69 million in total, about 9% of the backbone. And each class row is learned from only 43 to 45 photos.

Q3. What does parameter-efficient fine-tuning mean, and why would anyone use it?

Speaker: B

Parameter-efficient fine-tuning (PEFT) means we freeze (do not change) almost all weights of the pretrained model. We train only a small number of new or selected weights. We compared four PEFT methods:

- **BitFit**: trains only the biases (the small offset numbers in each layer), 0.12% of the backbone;
- **Adapter** and **AdaptFormer**: add a small "bottleneck" module to every block, about 1.39%;
- **LoRA**: adds two very small low-rank matrices next to each linear layer, 1.38%.

Note: these percentages do not include the head. The head is trained in every method.

The main benefit of PEFT is **storage**. Many tasks can share one backbone, and each task stores only a small file.

Easy to get wrong

PEFT saves the number of weights you train and store. It does not always save training memory or time. Our LoRA run used 19.9 GB of GPU memory, even more than full fine-tuning at 18.9 GB. The reason is in the section "Comparing the methods" of this chapter.

Answer you can say

PEFT means we freeze almost all pretrained weights and train only a small number of new or selected ones. We tried four: BitFit trains only the biases, 0.12% of the backbone; Adapter and AdaptFormer add small bottleneck modules, about 1.4%; LoRA adds low-rank matrices, 1.38%. The head is trained in every method. The main benefit is storage: one shared backbone plus a small file per task.

Possible follow-up questions

Follow-up: Does PEFT also save training memory?
Answer: Not always. In our runs LoRA needed 19.9 GB, more than full fine-tuning at 18.9 GB,

because the gradients still flow back through all 12 blocks.

Q4. What exactly does your model take in, and what does it output?

Speaker: A

Input: one color photo, resized to 224×224 pixels, with three color channels (red, green and blue). During training we also crop parts of the photo at random and flip it left to right. This is called data augmentation.

Output: 10,000 scores, one for each class. These raw scores are called **logits**. A softmax turns the scores into probabilities that add up to 1. The class with the highest probability is the prediction.

- **Top-1 accuracy**: the share of photos where the class with the highest probability is the right answer. This is the main number we report.
- **Top-5 accuracy**: a photo counts as correct if the right answer is among the 5 classes with the highest probabilities.

A small softmax example (only 3 classes)

Say the scores are $[2.0,\ 1.0,\ 0.1]$. First take the exponent of each: $e^{2.0} = 7.39$, $e^{1.0} = 2.72$, $e^{0.1} = 1.11$. They add up to 11.22. Then divide each one by the total. The probabilities are about $[0.66,\ 0.24,\ 0.10]$, so the prediction is class 1.

Answer you can say

The input is one RGB image, resized to 224 by 224 pixels. The output is 10,000 scores, one per class. A softmax turns the scores into probabilities, and the class with the highest probability is our prediction. Top-1 accuracy is the share of images where this prediction is correct, and that is the main metric we report.

Q5. How do you measure performance, and why should we trust your holdout set?

Speaker: A

The labels of the validation set (50,000 photos) and the test set (100,000 photos) are hidden. You only see a score after you upload predictions to CodaBench, and the number of uploads is limited. So we took 2 photos per class out of the training set, 20,000 photos in total, and used them as our own practice exam. We call this set the **holdout**. These photos:

- are never used for training, only for measuring accuracy;
- are chosen with a fixed random seed (seed 0), so every experiment uses the same photos.

Why we can trust it: its scores are very close to the CodaBench scores. For full fine-tuning it is 72.31% against 72.57%, and for LoRA 71.98% against 71.90%. Both gaps are within 0.3 points.

Everyday analogy

Before an exam, you keep one set of past papers that you have never done, and use it as a mock exam. If your mock-exam score is close to your real exam score, the mock exam is reliable. Then you can safely use it to compare different ways of studying.

Answer you can say

The validation and test labels are hidden, and submissions are limited, so we held out 2 training photos per class, 20,000 photos, as our own test set. They are never used for training, and the split is fixed. It tracks CodaBench closely: for LoRA, 71.98 on the holdout against 71.90 on validation, and full fine-tuning is also within 0.3 points. So all our comparisons use the holdout.

Possible follow-up questions

Follow-up: Doesn't holding out 2 photos per class hurt the final model?
Answer: A little. For full fine-tuning, training on all 45 photos instead of 43 added 0.26 points, so our final full model uses all 45.

5.3 How the techniques work

Q6. How does a Vision Transformer turn an image into a prediction?

Speaker: B

A ViT (Vision Transformer) works in five steps:

1. **Cut into patches**: the 224 × 224 image is cut into 16 × 16 squares, which gives $14 \times 14 = 196$ small patches. The "16" in "B/16" means patches of 16 pixels.
2. **Turn into vectors**: each patch becomes a vector of 768 numbers, called a **token**. A special [CLS] token is added, so there are 197 tokens.
3. **Add position information**: each token gets a **position embedding**, which tells the model where the patch sits in the image.
4. **Twelve Transformer blocks**: each block first does **self-attention**, so every token can use information from all other tokens. Then a small network, the **MLP**, processes each token on its own.
5. **Output**: the final 768 numbers of the [CLS] token are the feature of the whole image. The head turns them into scores.

Everyday analogy

196 people each hold one piece of a puzzle, and they hold 12 rounds of meetings. In each meeting, every person first listens to what the others hold (attention), and then thinks on their own (MLP). At the end, the chair, [CLS], sums up what the whole picture shows.

Answer you can say

ViT-B/16 cuts the 224-pixel image into 16-by-16 patches, which gives 196 patches. Each patch becomes a token of 768 numbers, and a special class token is added, so 197 tokens. Twelve Transformer blocks then mix information between all tokens with self-attention, and process each token with a small MLP. The class token's final 768 numbers are the image feature, and the head turns them into 10,000 class scores.

Possible follow-up questions

Follow-up: What is self-attention, in simple words?
Answer: Each token asks which other tokens are relevant to it, gives them weights, and takes a weighted average of their information. So a patch of a bird's wing can use information from the bird's head and from the background.

Follow-up: How many weights are in one block?
Answer: About 7.09 million. The MLP holds two thirds of them, which is why adding LoRA to the MLP helped.

Q7. Which loss function do you use, and why label smoothing?

Speaker: D

We use the **cross-entropy** loss. It means: the higher the probability the model gives to the correct class, the smaller the loss.

Cross-entropy

$$L = -\log p_y, \qquad p_y = \frac{e^{z_y}}{\sum_{j=1}^{10{,}000} e^{z_j}}$$

z_j is the score (logit) of class j; y is the correct class; p_y is the probability that the softmax gives to the correct class.
Example: if $p_y = 0.9$, then $L = 0.105$; if $p_y = 0.01$, then $L = 4.6$. The more wrong the answer, the bigger the penalty.

Label smoothing (we use 0.1): we do not ask the model to be 100% sure of the right answer. Instead, the target is 90% on the correct class, and the other 10% is spread evenly over all classes. With only 45 photos per class, the model can easily become over-confident and memorize the training photos. Label smoothing reduces this.

Answer you can say

We use cross-entropy, the standard loss for classification: it is small when the model gives a high probability to the correct class. We add label smoothing of 0.1, so the target is not 100 percent on one class but 90 percent, with the rest spread over all classes. With only 45 photos per class, this keeps the model from becoming over-confident on the training photos.

Possible follow-up questions

Follow-up: What loss does a random guess give?
Answer: About the natural log of 10,000, which is 9.21. Our randomly initialized head started at 9.28, very close to that.

Q8. How does your prototype head work, step by step?

Speaker: C

There are four steps, and **none of them uses a gradient update**:

1. Run the frozen backbone over all 450,000 training photos and get a 768-number feature for each one. This takes about 6 minutes, and we count it in the GPU budget.

2. Normalize each feature to length 1, so only its direction is kept. Add up all features of the same class, and normalize again. The result is the class **prototype**: the average direction of the class.
3. Multiply the prototypes by a temperature of 40 and copy them into the head as its weights. Set the biases to 0.
4. At prediction time, the score of a photo for a class is about $40 \times \cos(\text{photo feature},\ \text{class prototype})$. The more a photo looks like the "average look" of a class, the higher its score.

A small 2-D example

A class has two photos with features $(3, 4)$ and $(6, 8)$. After normalizing, both become $(0.6, 0.8)$. Their sum is $(1.2, 1.6)$, and normalizing again still gives $(0.6, 0.8)$. This is the class prototype.
A new photo has the normalized feature $(0.8, 0.6)$. The cosine similarity is $0.6 \times 0.8 + 0.8 \times 0.6 = 0.96$, so the score is $40 \times 0.96 = 38.4$.

Everyday analogy

To recognize people, you first "average" each person's photos into one standard face. Then you check which standard face a new photo looks most like.

Answer you can say

First, we run the frozen backbone once over all training photos and keep each 768-number feature. Then, for each class, we average the normalized features and normalize again; that is the class prototype. We copy the prototypes, times a temperature of 40, into the head. So each score is about 40 times the cosine similarity between a photo and its class mean. No gradient step is needed, so the head never fits individual training photos.

Possible follow-up questions

Follow-up: How long does it take?
Answer: About six minutes on the A100, and we count that time in our GPU budget.

Follow-up: How accurate is the prototype head before any training?
Answer: 44 percent on the holdout. The linear-probe head starts higher, at 62 percent, but ends lower after fine-tuning.

Q9. How did you choose the temperature of 40?

Speaker: C

Cosine similarity is always between -1 and 1. If we used it directly as the score, the scores of all 10,000 classes would sit in a very narrow range. After the softmax, every class would get almost the same probability, and the model would seem unsure about everything. Multiplying by a temperature s spreads the scores apart.

We tried 8 values: {5, 10, 15, 20, 30, 40, 60, 80}. For each value, we computed the cross-entropy on 10,000 training features, and we picked the lowest. The winner was 40. This only picks one number. It does not train the weights of the head.

Why we scale the scores up

Two classes have cosine similarities of 0.5 and 0.3.

- Without scaling, the softmax probabilities are about 0.55 and 0.45, so you can hardly tell them apart.
- Times 40, the scores become 20 and 12, and the probabilities are about 0.9997 and 0.0003. Now the difference is clear.

Answer you can say

Cosine similarity is always between minus one and one, so without scaling, the softmax over 10,000 classes would be almost flat. We tried eight values from 5 to 80 and picked the one with the lowest cross-entropy on 10,000 training features, which was 40. It is one number chosen by a simple search, not a trained head.

Q10. Why is a higher starting loss a good thing for the prototype head?

Speaker: C

Here are the average losses of the three heads over the first 100 steps of LoRA training:

- linear-probe head: 2.29 (the lowest);
- prototype head: 4.39;
- random head: 9.28 (the highest, close to random guessing, $\ln 10{,}000 = 9.21$).

These are averages of the training loss. They include label smoothing and data augmentation, so we can only compare which one is higher or lower. Do not turn them into "the probability of the correct class". You can read the three numbers like this:

- 9.28: knows nothing, like guessing at random among 10,000 classes;
- 4.39: roughly knows what each class looks like, but is far from sure, so the learning signal is strong;
- 2.29: has already been fitted on these training photos, so the learning signal is weak.

The linear-probe head was trained on **the same training photos**. So at the start it already "answers most training questions correctly", and its loss is low. A low loss means a small gradient (learning signal) going back to the backbone. And this gradient comes only from the few photos that the head still gets wrong. So the backbone learns very little.

The prototype head already points in the right direction (44% accuracy at the start), but it still has a lot to learn. So its learning signal is strong enough to push the backbone to adapt to the new task. The random head gives the strongest signal, but it points in random directions, so it learns the slowest at first: after epoch 1 it reaches 58.4%, while the prototype head reaches 63.3%. After 3 epochs, though, the two are level (71.95% against 72.00%).

Everyday analogy

A teacher marks homework. If a student copied the answer key (the linear-probe head was fitted on the training photos), almost everything is right, and the teacher has little to teach. The prototype head is like a student who answers from their own understanding. They make more mistakes, but the teacher's corrections really help them improve.

Answer you can say

The linear-probe head was fitted on the same training photos, so it starts with a low loss, 2.29. That means small gradients, coming from the few photos it still gets wrong, so the backbone barely adapts. The prototype head starts at 4.39: it already points in the right direction, at 44 percent accuracy, but it leaves a strong learning signal for the backbone. That is what we want, because our classes need the features to change.

Q11. How does LoRA work, and why can it be merged after training?

Speaker: B

LoRA (Low-Rank Adaptation) works like this:

- It does not change the original weight matrix W. Instead, it learns a small correction next to it: $\Delta W = BA$.
- A has size $r \times d_{\text{in}}$, and B has size $d_{\text{out}} \times r$. With $r = 8$, both matrices are very thin, so they hold few weights.
- The forward pass is $y = Wx + \frac{\alpha}{r} BAx$. We use $\alpha = 16$, so the scale is 2.
- B starts at 0, so at the start of training the model is exactly the original model.

After training, we add $\frac{\alpha}{r} BA$ into W: $W' = W + \frac{\alpha}{r} BA$. After this the model has exactly the original structure, so inference takes no extra time. This step is called **merging**.

Weights of the qkv layer

For the qkv layer, $d_{\text{in}} = 768$ and $d_{\text{out}} = 2304$.

- The original matrix: $768 \times 2304 = 1{,}769{,}472$ weights;
- LoRA: $8 \times 768 + 2304 \times 8 = 24{,}576$ weights, about 1.4% of the original matrix.

Over all four kinds of linear layers, each block has 98,304 LoRA weights. The 12 blocks have 1,179,648 in total, which is 1.375% of the backbone.

Answer you can say

LoRA keeps each pretrained weight matrix W frozen and learns a small update next to it, B times A, with rank 8. For example, next to qkv, A is 8 by 768 and B is 2304 by 8, which is only 24,576 weights. B starts at zero, so training starts from the original model. After training we add the update into W, so the deployed model has exactly the original structure and the same speed.

Possible follow-up questions

Follow-up: Why can't the adapters be merged in the same way?
Answer: Their bottleneck has a nonlinearity, a ReLU, between the two small layers. So the module is not a single matrix and cannot be added into W. LoRA's update is linear, so it can.

Follow-up: What does low-rank mean?
Answer: B times A is a 768-by-2304 matrix, but it is built from only 8 directions. So it can only make a simple, low-dimensional change, which is often enough for adapting a pretrained model.

Q12. What do the training tricks in your recipe do: AdamW, warmup and cosine, layer-wise decay, drop path?

Speaker: D

These are the settings of our final full fine-tuning model, with one or two sentences each:

- **AdamW**: the most common optimizer. It looks at each weight's past gradients and decides how big each step should be.
- **Weight decay** (0.05): at every step, pull the weights a little toward 0. This stops them from growing too large and reduces overfitting.
- **Warmup + cosine**: in the first epoch, the learning rate rises slowly from 0 to 10^{-4}. This avoids big early steps that could scramble the features learned in pretraining. After that, the learning rate falls slowly to 0 along a cosine curve, so the last steps are small and careful.
- **Layer-wise learning-rate decay** (0.75): layers closer to the input get a smaller learning rate. Each step down multiplies it by 0.75, so the first block gets only about 3%. Low layers hold general features, such as edges and textures, which need little change. Top layers are closer to the task and need more change.
- **Drop path** (0.1): during training, randomly skip the branch of some blocks, like random "days off". This forces the network not to rely too much on any one block, and it reduces overfitting.
- **Gradient clipping** (1.0): when a gradient is too large, shrink it, so that no single update is too big.
- **bf16**: compute with 16-bit numbers. It is faster and uses less memory, with almost no loss in accuracy.

Answer you can say

For full fine-tuning we use AdamW at a learning rate of 1e-4, with one warmup epoch and then a cosine decay to zero. The learning rate shrinks by 0.75 per block toward the input, so the first block gets only about 3 percent of it: low layers hold general features that should change little. Weight decay of 0.05 and drop path of 0.1 regularize the model, and gradient clipping at 1.0 keeps the updates stable.

Possible follow-up questions

Follow-up: Why does LoRA not use layer decay or drop path?
Answer: Drop path is mainly a regularizer for training many weights, and LoRA trains few, so we kept its setup simple: one learning rate of 1e-3 and weight decay of 1e-4. We now suspect that this weaker regularization is one reason LoRA overfits after 3 epochs.

5.4 Choosing the model and the backbone

Q13. Why did you use ViT-B/16, and why not a bigger or different backbone?

Speaker: A

There are four reasons:

1. **The project asks for it**: page 1 of the brief recommends this model (trained first on ImageNet-21k, then on ImageNet-1k). The instructor's email also said: do not change the backbone without a strong reason.
2. **Our pipeline is checked**: we reproduced the three reference results in the brief, all within 0.3 points (zero-shot 43.95 against 43.7, full fine-tuning 70.88 against 70.8, LoRA 68.57 against 68.3).
3. **We checked other versions**: after the deadline we tried two versions trained only on ImageNet-21k. Their linear probes reached only 61.05% and 51.07%, both below our 61.94%. So switching versions brings no gain.
4. **Budget**: a bigger model costs several times more per epoch, so the 1.2-hour budget would allow only a few epochs. We did not actually test this.

So we spent our effort on the adaptation methods, which is also what the instructor asked us to focus on.

Answer you can say

We kept ViT-B/16 because it is the backbone the brief recommends, and the instructor asked us not to change it without a strong reason. Our pipeline reproduces all three reference numbers within 0.3 points. We also checked two ImageNet-21k-only versions: their linear probes reach 61.05 and 51.07 percent, both below our 61.94. So we spent our effort on the adaptation methods instead.

Possible follow-up questions

Follow-up: Why compare backbones with a linear probe and not zero-shot?
Answer: Zero-shot only uses class means, so it is a weak test of the features. A linear probe trains a classifier on the frozen features, so it shows better how useful they are. One of the in21k-only versions even has a higher zero-shot score, 49.96 against our 43.95, but a lower linear probe.

Follow-up: Would a bigger backbone help?
Answer: Probably, but it costs several times more per epoch, so it would get only a few epochs in our 1.2-hour budget. We did not test it.

Q14. Why did you choose LoRA as your final parameter-efficient model?

Speaker: B

- **Accuracy**: with the same setup, LoRA was the most accurate of the 4 PEFT methods (71.98%), ahead of the Adapter (71.54%), AdaptFormer (71.17%) and BitFit (69.81%).
- **Inference speed**: LoRA merges into the original weights, so inference is as fast as the original model (7.61 ms). The two adapters are 2.4 to 2.9 ms slower.
- **Few weights**: it trains only 1.375% of the backbone, far below the 5% that the brief gives as an example.
- **Downside**: it trains the slowest (888 photos per second) and uses the most memory (19.9 GB).

Answer you can say

LoRA was the most accurate of the four PEFT methods under the same protocol, 71.98 percent on the holdout, with only 1.375 percent of the backbone trained. And after training it merges into the weights, so inference is as fast as the original model, 7.61 milliseconds per image, while the two adapters add 2.4 to 2.9 milliseconds. Its downside is training cost: it was the slowest to train and used the most memory.

5.5 Comparing the methods

Q15. What do you mean by "where you train beats how much"?

Speaker: B

This sentence compares two things: **where** the trained weights are, and **how many** weights are trained. There are two pieces of evidence:

1. LoRA trains only 1.38% of the weights, but it has some in every block, and it reaches 71.98%. Training only the last 4 blocks (33% of the weights) reaches 71.88%. The two are about the same.
2. BitFit trains only the biases, 0.12% of the weights, and reaches 69.81%. That is higher than training only the last block (8.3%), which reaches 67.97%.

Conclusion: small changes in every layer work better than big changes in the top layers. Our explanation: 10,000 fine-grained classes need the whole network, from the low layers to the high layers, to adjust a little. If only the top layers change, the low-level features never get a chance to adapt.

Everyday analogy

Redecorating a house: changing the lights and curtains in every room (small changes everywhere) often improves the whole house more than fully rebuilding one room (a big change in one place).

Answer you can say

It means the location of the trained weights matters more than their number. LoRA trains 1.38 percent of the weights, spread over every block, and matches training the last four blocks, which is 33 percent. BitFit trains only the biases, 0.12 percent, and beats training the last block, 8 percent. So a little capacity in every block beats a lot of capacity in a few top blocks.

Possible follow-up questions

Follow-up: Why would every block need to change?
Answer: Our 10,000 classes are fine-grained, so even low-level features like textures and shapes need some adjustment. Training only the top blocks leaves the lower blocks exactly as they were pretrained.

Q16. Why does LoRA need more memory than full fine-tuning, although it trains far fewer weights?

Speaker: B

During training, most memory goes to **activations**. In the forward pass, the output of every layer must be stored, so that the backward pass can compute the gradients. So memory depends mainly on how far back the gradients must go, not on how many weights are trained.

- **Partial fine-tuning, K = 4**: the gradients only go back through the last 4 blocks, so only those blocks' activations are stored. It needs just 7.0 GB.
- **LoRA**: every block has LoRA, so the gradients must go all the way back to block 1. Also, to compute the gradient of A, the input of every linear layer must be stored (just like in full fine-tuning). On top of that come the activations of 96 extra small matrix multiplications (12 blocks $\times$ 4 linear layers $\times$ 2 matrices). So it needs 19.9 GB.
- **Full fine-tuning**: the only extra part is the optimizer state. For 85.8 million weights this is about 1 GB, which is small at a batch size of 256. So it needs 18.9 GB.

Answer you can say

Training memory is mostly the activations stored for the backward pass, so it depends on how far back the gradients must flow, not on how many weights we train. LoRA sits in every block, so the gradients go all the way back, and to train its A matrices it must keep every linear layer's input, just like full fine-tuning, plus its own small activations. Full fine-tuning adds only about 1 GB of optimizer state on top. Training only the last four blocks needs 7 GB, because the gradients stop after four blocks.

Possible follow-up questions

Follow-up: Then why do BitFit and the adapters need less, 12.7 to 12.8 GB?
Answer: Their gradients also flow through every block, so they need more than partial fine-tuning. We think they need less than full fine-tuning because they do not have to keep the inputs of the frozen linear layers to compute weight gradients.

Q17. Why are Adapter and AdaptFormer slower at inference, but LoRA is not?

Speaker: B

- **Adapter**: adds a small bottleneck **in series** after the MLP of every block: $768 \to 64 \to 768$, with a ReLU in the middle.
- **AdaptFormer**: puts the same bottleneck **in parallel**, next to the MLP. It multiplies the bottleneck's output by 0.1 and adds it back.

These modules need extra computation at inference. And because of the ReLU (a nonlinear function) in the middle, they cannot be merged into the original weight matrices. At batch size 1, the plain model takes 7.33 ms, the Adapter 9.74 ms and AdaptFormer 10.23 ms. That is 2.4 to 2.9 ms more, or about 33% to 40%.

LoRA's correction BA is purely linear, so it can be added straight into W. After merging it takes 7.61 ms. This is within the measurement noise of other models with the same structure (such models already differ by about 0.5 ms).

Answer you can say

The adapters add small extra layers to every block, and because of the ReLU inside them, they cannot be folded into the frozen weights. So at batch size 1 they take 9.74 and 10.23 milliseconds, against 7.33 for the plain model. LoRA's update is linear, B times A, so we add it into W after training, and the model keeps its original structure and speed: 7.61 milliseconds, within normal timing noise.

Q18. Is LoRA really better than the Adapter?

Speaker: B

In accuracy, LoRA has 71.98% and the Adapter 71.54%, a gap of only 0.44. We ran each setting only once, and the holdout noise is about ± 0.3. So 0.44 is only a little larger than the noise, and we cannot say that LoRA is surely more accurate.

Each method has a real advantage:

- **LoRA**: it can be merged, so inference does not slow down;
- **Adapter**: it trains 1.6 to 1.7 times faster and uses 36% less memory.

The honest answer: their accuracy is about the same, so choose based on what your deployment needs.

Answer you can say

Only by a small margin: 71.98 against 71.54, a gap of 0.44 points, which is close to our noise level of about 0.3. So we would not claim LoRA is clearly more accurate. Its clear advantage is merging: no extra latency, 7.61 against 9.74 milliseconds. The Adapter, on the other hand, trains 1.6 to 1.7 times faster and uses 36 percent less memory.

Q19. Did you try other LoRA ranks?

Speaker: B

No. We always used rank $r = 8$, as in the brief's reference setup. We added capacity by changing the **placement** instead. The reference puts LoRA only on the qkv layer. We extended it to all four kinds of linear layers: qkv, proj, fc1 and fc2. The result:

- accuracy +0.88 (68.57 $\rightarrow$ 69.45; this experiment used the linear-probe head);
- the trained share went from 0.344% to 1.375%, still far below 5%.

The reason: two thirds of each block's weights sit in the MLP (fc1 and fc2), so changing only the attention layers is not enough.

Answer you can say

No, we kept rank 8 from the reference setup. Instead, we added capacity by placement: the reference puts LoRA only on qkv, and we added it to proj, fc1 and fc2 as well. That gave plus 0.88 points, and it is still only 1.375 percent of the backbone. Two thirds of each block's weights sit in the MLP, so adapting the MLP matters. A rank sweep would be a natural next experiment.

Q20. Why is every parameter-efficient method below full fine-tuning here?

Speaker: D

Under the same setup (3 epochs, prototype head), full fine-tuning reaches 73.38% and the best PEFT method (LoRA) 71.98%, a gap of 1.4. We think there are two possible reasons:

1. Each class has only 45 photos, but there are 450,000 photos in total. That is enough data to train all the weights.
2. The differences between 10,000 fine-grained classes are very small, so the whole network needs to change. PEFT has only limited room to change it.

Also, longer training has opposite effects on the two: full fine-tuning gains +2.21 on validation from 3 to 8 epochs, while LoRA loses 0.46 from 3 to 6 epochs. That is why the final gap is larger (validation 76.22

against 71.90).

Still, LoRA trains only 1.4% of the weights and reaches 98% of the accuracy of full fine-tuning.

Answer you can say

Under the same protocol, full fine-tuning reaches 73.38 and LoRA 71.98, so LoRA keeps 98 percent of the accuracy with 1.4 percent of the weights. We think full fine-tuning wins because 450,000 photos in total is enough data to train every weight, and 10,000 fine-grained classes need changes in every layer. The final gap is larger because longer training helped full fine-tuning but hurt LoRA.

Q21. Is your final comparison between full fine-tuning and LoRA fair?

Speaker: D

There are two kinds of comparison:

1. The **like-for-like comparison** is on slide 5. All 7 methods use the prototype head, 3 epochs, the same holdout and the same GPU. Full fine-tuning gets 73.38 and LoRA 71.98.
2. The **final results on slide 12** use each model's best setting. Full fine-tuning uses 8 epochs, all 45 photos and flip TTA. LoRA uses 3 epochs (longer training made it worse), 43 photos and no TTA. So slide 12 is not a like-for-like comparison, and the slide says so.

We should also admit one limit up front: the PEFT methods share one learning rate, 10^{-3}, which we did not tune for each method. We only checked LoRA's: halving it gave -0.12, which means no change.

Answer you can say

There are two comparisons. The like-for-like one is on slide 5: same head, 3 epochs, same holdout and GPU, and there full fine-tuning leads LoRA, 73.38 to 71.98. The final results on slide 12 give each model its best setting, for example 8 epochs for full fine-tuning, because longer training hurt LoRA. One limit we are open about: the efficient methods share one learning rate that we did not tune per method.

5.6 Reading the results

Q22. Why does the linear-probe head, the LP-FT recipe, hurt in your setting?

Speaker: C

A common recipe, **LP-FT** (Kumar et al., ICLR 2022), has two steps. First, freeze the backbone and train a linear classifier on its features (a **linear probe**). Then use this classifier as the head and fine-tune further.

In our task, it **starts high and ends low**:

- Before training: 62.1%, against 44.0% for the prototype head, 18 points ahead.
- After training: full fine-tuning 70.88 against 73.38, LoRA 69.45 against 72.00, AdaptFormer 69.05 against 71.17; with the final recipe, validation 73.10 against 76.22.

Our explanation has three steps:

1. The linear probe was trained on **the same training photos**, so it has already "memorized" them.
2. So its starting loss is very low (2.29). The signal to the backbone is weak, and it comes only from

the few photos it gets wrong. As soon as the backbone starts to change, the fitted head no longer matches the features: with LoRA, the training loss rose from 2.22 to 2.69 in the first epoch.

3. In the end, the model fits the 45 photos per class themselves, instead of learning features that generalize: a lower training loss (1.77 against 1.85), but a lower validation accuracy (73.10 against 76.22). This is overfitting, also called memorization.

Answer you can say

The linear probe is fitted on the same photos we then fine-tune on, so it starts with a low loss and gives the backbone only a weak learning signal. When the features start to move, the fitted head no longer matches them; with LoRA, the training loss even rose in the first epoch. In the end the model memorizes the 45 photos per class: a lower training loss, 1.77 against 1.85, but a lower validation accuracy, 73.10 against 76.22.

Possible follow-up questions

Follow-up: Is this proven?
Answer: It is consistent with all our loss curves and with the random-head control, but it is not a formal proof. We say that in the report.

Follow-up: Did it hurt every method?
Answer: It hurt all three methods we tested it with, full fine-tuning, LoRA and AdaptFormer, by 2.1 to 3.1 points.

Q23. The LP-FT paper says a random head distorts the pretrained features. How does that fit your random-head result?

Speaker: C

The view of the LP-FT paper: a randomly initialized head produces large, messy gradients early in fine-tuning. These gradients can bend the good pretrained features out of shape (**feature distortion**). This hurts most on data from a different domain (**out-of-distribution** data). So the paper trains a good linear head first.

Our control experiment: LoRA trained with a random head for 3 epochs reached 71.95%. That is level with the prototype head (72.00%) and 2.5 points above the linear-probe head (69.45%).

This means that in our task, changing the features is not a bad thing, because 10,000 fine-grained classes need the features to change. The real problem is that the head was fitted on the same training photos. The benefit of the prototype head is a faster start: after epoch 1 it reaches 63.3%, against 58.4% for the random head.

Answer you can say

LP-FT protects the pretrained features, mainly to keep robustness on new domains. In our task the features need to change, because the classes are new and fine-grained. And our control shows that a random head does fine here: with LoRA it ends at 71.95, level with the prototype head at 72.00, and well above the linear-probe head. So the harm comes from fitting the head on the training photos, not from changing the features.

Possible follow-up questions

Follow-up: Then why use prototypes at all, if a random head ties?
Answer: Prototypes learn faster: after one epoch they lead a random head, 63.3 against 58.4 percent, and they cost only six minutes. With a tight budget, a faster start is useful.

Q24. Is the prototype head your own idea? What is actually new in your work?

Speaker: C

The method itself is not ours. Using the average feature of each class directly as the classifier weights is called **weight imprinting** (Qi et al., CVPR 2018). It is the same idea as **prototypical networks**.

What is new is our finding. With 10,000 classes, 45 photos per class and a short training budget, the recommended LP-FT start made all three methods we tested worse, by 2.1 to 3.1 points. And a random head tied with the prototype head, which shows that the problem is a head fitted on the same photos.

So the novelty we claim is **the finding and its explanation**, not the technique itself. Being honest about this makes our claim more believable.

Answer you can say

No, the technique is known: it is weight imprinting, from Qi and colleagues at CVPR 2018. What is new is our finding: with 10,000 classes and 45 photos per class, the recommended LP-FT start hurt all three methods we tested, by 2.1 to 3.1 points, while prototypes and even a random head did better. So we claim the finding and its explanation, not the method.

Q25. Why did LoRA get worse after 3 epochs, while full fine-tuning kept improving?

Speaker: D

What we saw: LoRA trained for 6 epochs scored 0.46 lower than LoRA trained for 3 epochs (71.52 against 71.98), while its training loss fell from 1.95 to 1.70. Better on the training set but worse on the holdout: that is overfitting. Full fine-tuning is the opposite: from 3 to 8 epochs, validation went up by 2.21.

Our hypothesis (not tested yet): 87% of the weights that LoRA trains are actually the head (7.69 of 8.87 million). And the LoRA recipe treats the head very differently from full fine-tuning:

- learning rate 10^{-3}, against 10^{-4} in full fine-tuning: 10 times higher;
- weight decay 10^{-4}, against 0.05: 500 times smaller;
- no drop path, against 0.1.

So the longer the training, the more easily the head memorizes the 45 photos per class. This is the same problem as with the linear-probe head, which we explain in Part 3 of the talk (slides 6 to 8).

The test is ready (notebook 10): lower only the head's learning rate to 10^{-4}, train for 6 epochs, and see whether the score still drops.

Answer you can say

With LoRA, 6 epochs gave 71.52 against 71.98 for 3, while the training loss kept falling, so it overfits. Our hypothesis, not yet tested, is the head: 7.69 of LoRA's 8.87 million trainable weights are the head, and our LoRA recipe trains it with a ten times higher learning rate than full fine-tuning, much less weight decay and no drop path. We have prepared the test: 6 LoRA epochs with a head learning rate

of 1e-4.

Possible follow-up questions

Follow-up: What result would support your hypothesis?
Answer: If the 6-epoch run with the slower head is clearly above 71.52, by more than our noise of 0.3, then longer LoRA training no longer hurts, and the head was the cause.

Q26. Why did you train full fine-tuning for exactly 8 epochs?

Speaker: D

There are two reasons:

1. **Accuracy was still rising**: with the final recipe, going from 3 to 8 epochs raised validation from 74.01 to 76.22 (+2.21).
2. **The budget just fits**: 8 epochs took 1.03 GPU-hours, inside the 1.2-hour budget. One epoch takes about 6.5 to 7 minutes, so more epochs would go over the budget.

Answer you can say

Two reasons. Accuracy kept improving: going from 3 to 8 epochs added 2.21 points on validation, from 74.01 to 76.22. And 8 epochs fit our budget: the full run took 1.03 GPU-hours, below the 1.2-hour limit, with one epoch taking about 6.5 to 7 minutes.

Q27. Why does your final LoRA model use no test-time augmentation and only 43 photos per class?

Speaker: D

TTA (test-time augmentation): at test time, we run each photo and its left-right flipped copy through the model, and add the two scores. The result is more stable. The cost is twice the inference time: the 14.95 ms of the final full fine-tuning model is the time of two forward passes. For full fine-tuning, TTA added +0.27.

- **No TTA**: LoRA's main selling point is zero extra latency after merging, and TTA would double the latency.
- **43 photos**: the final LoRA model kept the 2 holdout photos per class, so we could check it locally (holdout 71.98 against validation 71.90).

For full fine-tuning, using all 45 photos added only 0.26, close to the noise. Each of these two changes is worth only about 0.25 points, so they would not change the order of the two models.

Answer you can say

For LoRA, the selling point is zero extra latency after merging, and flip TTA would double the inference time, so we kept it single-pass. We also kept its 2 holdout photos per class, so we could check it locally: 71.98 on the holdout against 71.90 on validation. For full fine-tuning, these two changes added only about 0.27 and 0.26 points, near our noise level.

Q28. Why are your test scores so close to your validation scores?

Speaker: D

- The validation set has 50,000 photos and the test set 100,000. Both are large, and both come from the same distribution.
- We chose models and settings mainly on our own holdout. We never tuned anything on the test set.
- Phase 2 used the same models as Phase 1.

So validation and test are very close (76.22 against 76.09, and 71.90 against 71.67, both within 0.25). This shows that the scores reflect real generalization, not a lucky fit to the validation set.

Answer you can say

Both sets are large, 50,000 and 100,000 photos, from the same distribution, and we never tuned anything on the test set. We chose our settings on our own holdout, and the test predictions come from the same Phase 1 models. So the small gap, 76.22 against 76.09 and 71.90 against 71.67, shows that the scores are real and not a lucky fit to validation.

Q29. What is your resolution result, and why is it not in your final results?

Speaker: D

After the deadline we found that simply changing the input image size of the same full fine-tuning model clearly raises the accuracy. The position embeddings are interpolated (stretched smoothly) to fit the new grid, so no parameter is added. All results below are 3-epoch runs on the holdout:

Train size / test size	Top-1	Cost
224 / 224 (baseline)	73.38%	0.48 GPU-hours, 18.9 GB
224 / 288 (no retraining)	75.50% (+2.1)	same as above
288 / 288	77.23% (+3.9)	0.68 GPU-hours, 30.2 GB
288 / 320	77.78% (+4.4)	same as above

A larger image means more tokens: 197 at 224, 325 at 288 and 401 at 320. At 288 the compute is about 1.7 times the original.

Why it is not in our final results:

1. We found it after the deadline.
2. Training the final recipe (8 epochs, all data) at 288 takes 1.61 GPU-hours, which is over the 1.2-hour budget.
3. Phase 2 had to use the same model as Phase 1, so this model has no test score.

It is a lesson, not a result.

Answer you can say

After the deadline we found that image size is a big lever. The same 3-epoch full fine-tuning model, trained at 288 pixels and tested at 320, reaches 77.78 percent on the holdout, 4.4 points above the 224-pixel baseline, in 0.68 GPU-hours, but with 30 GB of memory instead of 19. We did not add it to our results: we found it after the deadline, the full 8-epoch recipe at 288 pixels takes 1.61 GPU-hours, which is over the budget, and it has no test score.

Possible follow-up questions

Follow-up: Why does a larger test image help even without retraining?
Answer: Random cropping during training zooms in on objects, so the model gets used to seeing them large. At test time, a slightly larger image makes objects look about as large again. This is known as FixRes, from Touvron and colleagues, 2019.

Follow-up: Does the model need new weights for a larger image?
Answer: No. A larger image just gives more patches, 18 by 18 instead of 14 by 14, and we interpolate the position embeddings to the new grid. No parameter is added.

Q30. How large is the noise in your numbers? Can we trust a gain of 0.26 points?

Speaker: A

The holdout has 20,000 photos. At about 72% accuracy, the sampling error (the standard error) is about 0.3 percentage points:

Standard error

$$\mathrm{SE} = \sqrt{\frac{p(1-p)}{n}} = \sqrt{\frac{0.72 \times 0.28}{20{,}000}} \approx 0.0032 = 0.32 \text{ points}$$

p is the accuracy and n is the number of photos. It means: if you tested on another set of photos of the same size, the accuracy would move up or down by about 0.3 points.

We trained each setting only once (one random seed), with no repeats. So small gaps need care, such as the +0.26 from 2 extra photos per class, the -0.12 from halving LoRA's learning rate, and the 0.44 between LoRA and the Adapter.

Large gaps are far bigger than the noise, so they are reliable: +2.1 to 3.1 from the prototype head, and +4.4 from resolution.

Answer you can say

Each setting was trained once, and with 20,000 holdout photos at about 72 percent accuracy, the sampling error is about 0.3 points. So we treat smaller differences, like the 0.26-point gain from two extra photos per class, as noise. Our main findings are far larger: the prototype head adds 2.1 to 3.1 points, and the resolution change adds 4.4.

Possible follow-up questions

Follow-up: Why not run several seeds?
Answer: Each run costs up to an hour of our budget, so we spent it on new questions instead. Repeating the key head comparison with more seeds is on our list.

5.7 Real-world use

Q31. If you had to deploy this, would you use full fine-tuning or LoRA?

Speaker: D

It depends on the need:

- **One task, highest accuracy**: use full fine-tuning, 76.09% on test.
- **Many tasks** (for example, many customers, each with their own classes): use LoRA. All tasks share one backbone, and each task stores only a small file of 35.5 MB (30.8 MB of it is the head). Full fine-tuning would need a full 374 MB model for every task. At deployment, LoRA is merged into the weights, so it runs as fast as the original model.

Answer you can say

It depends on the use case. For one task where accuracy matters most, full fine-tuning: 76.09 percent on test. For many tasks, LoRA: one shared backbone and a small file per task, 35.5 megabytes, of which 30.8 is the head, instead of a 374-megabyte copy of the whole model. And after merging, LoRA runs at the original speed.

Q32. How did you measure latency, and why does it matter?

Speaker: B

Latency is the time it takes to process one image. We measured it on one A100 with random inputs. We timed only the model's forward pass (not loading or decoding the images), in bf16. At batch size 1 we ran 100 times and took the average, which gives the latency. At batch size 256 we ran 20 times, which gives the **throughput** (how many images per second).

The final full fine-tuning model takes 14.95 ms, because flip TTA runs two passes. Why it matters: in real use, a user is waiting for the result, and the brief asks us to report this number.

Answer you can say

We measured on one A100 with random inputs, forward pass only, in bfloat16. Latency is the average over 100 runs at batch size 1; throughput uses batch size 256. Our full model takes 14.95 milliseconds because flip TTA runs two passes; LoRA after merging takes 7.61.

Q33. What would you do next with more time or compute?

Speaker: D

1. Within the budget, trade fewer epochs for larger images (288 px), because resolution gave +4.4 on the holdout.
2. Run the prepared head-learning-rate experiment, to test our hypothesis about why LoRA overfits.
3. Tune the learning rate separately for each PEFT method.
4. Run several random seeds, to put error bars on the main findings.
5. Try other LoRA ranks, and prompt-based methods such as VPT.

Answer you can say

First, trade some epochs for larger images within the budget, since resolution gave 4.4 holdout points. Second, run our prepared test of the LoRA head learning rate. Third, tune the learning rate for each efficient method and repeat the key comparisons with more seeds. We would also like to try other LoRA ranks and prompt-based methods like VPT.

5.8 Process, budget and integrity

Q34. How do you count GPU-hours, and do your models fit the budget?

Speaker: A

The rule: each training run may use at most 2 hours $\times$ 48 GB of GPU memory $= 96$ GB·h. We use an 80 GB A100, so $96 \div 80 = 1.2$ hours (up to 1.32 hours with the allowed $\pm 10\%$).

Counted in the budget: extracting features for the head (about 6 minutes), the training, and the holdout evaluation during training. **Not counted**: downloading and unpacking the data.

The result: the final full fine-tuning run took 1.03 GPU-hours, and LoRA took 0.54 GPU-hours. Both are inside the budget.

Answer you can say

The rule is 2 hours on a 48-gigabyte GPU, which is 96 gigabyte-hours. On our 80-gigabyte A100 that is 1.2 hours per run. We count everything the GPU does for a run: extracting features for the head, about 6 minutes, the training, and the holdout evaluation during training. Our final runs took 1.03 GPU-hours for full fine-tuning and 0.54 for LoRA, both inside the budget.

Q35. How did you decide what to try next, and what did the failed ideas teach you?

Speaker: B

In each round we asked one question and changed one setting (the 8 rounds on slide 4). The failed ideas were useful too:

1. **We suspected the learning rate, but it was not the cause.** LoRA's loss rose in the first epoch, so our first suspect was a learning rate that was too high. Halving it changed nothing (-0.12). That ruled out the optimizer, so we turned to the head, and found our biggest gain (+2.5).
2. **Training LoRA longer made it worse.** The training loss fell, and so did the holdout accuracy. This showed that LoRA is limited by memorization, not by learning too little.
3. **A good start can mislead.** The linear-probe head is 18 points ahead before training, but 2.5 points behind after it. So we always judge on the holdout after training, never on the training loss or the starting accuracy.

Answer you can say

We asked one question per round and changed one setting at a time. The failed ideas pointed the way: when LoRA's loss rose, we first suspected the learning rate, but halving it changed nothing, which pointed us to the head and our biggest gain. Training LoRA longer lowered its training loss but also its accuracy, which showed memorization, not underfitting. And we learned that a good start can

mislead, so we always judged on held-out photos after training.

Q36. The top team reached about 84 percent. Why is your score lower?

Speaker: D

Do not guess what other teams did; we do not know. You can answer like this:

- The instructor's email said that the process matters more than the ranking.
- We kept the recommended backbone and the budget, and we spent our runs on understanding the adaptation methods: a like-for-like comparison of 7 methods, and finding and explaining the head problem.
- After the deadline we found that changing only the input resolution adds 4.4 points on the holdout. This is a lever we found too late.

Answer you can say

We do not know what the top team did, so we would not guess. We kept the recommended backbone and the budget, and we spent our runs on understanding the adaptation methods rather than only chasing accuracy. After the deadline we found that input resolution alone adds 4.4 points on our holdout, so that is one lever we found too late.

Q37. Which models made your test predictions, and did you use the test set in any way?

Speaker: A

The course rule: Phase 2 must use the same model as Phase 1 to predict the test set. Both our test results come from Phase 1 models:

- full fine-tuning, 1019_1319_1_full: 76.09%;
- LoRA, 1019_1319_3_lora: 71.67%.

We never used the test set to choose models or to tune settings. All our choices were made on our own holdout. The post-deadline 288 px model was evaluated only on the holdout, and it has no test score.

Answer you can say

Both test files come from our Phase 1 models, as the course rule requires: 76.09 percent for full fine-tuning and 71.67 for LoRA. We never used the test set to choose settings; all our choices were made on our own holdout. The post-deadline 288-pixel model was only evaluated on the holdout and has no test score.

5.9 The 10 questions you are most likely to get

Last look before the talk: the 10 most likely questions, with one-line answers

1. **Why ViT-B/16?** The brief recommends it; we reproduce the references within 0.3; ImageNet-21k-only versions probe worse (61.05 and 51.07 against 61.94).
2. **Why does the prototype head beat the linear-probe head?** The probe is fitted on the same photos, so it gives a weak signal and the model memorizes; prototypes add 2.1 to 3.1 points.

3. **Is the prototype head new?** No, it is weight imprinting (Qi et al., 2018); our finding about LP-FT in this setting is what is new.
4. **Why LoRA as the efficient model?** It is the most accurate PEFT method (71.98) with 1.375% of the weights, and it merges, so there is no extra latency.
5. **Why does LoRA need more memory than full fine-tuning?** Memory follows how far the gradients flow; LoRA keeps every linear layer's input, plus its own activations.
6. **Why is PEFT below full fine-tuning?** 450,000 photos are enough to train every weight, and fine-grained classes need every layer to change; LoRA still keeps 98%.
7. **Is the comparison fair?** The like-for-like test is slide 5 (73.38 against 71.98); slide 12 gives each model its best setting; one shared, untuned learning rate is a limit.
8. **Why trust the holdout?** It matches CodaBench within 0.3, and it is never used for training.
9. **What about resolution?** +4.4 on the holdout after the deadline; the 8-epoch 288 px run is over budget (1.61 h) and has no test score.
10. **Why does LoRA get worse after 3 epochs?** It overfits; we suspect its fast, weakly regularized head, and the test is prepared.

Chapter 6 Final summary

6.1 The core logic of the whole presentation

The talk tells one story: ”find a problem → make a guess about the cause → test it → draw a conclusion”. Each step answers the question left by the step before.

Slide	What this step does	One sentence and key numbers
1	Opening	7 methods, one budget; what matters most is how the head starts; test 76.09% and 71.67%
2	Analyze the problem	10,000 classes, 45 photos per class, 1.2 hours, the recommended backbone; each difficulty leads to one design choice
3	Check that our pipeline is right	all three reference baselines reproduced within 0.3 points; the 7 methods in two families
4	Show the process	8 rounds of experiments, one question per round; failed ideas ruled out wrong explanations
5	Compare the methods	where you train matters more than how much; memory depends on how far back gradients flow; LoRA can be merged
6	Find a problem	the linear-probe head starts 18 points ahead but ends 2.5 points behind
7	Fix the problem	the prototype head: 0 training steps, 44% before training, starting loss 4.39
8	Explain why	lower training loss but lower accuracy means memorization; a random head ties the prototype head
9	Test each setting	head $+3.12$; LoRA on four layers $+0.88$; LoRA for 6 epochs -0.46; full fine-tuning for 8 epochs $+2.21$
10–11	The two final models	full fine-tuning (8 epochs, layer-wise LR decay, flip TTA); LoRA (1.375%, 3 epochs, merged)
12	Final results	76.22/76.09 and 71.90/71.67; test within 0.25 of validation; both inside the budget
13	Found after the deadline	larger photos $+4.4$ (holdout only, no test score)
14	Takeaways	three findings, our limits, and the next step

The whole project in three sentences

1. With the same budget and the same backbone, we compared 7 adaptation methods. Full fine-tuning is the most accurate, LoRA is the best efficient method, and ”small changes in every block” pays off much better than ”big changes in the top blocks”.
2. The biggest gain came from how the head starts. The common linear-probe head was fit on the same training photos, so it makes the model memorize. Starting from class prototypes adds 2.1–3.1 points for all three methods we tested.

3. Our final full fine-tuning model scores 76.09% on the test set (reference: 70.8%), and our LoRA model 71.67% (reference: 68.3%). After the deadline we also found that larger input photos add another 4.4 points.

6.2 The 10 most important points

1. **Pretraining plus fine-tuning:** we do not train from scratch. We take ViT-B/16, which already knows how to look at photos, and adapt it to the new task. The backbone gives the features; the new head gives the 10,000 class scores.
2. **How ViT looks at a photo:** a 224-pixel photo is cut into 196 patches of 16 $\times$ 16 pixels. With one [CLS] token that makes 197 tokens. They pass through 12 Transformer blocks, and the 768 numbers of the [CLS] token are used for classification.
3. **The real difference between the 7 methods:** which parameters they train, and how many. The linear probe trains only the head. Partial fine-tuning trains the top K blocks. Full fine-tuning trains everything. BitFit trains only the biases. Adapter and AdaptFormer add small bottleneck modules. LoRA learns a low-rank update $\Delta W = BA$.
4. **Where you train matters more than how much:** LoRA at 1.38% $\approx$ partial fine-tuning at 33%; BitFit at 0.12% $>$ partial fine-tuning at 8%.
5. **LoRA's strengths:** few parameters, and it can be merged (no slower at inference). **Weaknesses:** it does not save training memory (19.9 GB), and it trains the slowest.
6. **Why the linear-probe head hurts:** it was fit on the same training photos $\rightarrow$ low starting loss and a weak learning signal $\rightarrow$ once the features change, it no longer fits them $\rightarrow$ the model memorizes.
7. **Why the prototype head works:** each class weight is the average direction of that class's features $\times$ 40. It takes 0 training steps and memorizes nothing, yet scores 44% before training. The random-head control shows that the harm comes from "fitting the head on the training photos".
8. **Judge with the holdout, not the training loss:** the holdout and CodaBench agree within 0.3 points; differences below about 0.3 points are noise; each setting was run once.
9. **Every setting in the recipe has a reason:** full fine-tuning for 8 epochs (still improving, inside the budget), layer-wise learning-rate decay, label smoothing, drop path; LoRA for 3 epochs (6 epochs overfit) on all four linear layers ($+0.88$).
10. **The resolution finding after the deadline:** train at 288, test at 320: $+4.4$ on the holdout. The reason is FixRes (random crops make objects look bigger during training). It has only a holdout score, no test score.

Term	In one sentence

6.3 Terms you must understand

These words will almost surely come up in the talk and the questions. Chapter 3 has the full glossary.

Term	In one sentence
backbone / head	the main body of the model (turns a photo into features) / the last classification layer (turns features into 10,000 scores)
pretraining / fine-tuning	learning first on a big dataset / continuing to learn on a new task
PEFT	parameter-efficient fine-tuning: training only a small share of the parameters
full fine-tuning	training every parameter
linear probe	freezing the backbone and training only the head
LP-FT	the popular recipe: first train a linear-probe head, then fine-tune the whole model
prototype / imprinting	the average direction of a class's features / copying the prototypes straight into the head's weights
LoRA, rank r, α	learns a low-rank update $\Delta W = BA$; r is the small middle size (8); the update is multiplied by $\alpha/r = 2$
merge	after training, add LoRA's update back into the original weights, so inference is no slower
Adapter / AdaptFormer	small bottleneck modules in series / in parallel; they cannot be merged
BitFit	training only the biases
token / patch / [CLS]	the vector made from one photo piece / a 16×16 piece of the photo / the special token that sums up the whole photo
self-attention	lets each patch use information from all the other patches
logits / softmax	the raw scores the model outputs / turns the scores into probabilities that add up to 1
cross-entropy / label smoothing	the loss that measures how low the probability of the right answer is / do not ask for 100% certainty; the target for the right class is about 0.9
overfitting / memorization	good on the training data, worse on new data / learning the training photos by heart
holdout / validation / test	our own practice exam / the CodaBench monthly test / the final exam
top-1 accuracy	the share of photos where the first answer is right
epoch / step / batch	one pass over all the data / one parameter update / how many photos per update (256)
learning rate / warmup / cosine	how big each step is / start with small steps and grow / later shrink along a cosine curve
weight decay / drop path	pull the parameters a little toward 0 / randomly skip some blocks during training; both fight overfitting
layer-wise LR decay	layers closer to the input get a smaller learning rate ($\times$ 0.75 per layer)

Term	In one sentence
flip TTA	at test time, predict on the photo and on its mirror image, then combine
GPU-hours / latency / throughput	how many GPU hours were used / milliseconds per photo / photos per second
FixRes	random crops make objects look bigger in training, so larger test photos match better
ablation	change one setting at a time and measure its effect alone

6.4 The core difference between each model and method

Method	What it trains	Share / holdout	Core difference (one sentence)
Linear probe	the head only	0% / 61.94%	the features never change; fastest and cheapest, but a low ceiling
Partial fine-tuning, K = 4	the last 4 blocks	33% / 71.88%	changes only the top; the bottom never adapts; the least training memory (7.0 GB)
Full fine-tuning	every parameter	100% / 73.38%	the most accurate; every task stores a full copy of the model (374 MB)
BitFit	the biases only	0.12% / 69.81%	the fewest parameters, no slower at inference, but too little capacity
Adapter	a bottleneck in series	1.39% / 71.54%	sits on the main path after the MLP; 2.4 ms slower at inference
AdaptFormer	a bottleneck side branch	1.39% / 71.17%	runs beside the MLP, output $\times$ 0.1; about 2.9 ms slower
LoRA	low-rank matrices A and B	1.38% / 71.98%	the most accurate efficient method; can be merged, so no slower; the most training memory
Random head	(a way to start the head)	–	knows nothing; starting loss 9.28; ends level with the prototype head (LoRA)
Linear-probe head	(a way to start the head)	–	62.1% before training, but fit on the training photos; ends 2.1–3.1 points lower
Prototype head	(a way to start the head)	–	44.0% before training, 0 training steps, memorizes nothing; our choice

6.5 The technical questions the instructor is most likely to ask

Chapter 5 has the full answers. Here are the one-sentence versions:

1. **Why does the prototype head beat the linear-probe head?** The linear-probe head was fit on the same training photos, so the learning signal is weak and the model memorizes. The prototype head was not fit to any photo. Evidence: lower training loss but lower validation accuracy (1.77 vs. 1.85; 73.10 vs. 76.22), and a random head ties the prototype head.
2. **Did you invent the prototype head?** No. It is called weight imprinting (Qi et al., 2018). Our new finding is that, with many classes and few photos per class, it beats the recommended LP-FT start.
3. **Why not use a bigger or newer backbone?** The course recommends this one. We checked the versions trained only on ImageNet-21k, and their linear probes were worse (61.05% and 51.07% vs. 61.94%). Also, the budget is only 1.2 hours.
4. **Why does LoRA use even more memory than full fine-tuning?** Like full fine-tuning, it must keep the input of every linear layer to compute gradients, and it also keeps the results of 96 extra small matrix multiplications. The extra optimizer memory of full fine-tuning is only about 1 GB.
5. **Why is LoRA no slower at inference, while the Adapter is slower?** LoRA's update is linear, so it can be added back into the original weights. The Adapter is an extra nonlinear layer that must be computed separately.
6. **Why is PEFT below full fine-tuning?** 450,000 photos in total is actually a lot, enough for full fine-tuning to shine, and 10,000 fine-grained classes need large changes in every layer. Under the same settings the gap is 1.4 points (73.38 vs. 71.98).
7. **Why did LoRA get worse after 6 epochs?** The training loss went down while the holdout went down too: that is overfitting. Our guess (not yet tested): 87% of what LoRA trains is the head, and the head gets 10 times the learning rate of full fine-tuning and almost no weight decay.
8. **Are your conclusions reliable?** Each setting was run once, and differences below about 0.3 points are noise. But the main findings (the head's $+2.1$ to $+3.1$, the big gaps between methods) are far larger than the noise. The holdout agrees with CodaBench, and the test agrees with the validation set.
9. **Did you stay inside the budget?** Yes. The two final models used 1.03 and 0.54 GPU-hours, both under 1.2. The timing includes feature extraction, training and the evaluations during training.
10. **Why don't you use the resolution result?** We found it after the deadline; it has only a holdout score; the 8-epoch version needs 1.61 hours, over the budget; and the Phase 2 rule requires the same model as in Phase 1.

6.6 What to review before the presentation

6.6.1 Everyone must know

- The three-sentence summary in Section 6.1. Practice saying it out loud.
- The core numbers in Appendix A: 76.22 / 76.09, 71.90 / 71.67, 70.8, 68.3, 73.38 / 71.98, 1.2 hours, 1.375%, +2.1 to +3.1, +4.4.
- The three kinds of scores: the holdout (our own 20,000 photos), the validation set (CodaBench), and the test set (the final ranking). When you say a number, say which one it is.
- The two kinds of ”linear probe”: an adaptation method (slide 16), and a way to start the head (slide 6).

6.6.2 By speaker

Speaker	Focus on
A (slides 1–3)	how the budget is computed ($96/80 = 1.2$); why we kept the backbone (recommended, the 21k versions are worse, the budget); how the holdout was built and why we trust it; the three baseline numbers; one sentence for each of the 7 methods
B (slides 4–6)	the chain of logic across the 8 rounds; how to read the plot on slide 5; why memory depends on how far back gradients flow; LoRA can be merged, adapters cannot; what LP-FT is; how to count LoRA’s parameters (backup slides 16–20 are yours too)
C (slides 7–9)	the four steps of the prototype head; what cosine similarity and the temperature 40 mean; the three starting losses 4.39 / 2.29 / 9.28; the three-step explanation of ”lower training loss but lower accuracy”; the random-head control; the reason behind each row on slide 9
D (slides 10–15)	every setting of the two final models (loss, learning rate, epochs, parameters, GPU-hours, memory, latency); the fairness note on slide 12; why the resolution result does not count; the limits and next step on slide 14; who answers which questions

6.6.3 Words that are easy to say wrong

- LoRA: say ”LOR-uh” (stress on the first part), not letter by letter.
- ViT: ”vit” or ”V-I-T”; the team should pick one.
- AdamW: ”Adam W”. GELU: ”gee-loo”. bf16: ”B-F-sixteen”.
- CodaBench: ”co-da-bench”. FixRes: ”fix-res”.

6.6.4 Final reminders

Easy to get wrong

- The cover and the last slide still show the placeholders [Name A]–[Name D]. Replace them with your real names before the talk.
- The 288-pixel model has **no test score**. Do not present it as an official result.

- Do not guess what the top team on the leaderboard did. You can say we focused on the process, and that we found one lever too late (resolution).
- If you do not know, say so: ”We did not test that, but our guess is …”

Appendix A Key numbers at a glance

All numbers come from the slides, the speaker notes and the report. Each one says which kind of score it is: the holdout (our 20,000 held-out photos), the validation set (CodaBench val), or the test set (CodaBench test).

A.1 Task, data and budget

Item	Number
Number of classes	10,000 (fine-grained, class names hidden)
Training set	450,000 photos, 45 per class
Validation set / test set	50,000 photos (5 per class) / 100,000 photos (10 per class); labels hidden
Holdout	2 photos per class, 20,000 in total; holdout runs train on 43 photos per class
Holdout vs. CodaBench	72.31 vs. 72.57 (full fine-tuning); 71.98 vs. 71.90 (LoRA)
Noise	about ± 0.3 points (for 20,000 photos at 72% accuracy, the standard error is about 0.32 points); each setting was run once
Budget	2 hours $\times$ 48 GB $=$ 96 GB·h; 1.2 hours on an A100-80GB (1.32 hours with the extra 10%)

A.2 The model (ViT-B/16)

Item	Number
Version	`vit_base_patch16_224.augreg2_in21k_ft_in1k` (pretrained on ImageNet-21k $\rightarrow$ fine-tuned on ImageNet-1k)
Backbone parameters	85,798,656 (about 85.8 million)
Structure	12 blocks of 7,087,872 parameters each; width 768; 12 attention heads $\times$ 64; MLP 768 $\rightarrow$ 3072 $\rightarrow$ 768
Input	224 $\times$ 224; patches of 16 $\times$ 16 $\rightarrow$ 196 patches + [CLS] $=$ 197 tokens
New head	768 $\rightarrow$ 10,000, 7,690,000 parameters (9.0% of the backbone)
21k-only versions (linear probe)	augreg_in21k 61.05%, orig_in21k 51.07%, ours 61.94%

A.3 Reference baselines (holdout)

Baseline	Reference	Ours
Zero-shot (prototype head, no training)	43.7%	43.95%
Full fine-tuning, 3 epochs (linear-probe head)	70.8%	70.88%
LoRA r = 8 on qkv only (294,912 parameters, 0.34%), 3 epochs (linear-probe head)	68.3%	68.57%

The baselines were reproduced with the linear-probe head. With the prototype head, the same 3-epoch full fine-tuning scores 73.38% (slide 5).

A.4 The seven methods (prototype head, 3 epochs, holdout)

Method	Share / parameters	Holdout	Training memory	Latency (batch 1)
Linear probe	0%	61.94%	3.2 GB	6.96 ms
Partial fine-tuning, K = 4	33%	71.88%	7.0 GB	7.41 ms
Full fine-tuning	100% (85.8M)	73.38%	18.9 GB	7.78 ms
BitFit	0.12% (102,912)	69.81%	12.7 GB	7.33 ms
Adapter	1.39% (1,189,632)	71.54%	12.8 GB	9.74 ms
AdaptFormer	1.39% (1,189,632)	71.17%	12.8 GB	10.23 ms
LoRA (four linear layers)	1.38% (1,179,648)	71.98%	19.9 GB	7.61 ms

All partial fine-tuning results: K = 1, 2, 4, 6 train 8.26%, 16.5%, 33% and 49.6% of the parameters; holdout 67.97%, 70.99%, 71.88% and 72.89%; training memory 3.8–10.0 GB.

A.5 How the head starts

Setting	Linear-probe head	Prototype head	Random head
Before training (head only)	62.1%	44.0%	about 0
Starting training loss (mean of the first 100 LoRA steps)	2.29	4.39	9.28
Full fine-tuning, 3 epochs (holdout)	70.88%	73.38%	–
LoRA, 3 epochs (holdout)	69.45%	72.00%	71.95%
AdaptFormer, 3 epochs (holdout)	69.05%	71.17%	–
Full fine-tuning, final recipe (validation)	73.10%	76.22%	–
Final training loss, final recipe	1.77	1.85	–

With the linear-probe head, LoRA's loss rose from 2.22 to 2.69 during the first epoch. After the first epoch, the prototype head led the random head: 63.3% vs. 58.4%.

A.6 Ablations (one change at a time)

Change	Before → after	Effect
Head: linear probe → prototype (full fine-tuning, 8 epochs, validation)	73.10 → 76.22	+3.12
Half the LoRA learning rate (holdout)	68.57 → 68.45	−0.12
LoRA: qkv only → four linear layers (holdout)	68.57 → 69.45	+0.88
LoRA: 3 → 6 epochs (holdout; training loss 1.95 → 1.70)	71.98 → 71.52	−0.46
Full fine-tuning: 3 → 8 epochs (validation)	74.01 → 76.22	+2.21
Flip TTA (validation)	72.57 → 72.84	+0.27
43 → 45 photos per class (validation)	72.84 → 73.10	+0.26

A.7 The two final models

Item	Full fine-tuning (1019_1319_1_full)	LoRA (1019_1319_3_lora)
What is trained	all 85.8M + the head	LoRA 1,179,648 (1.375%) + the head
Data	45 photos per class	43 photos per class
Loss	cross-entropy + label smoothing 0.1	the same
Optimizer	AdamW, weight decay 0.05, gradient clipping 1.0	AdamW, weight decay 10^{-4}, gradient clipping 1.0
Learning rate	10^{-4}, $\times$ 0.75 per layer toward the input	10^{-3}, no layer-wise decay
Other	drop path 0.1, bf16, batch 256	no drop path, bf16, batch 256, r = 8, α = 16
Epochs / steps	8 epochs (1 warmup + cosine) / 14,056 steps	3 epochs (0.3 warmup + cosine) / 5,037 steps
GPU-hours / memory	1.03 / 18.9 GB	0.54 / 19.9 GB
Inference	flip TTA: 14.95 ms, 1,919 photos per second	merged: 7.61 ms, 3,869 photos per second
Validation / test	76.22% / **76.09%** (reference 70.8%, +5.4)	71.90% / **71.67%** (reference 68.3%, +3.6)

A.8 Resolution (after the deadline, holdout, 3 epochs)

Setting (train / test pixels)	Tokens	Holdout	GPU-hours	Memory
224 / 224	197	73.38%	0.48	18.9 GB
224 / 288	325	75.50% (+2.1)	0.48	18.9 GB
288 / 288	325	77.23% (+3.9)	0.68	30.2 GB
288 / 320	401	77.78% (+4.4)	0.68	30.2 GB
Final recipe, 8 epochs, at 288 / 320	401	no test score	1.61 (over budget)	30.2 GB